



Università degli Studi di Padova

Laurea: Informatica

Corso: Ingegneria del Software

Anno Accademico: 2025/26



Gruppo 17

Nome: BitByBit

Email: swe.bitbybit@gmail.com

Specifica Tecnica



Stato:	In lavorazione
Versione:	0.8.0
Responsabile:	Gabriele Scaggiante
Redattori:	Gabriele Scaggiante Dennis Parolin Ferdinando Fracasso Giovanni Visentin Marco Sanguin Riccardo Manisi
Verificatori:	Riccardo Manisi Dennis Parolin Marco Sanguin Giovanni Visentin Gabriele Scaggiante Ferdinando Fracasso
Destinatari:	Miriade srl Prof. Tullio Vardanega Prof. Riccardo Cardin Gruppo BitByBit



Registro delle modifiche

Versione	Data	Autore	Descrizione	Verificatore
0.9.0	2026-05-04	Marco Sanguin	Aggiornamento dell' Architettura ^G frontend con corrispettivi UML	Riccardo Manisi
0.8.0	2026-05-02	Giovanni Visentin	Sistemazione della tabella dei requisiti	Dennis Parolin
0.7.1	2026-05-07	Ferdinando Fracasso	Aggiornato sezione Librerie	Riccardo Manisi
0.7.0	2026-04-29	Riccardo Manisi	Redatta sezione Architettura classi microservizio contatti fidati , Architettura classi microservizio luoghi sicuri	Gabriele Scaggiante
0.6.1	2026-04-27	Gabriele Scaggiante	Aggiunte librerie nella sezione Librerie	Ferdinando Fracasso
0.6.0	2026-04-26	Gabriele Scaggiante	Completata la sezione Architettura chatbot introducendo la sezione Architettura logica chatbot e aggiornando la sezione dedicata alle API	Dennis Parolin
0.5.3	2026-04-24	Ferdinando Fracasso	Aggiornato la sezione Architettura funzionalità diari a seguito dell'implementazione della funzionalità	Gabriele Scaggiante
0.5.2	2026-04-23	Gabriele Scaggiante	Corretta la sezione Architettura chatbot dopo l'introduzione di due tabelle DynamoDB	Giovanni Visentin



0.5.1	2026-04-20	Dennis Parolin	Aggiornati i pattern presenti nella la sezione Design pattern utilizzati , aggiunti i pattern Dependency Injection , Command e Adapter .	Gabriele Scaggiante
0.5.0	2026-04-18	Gabriele Scaggiante	Nella sezione Architettura backend , aggiornata Architettura^G Auth, Luoghi Sicuri e Materiale Informativo; redatte sezioni Diario, Chatbot e DMS.	Dennis Parolin
0.4.0	2026-04-17	Riccardo Manisi	Redatta la sezione Architettura InvioSOS	Gabriele Scaggiante
0.3.0	2026-04-17	Riccardo Manisi	Redatta la sezione Architettura auth	Ferdinando Fracasso
0.2.1	2026-04-12	Marco Sanguin	Redatta la sezione Architettura luoghi sicuri	Riccardo Manisi
0.2.0	2026-04-12	Marco Sanguin	Redatta la sezione Architettura funzionalità Dead man's switch	Ferdinando Fracasso
0.1.9	2026-04-12	Dennis Parolin	Creata la sezione Architettura Back End con relative sottosezioni Autenticazione , Luoghi sicuri e Materiale informativo	Giovanni Visentin
0.1.8	2026-04-11	Marco Sanguin	Aggiunta la sezione Stato dei requisiti funzionali	Ferdinando Fracasso



0.1.7	2026-04-10	Marco Sanguin	Redatta la sezione Architettura Area Informativa	Ferdinando Fracasso
0.1.6	2026-04-09	Ferdinando Fracasso	Redatta la sezione Design pattern utilizzati	Giovanni Visentin
0.1.5	2026-04-06	Ferdinando Fracasso	Redatta la sezione Architettura funzionalità diari	Marco Sanguin
0.1.4	2026-04-05	Giovanni Visentin	Aggiunta della funzionalità dei contatti fidati	Dennis Parolin
0.1.3	2026-04-04	Gabriele Scaggiante	Iniziata la sezione Architettura frontend , completata Architettura chatbot	Marco Sanguin
0.1.2	2026-04-02	Dennis Parolin	Redatta la sezione Architettura logica e Architettura di deployment	Marco Sanguin
0.1.1	2026-03-31	Gabriele Scaggiante	Aggiunta di EventBridge, SAM, CloudFormation e Docker nella sezione tecnologie.	Dennis Parolin
0.1.0	2026-03-30	Gabriele Scaggiante	Creazione del documento. Scrittura dell'introduzione e delle tecnologie utilizzate.	Riccardo Manisi



Indice

1	Introduzione	14
1.1	Scopo del documento	14
1.2	Riferimenti	14
1.2.1	Riferimenti normativi	15
1.2.2	Riferimenti informativi	15
2	Tecnologie	16
2.1	Framework	16
2.1.1	Flutter	16
2.2	Linguaggi di programmazione	16
2.2.1	Dart	16
2.2.2	Python	17
2.3	Servizi AWS	17
2.3.1	Lambda	17
2.3.2	API Gateway	17
2.3.3	DynamoDB	17
2.3.4	S3	18
2.3.5	Cognito	18
2.3.6	Bedrock	18
2.3.7	SES	18
2.3.8	CloudWatch	18
2.3.9	EventBridge	19
2.3.10	SAM	19
2.3.11	CloudFormation	19
2.4	Librerie	19
2.4.1	http	19
2.4.2	image_picker	20
2.4.3	path_provider	20
2.4.4	provider	20
2.4.5	url_launcher	20
2.4.6	flutter_dotenv	21
2.4.7	flutter_web_auth_2	21
2.4.8	flutter_map	21
2.4.9	geolocator	21
2.4.10	get_it	22
2.4.11	latlong2	22
2.4.12	path	22
2.4.13	flutter_secure_storage	22



2.4.14	just_audio	22
2.4.15	file_picker	23
2.4.16	url_launcher_platform_interface	23
2.4.17	command_it	23
2.4.18	listen_it	23
2.4.19	mocktail	23
2.4.20	shared_preferences	24
2.4.21	connectivity_plus	24
2.4.22	audio_session	24
2.4.23	python-ulid	24
2.4.24	rank-bm25	24
2.4.25	boto3	25
2.5	Tecnologie di analisi statica	25
2.5.1	SonarQube	25
2.6	Tecnologie di analisi dinamica	25
2.6.1	Pytest	25
2.6.2	Moto	25
2.7	Altro	26
2.7.1	Ollama	26
2.7.2	Docker	26
3	Architettura	26
3.1	Architettura logica	26
3.1.1	Livello Client (Frontend Mobile)	27
3.1.2	Livello di Servizio (Backend)	27
3.2	Design pattern utilizzati	28
3.2.1	Model-view-viewmodel	28
3.2.2	Command	30
3.2.3	Observer	31
3.2.4	Proxy	33
3.2.5	Singleton	34
3.2.6	Dependency Injection e Service Locator	35
3.2.7	Adapter	36
3.3	Architettura di deployment	37
3.3.1	Client Mobile	37
3.3.2	Infrastruttura AWS (Serverless Cloud)	37
3.3.3	Orchestrazione Event-Driven	38
3.3.4	Infrastructure as Code (IaC)	38
3.4	Architettura Front End	39
3.4.1	Autenticazione	39
3.4.2	Chatbot	42

3.4.3	Diari	50
3.4.4	Contatti Fidati	61
3.4.5	Luoghi Sicuri	65
3.4.6	Dead Man's Switch	69
3.4.7	Materiale Informativo	73
3.4.8	Main App, Home e Sistema SOS	77
3.5	Architettura Back End	81
3.5.1	Autenticazione	81
3.5.2	Luoghi sicuri	82
3.5.3	Materiale informativo	88
3.5.4	Diario criptato e fittizio	93
3.5.5	Chatbot	95
3.5.6	Contatti fidati, invio SOS e Dead man's switch	112
4	Stato dei requisiti funzionali	128
4.1	Requisiti Funzionali obbligatori	128
4.2	Requisiti Funzionali desiderabili	140
4.3	Requisiti Funzionali opzionali	142



Elenco delle tabelle

2	Tabella dello stato dei requisiti funzionali obbligatori	140
3	Tabella Requisiti Funzionali Desiderabili	142
4	Tabella dello stato dei requisiti funzionali opzionali	143

Elenco delle figure

1	Rappresentazione del pattern MVVM nel modulo del materiale informativo	28
2	Rappresentazione del pattern Command	30
3	Rappresentazione del pattern Observer applicato nel modulo dell'autenticazione	31
4	Rappresentazione del pattern Proxy applicato alla gestione delle chat . . .	33
5	Rappresentazione del pattern Singleton applicato alla sessione del diario . .	34
6	Rappresentazione del pattern Dependency Injection nel modulo di Autenticazione	35
7	Rappresentazione del pattern Adapter all'ApiClient	36
8	Diagramma delle classi relativo all'autenticazione	39
9	Diagramma della classe LoginScreen	40
10	Diagramma della classe User	40
11	Diagramma della classe UserDTO	41
12	Diagramma della classe AuthViewModel	41
13	Diagramma della classe AuthRepository	42
14	Diagramma della classe AuthService	42
15	Diagramma delle classi relativo alle funzionalità del Chatbot	43
16	Diagramma della classe ChatbotScreen	43
17	Diagramma della classe ChatWidget	44
18	Diagramma della classe ChatHistoryWidget	44
19	Diagramma della classe ChatbotModeToggleWidget	44
20	Diagramma della classe ChatbotSendMessageWidget	45
21	Diagramma della classe ChatbotCreateChatWidget	45
22	Diagramma della classe ChatbotViewModel	46
23	Diagramma della classe Chat	46
24	Diagramma della classe ProxyChat	47
25	Diagramma della classe LocalChat	47
26	Diagramma della classe ChatMessage	48
27	Diagramma della classe ChatMode	48
28	Diagramma della classe MessageResponse	48
29	Diagramma della classe ChatbotRepository	49
30	Diagramma della classe ChatbotService	49



31	Diagramma dell'interfaccia ApiClient	50
32	Diagramma della classe HttpClient	50
33	Diagramma della classe ChatDTO	50
34	Diagramma delle classi relativo alle funzionalità dei Diari	51
35	Diagramma della classe DiaryAccessScreen	51
36	Diagramma della classe PasswordFormWidget	52
37	Diagramma della classe DiaryAccessViewModel	52
38	Diagramma della classe DiaryAccountRepository	52
39	Diagramma della classe DiaryAccountService	53
40	Diagramma della classe DiaryAccessResult	53
41	Diagramma della classe DiaryScreen	53
42	Diagramma della classe NoteListWidget	54
43	Diagramma della classe NoteActionsWidget	54
44	Diagramma della classe NoteEditorWidget	54
45	Diagramma della classe DiaryViewModel	55
46	Diagramma della classe DiarySession	55
47	Diagramma della classe DiaryType	56
48	Diagramma della classe Note	56
49	Diagramma della classe ProxyNote	57
50	Diagramma della classe LocalNote	57
51	Diagramma della classe NoteElement	58
52	Diagramma della classe NoteTextElement	58
53	Diagramma della classe NoteImageElement	58
54	Diagramma della classe NoteAudioElement	59
55	Diagramma della classe NoteRepository	59
56	Diagramma della classe NoteService	60
57	Diagramma della classe NoteDTO	60
58	Diagramma della classe NoteElementDTO	60
59	Diagramma delle classi relativo alle funzionalità dei Contatti Fidati	61
60	Diagramma della classe TrustedContactScreen	61
61	Diagramma della classe TrustedContactListWidget	62
62	Diagramma della classe TrustedContactFormWidget	62
63	Diagramma della classe TrustedContactActionsWidget	62
64	Diagramma della classe TrustedContactViewModel	63
65	Diagramma della classe TrustedContact	63
66	Diagramma della classe TrustedContactRepository	64
67	Diagramma della classe TrustedContactService	64
68	Diagramma della classe TrustedContactDTO	64
69	Diagramma delle classi complessivo della funzionalità Luoghi Sicuri	65
70	Diagramma della classe SafePlaceMapScreen	66



71	Diagramma della classe SafePlaceMapWidget	66
72	Diagramma della classe SafePlaceViewModel	67
73	Diagramma della classe SafePlace	67
74	Diagramma della classe SafePlaceCategory	68
75	Diagramma della classe SafePlaceRepository	68
76	Diagramma della classe LocationService	69
77	Diagramma della classe SafePlaceService	69
78	Diagramma della classe SafePlaceDTO	69
79	Diagramma delle classi relativo alla funzionalità del Dead Man's Switch . .	70
80	Diagramma della classe SettingsScreen	70
81	Diagramma della classe DeadManFormWidget	71
82	Diagramma della classe DeadManViewModel	71
83	Diagramma della classe DeadManSettings	72
84	Diagramma della classe DeadManRepository	72
85	Diagramma della classe DeadManService	73
86	Diagramma della classe DeadManSettingsDTO	73
87	Diagramma delle classi complessivo della funzionalità Materiale Informativo	74
88	Diagramma della classe MaterialScreen	74
89	Diagramma della classe MaterialListWidget	75
90	Diagramma della classe MaterialViewModel	75
91	Diagramma della classe Resource	76
92	Diagramma della classe ResourceType	76
93	Diagramma della classe MaterialRepository	77
94	Diagramma della classe MaterialService	77
95	Diagramma della classe ResourceDTO	77
96	Diagramma delle classi complessivo di Main App, Home e SOS	78
97	Diagramma della classe MainAppWidget	78
98	Diagramma della classe HomeScreen	79
99	Diagramma della classe HomeTabView	79
100	Diagramma della classe HomeDashboardWidget	79
101	Diagramma della classe HomeViewModel	80
102	Diagramma della classe SosPullTopWidget	80
103	Diagramma della classe SosViewModel	80
104	Architettura e Flusso di Autenticazione	81
105	Architettura moduli Luoghi Sicuri	82
106	Componenti del microservizio luoghi sicuri	84
107	Diagramma della classe Marker	85
108	Diagramma della classe SafePlacesController	86
109	Diagramma dell'interfaccia GetMarkerPort	86
110	Diagramma della classe SafePlacesService	87



111	Diagramma dell'interfaccia SafePlacesRepositoryPort	87
112	Diagramma della classe S3SafePlacesRepository	87
113	Architettura moduli Materiale Informativo	88
114	Componenti del microservizio materiale informativo	89
115	Diagramma della classe Resource	90
116	Diagramma della classe ResourcesController	91
117	Diagramma della classe GetResourcesPort	91
118	Diagramma della classe ResourcesService	92
119	Diagramma della classe ResourcesRepositoryPort	92
120	Diagramma della classe S3ResourcesRepository	92
121	Architettura diari	93
122	Architettura chatbot	95
123	Componenti del microservizio Chatbot	102
124	Diagramma della classe Chat	102
125	Diagramma della classe ChatMessage	103
126	Diagramma dell'interfaccia ChatbotLLMPort	103
127	Diagramma della classe BedrockLLMAdapter	104
128	Diagramma della classe ChatbotLLMService	104
129	Diagramma dell'interfaccia ChatsRepositoryPort	105
130	Diagramma della classe DynamoChatAdapter	105
131	Diagramma dell'interfaccia CreateChatPort	106
132	Diagramma dell'interfaccia GetChatPort	106
133	Diagramma dell'interfaccia UpdateChatPort	106
134	Diagramma dell'interfaccia DeleteChatPort	107
135	Diagramma dell'interfaccia ChatMessagePort	107
136	Diagramma della classe ChatbotCRUDService	108
137	Diagramma della classe CreateChatCmd	108
138	Diagramma della classe GetChatCmd	108
139	Diagramma della classe GetChatListCmd	109
140	Diagramma della classe UpdateChatCmd	109
141	Diagramma della classe DeleteChatCmd	109
142	Diagramma della classe AddChatMessageCmd	110
143	Diagramma della classe ChatDTO	110
144	Diagramma della classe ChatMessageDTO	111
145	Diagramma della classe LambdaHandler	111
146	Architettura del sistema di contatti fidati, SOS e Dead man's switch	112
147	Componenti del microservizio contatti fidati, SOS e Dead man's switch	114
148	Diagramma della classe TrustedContact	115
149	Diagramma della classe TrustedContactDTO	115
150	Diagramma della classe AddTrustedContactCmd	116



151	Diagramma della classe GetTrustedContactCmd	116
152	Diagramma della classe DeleteTrustedContactCmd	117
153	Diagramma della classe DmsConfigurationSettings	117
154	Diagramma della classe DmsConfigurationSettingsDTO	118
155	Diagramma della classe AlertCmd	118
156	Diagramma della classe EmailMessage	119
157	Diagramma della classe TrustedContactController	119
158	Diagramma della classe SetDmsHeartBeatPort	120
159	Diagramma della classe GetDmsConfigurationSettingsPort	120
160	Diagramma della classe SetDmsConfigurationSettingsPort	120
161	Diagramma della classe UpdateDmsAlertPort	121
162	Diagramma della classe SetTrustedContactPort	121
163	Diagramma della classe GetTrustedContactPort	122
164	Diagramma della classe DeleteTrustedContactPort	122
165	Diagramma della classe SendAlertPort	122
166	Diagramma della classe DmsCRUDService	123
167	Diagramma della classe DmsAlertService	123
168	Diagramma della classe TrustedContactCRUDService	124
169	Diagramma della classe SOSAlertService	124
170	Diagramma della classe DmsRepositoryPort	125
171	Diagramma della classe TrustedContactRepositoryPort	126
172	Diagramma della classe NotificationPort	126
173	Diagramma della classe DynamoDmsAdapter	127
174	Diagramma della classe DynamoTrustedContactAdapter	127
175	Diagramma della classe SesNotificationAdapter	127



1. Introduzione

1.1. Scopo del documento

Il presente documento ha lo scopo di fornire una descrizione completa, formale e strutturata del prodotto software, delineandone in modo rigoroso le caratteristiche tecniche e architetturali.

La **Specifica Tecnica^G** rappresenta il riferimento principale per tutte le attività di progettazione, sviluppo, **Verifica^G** e manutenzione del sistema, assicurando coerenza rispetto ai requisiti definiti nel documento di **Analisi dei requisiti^G** e alle scelte progettuali adottate.

In particolare, il documento si propone di:

- Definire l'**Architettura^G** complessiva del sistema, descrivendo le componenti software, le loro responsabilità e le modalità di interazione attraverso appositi diagrammi
- Specificare i moduli funzionali e le interfacce esposte, includendo i contratti di comunicazione e i formati dei dati scambiati
- Descrivere l'**Architettura^G** di **Deployment^G**, evidenziando la distribuzione delle componenti nei diversi ambienti di esecuzione e le relative dipendenze infrastrutturali
- Documentare le tecnologie, i linguaggi di programmazione, i **Framework^G** e gli strumenti adottati, motivando le scelte effettuate in relazione ai requisiti di progetto e al **Capitolato^G**
- Illustrare i principali design pattern e le soluzioni architetturali utilizzate, con particolare attenzione agli aspetti di scalabilità, manutenibilità e sicurezza
- Fornire indicazioni sui metodi di testing e di **Validazione^G**
- Supportare le attività di evoluzione e manutenzione del software, garantendo una base documentale chiara e coerente.

1.2. Riferimenti

Al fine di evitare ambiguità relative al linguaggio e ai termini utilizzati all'interno dei documenti formali, viene allegato il [Glossario](#). I termini tecnici, gli acronimi e le parole con significato specifico all'interno del progetto sono contrassegnati da una lettera 'G' in apice (es. **Agile^G**) e sono descritti in dettaglio nel documento apposito.



1.2.1 Riferimenti normativi

- **Capitolato^G d'appalto C4: *L'app che Protegge e Trasforma***
 Parti consultate: documento completo
 Versione: 1.0
 Ultimo accesso: 2026-01-14
<https://www.math.unipd.it/~tullio/IS-1/2025/Progetto/C4.pdf>
- **Norme di Progetto**
 Parti consultate: documento completo
 Versione: v.1.0.0
 Ultimo accesso: 2026-02-09
https://swe-bitbybit.github.io/SWE-docs/docs/RTB/documenti_interni/norme_di_progetto.pdf

1.2.2 Riferimenti informativi

- **Slide sulla Progettazione Software del Prof. Vardanega**
 Parti consultate: documento completo
 Versione: A.A.2025/2026
 Ultimo accesso: 2026-03-30
<https://www.math.unipd.it/~tullio/IS-1/2025/Dispense/T06.pdf>
- **Materiale relativo alla parte pratica del corso di Ingegneria del Software**
 Parti consultate: documento completo
 Versione: A.A.2022/2023
 Ultimo accesso: 2026-03-30
<https://www.math.unipd.it/~rcardin/swea/2022/>
- **Repository^G Github del Prof. Cardin**
 Ultimo accesso: 2026-03-30
<https://github.com/rcardin/swe-imdb>
- **Guida Flutter sull'Architettura^G di un'applicazione**
 Ultimo accesso: 2026-03-30
<https://docs.flutter.dev/app-architecture/guide>
- **Glossario**
 Parti consultate: documento completo
 Versione: v.1.0.0
 Ultimo accesso: 2026-02-27
https://swe-bitbybit.github.io/SWE-docs/docs/RTB/documenti_interni/Glossario.pdf



2. Tecnologie

Le tecnologie utilizzate sono state scelte trovando un punto d'incontro tra la necessità di garantire la sicurezza e la tutela dei dati personali degli utenti e il bisogno di realizzare un'**Architettura**^G solida, modulare e facilmente estendibile. Di seguito sono riportate tutte le tecnologie adottate

2.1. Framework

2.1.1 Flutter

Flutter è un **Framework**^G open-source sviluppato da Google per la creazione di applicazioni multi-piattaforma a partire da un unico codice sorgente. Consente di sviluppare interfacce per dispositivi mobili, web e desktop utilizzando il linguaggio Dart. Flutter si basa su un sistema di widget componibili, che possono rappresentare qualsiasi elemento dell'interfaccia utente. Flutter include un proprio motore grafico (come Impeller o storicamente Skia) che disegna direttamente i componenti a schermo, garantendo elevata coerenza visiva e prestazioni indipendenti dalla piattaforma sottostante. È il **Framework**^G alla base dell'App Che Protegge e Trasforma, il che consente di poter generare build per iOS e Android da un unico codice sorgente, a fronte di un maggiore utilizzo di risorse rispetto alle soluzioni native e di una minore possibilità di controllo diretto e personalizzazione delle funzionalità native della piattaforma.

- **Versione utilizzata:** 3.41.6
- **Documentazione:** <https://docs.flutter.dev/>

2.2. Linguaggi di programmazione

2.2.1 Dart

Dart è un linguaggio di programmazione sviluppato da Google, progettato per la realizzazione di applicazioni client moderne. È un linguaggio tipizzato (con supporto sia statico che dinamico) e orientato agli oggetti. Viene utilizzato principalmente in combinazione con Flutter, dove il codice Dart viene compilato ahead-of-time in codice nativo per migliorare le prestazioni, oppure eseguito tramite compilazione just-in-time durante lo sviluppo. L'utilizzo di Flutter rende di fatto quasi obbligatoria l'adozione di Dart come linguaggio.

- **Versione utilizzata:** 3.11.4
- **Documentazione:** <https://dart.dev/docs>



2.2.2 Python

Python è un linguaggio di programmazione ad alto livello, interpretato e orientato agli oggetti, noto per la sua sintassi semplice e leggibile. Il codice Python viene eseguito da un interprete che traduce le istruzioni in bytecode, poi eseguito dalla macchina virtuale Python. Nel progetto è utilizzato lato **Backend^G** per la scrittura delle funzioni AWS Lambda e per i test tramite Pytest.

- **Versione utilizzata:** 3.14.3
- **Documentazione:** <https://docs.python.org/3/>

2.3. Servizi AWS

2.3.1 Lambda

AWS Lambda è un servizio **Serverless^G** che consente di eseguire codice senza gestire direttamente server o infrastruttura. Le funzioni vengono attivate in risposta a eventi e scalano automaticamente in base al carico. Nel progetto è utilizzato per implementare la logica **Backend^G**, eseguendo funzioni scritte in Python invocate tramite API Gateway.

- **Versione utilizzata:** Runtime Python 3.14
- **Documentazione:** <https://docs.aws.amazon.com/lambda/>

2.3.2 API Gateway

AWS API Gateway è un servizio che permette di creare, pubblicare e gestire **API REST^G** e HTTP, fungendo da punto di ingresso per le richieste client. Gestisce autenticazione, routing e integrazione con altri servizi AWS. Nel progetto è utilizzato per esporre endpoint HTTP che invocano le funzioni Lambda.

- **Versione utilizzata:** HTTP API (v2)
- **Documentazione:** <https://docs.aws.amazon.com/apigateway/>

2.3.3 DynamoDB

Amazon DynamoDB è un database NoSQL completamente gestito, progettato per offrire alte prestazioni e scalabilità automatica. I dati sono memorizzati in tabelle con chiavi primarie e accessibili con bassa latenza. Nel progetto è utilizzato per memorizzare dati strutturati come le note testuali del diario criptato e le chat con il chatbot.

- **Documentazione:** <https://docs.aws.amazon.com/dynamodb/>



2.3.4 S3

Amazon S3 (Simple Storage Service) è un servizio di storage a oggetti che consente di archiviare e recuperare dati in modo scalabile e sicuro. I file sono organizzati in bucket e accessibili tramite API. Nel progetto è utilizzato per memorizzare contenuti multimediali come immagini e file audio.

- **Documentazione:** <https://docs.aws.amazon.com/s3/>

2.3.5 Cognito

Amazon Cognito è un servizio di gestione delle identità che consente autenticazione, autorizzazione e gestione degli utenti. Supporta integrazione con provider esterni come Google. Nel progetto è utilizzato per gestire l'autenticazione degli utenti tramite account Google e per il controllo degli accessi.

- **Documentazione:** <https://docs.aws.amazon.com/cognito/>

2.3.6 Bedrock

Amazon Bedrock è un servizio che consente di utilizzare modelli di intelligenza artificiale generativa tramite API, senza gestire direttamente l'infrastruttura sottostante. Permette l'accesso a diversi modelli foundation per task come generazione di testo. Nel progetto è utilizzato per implementare la funzionalità di chatbot.

- **Documentazione:** <https://docs.aws.amazon.com/bedrock/>

2.3.7 SES

Amazon SES (Simple Email Service) è un servizio per l'invio e la ricezione di email in modo scalabile e affidabile. Supporta integrazione via API e SMTP. Nel progetto è utilizzato per inviare email ai contatti fidati degli utenti.

- **Documentazione:** <https://docs.aws.amazon.com/ses/>

2.3.8 CloudWatch

Amazon CloudWatch è un servizio di monitoraggio e osservabilità che raccoglie metriche, log ed eventi dai servizi AWS. Consente analisi, visualizzazione e configurazione di allarmi. Nel progetto è utilizzato per la raccolta e l'analisi dei log generati dalle funzioni Lambda.

- **Documentazione:** <https://docs.aws.amazon.com/cloudwatch/>



2.3.9 EventBridge

Amazon EventBridge è un servizio di event bus **Serverless^G** che consente di gestire e instradare eventi tra diversi servizi AWS. Esso permette di definire regole per reagire automaticamente a eventi generati da servizi AWS o da fonti esterne. Nel progetto è utilizzato per orchestrare i flussi di controllo di utilizzo dell'app da parte degli utenti, e l'invio delle email secondo le tempistiche stabilite dal Dead man's switch.

- **Documentazione:** <https://docs.aws.amazon.com/eventbridge/>

2.3.10 SAM

AWS **Serverless^G** Application Model (SAM) è un **Framework^G** open-source che semplifica la definizione e il **Deployment^G** di applicazioni **Serverless^G** su AWS. Estende AWS CloudFormation introducendo una sintassi più compatta per risorse come Lambda, API Gateway e DynamoDB e include strumenti per build, testing locale e deploy delle applicazioni. Nel progetto è utilizzato per definire e distribuire l'infrastruttura **Serverless^G** in modo semplificato.

- **Documentazione:** <https://docs.aws.amazon.com/serverless-application-model/>

2.3.11 CloudFormation

AWS CloudFormation è un servizio di **Infrastructure as Code^G** che consente di modellare e gestire risorse AWS tramite template dichiarativi in formato **YAML^G** o JSON. Nel progetto è utilizzato come base per il provisioning dell'infrastruttura, anche tramite template generati da SAM.

- **Documentazione:** <https://docs.aws.amazon.com/cloudformation/>
- **Versioni:** CloudFormation non prevede versioni di servizio esplicite; eventuali aggiornamenti sono gestiti direttamente da AWS. I template possono però utilizzare versioni di specifiche risorse e trasformazioni (es. `AWS::ServerlessG`).

2.4. Librerie

2.4.1 http

http è una libreria per il linguaggio Dart che consente di effettuare richieste HTTP in modo semplice e asincrono. Fornisce metodi per eseguire operazioni come GET, POST, PUT e DELETE, gestendo automaticamente la serializzazione delle richieste e la ricezione delle risposte. È progettata per essere leggera e facilmente integrabile in applicazioni client.



- **Versione utilizzata:** 1.6.0
- **Documentazione:** <https://pub.dev/packages/http>

2.4.2 image_picker

`image_picker` è una libreria Flutter che permette di accedere alla fotocamera e alla galleria del dispositivo per selezionare o acquisire immagini e video. Fornisce un'interfaccia unificata tra piattaforme diverse, gestendo le differenze tra iOS e Android.

- **Versione utilizzata:** 1.2.1
- **Documentazione:** https://pub.dev/packages/image_picker

2.4.3 path_provider

`path_provider` è una libreria Flutter che consente di ottenere i percorsi delle directory del file system del dispositivo, come directory temporanee o documenti applicativi. Facilita l'accesso a percorsi corretti e compatibili tra piattaforme per la gestione dei file.

- **Versione utilizzata:** 2.1.5
- **Documentazione:** https://pub.dev/packages/path_provider

2.4.4 provider

`provider` è una libreria per Flutter che offre un meccanismo di gestione dello stato e di propagazione delle dipendenze lungo l'albero dei widget. Si basa sul sistema nativo di `InheritedWidget` di Flutter, semplificandone notevolmente l'utilizzo attraverso un'API dichiarativa. Consente di rendere disponibili oggetti (come i `ViewModel`) a interi sottoalberi dell'interfaccia, facilitando la comunicazione reattiva tra i layer dell'applicazione.

- **Versione utilizzata:** 6.1.5+1
- **Documentazione:** <https://pub.dev/packages/provider>

2.4.5 url_launcher

`url_launcher` è una libreria Flutter che consente di aprire URL nel browser predefinito del dispositivo, avviare applicazioni di posta elettronica, effettuare chiamate telefoniche o aprire altri tipi di risorse tramite i rispettivi handler nativi. Fornisce un'interfaccia unificata e compatibile tra le diverse piattaforme supportate da Flutter.

- **Versione utilizzata:** 6.3.2
- **Documentazione:** https://pub.dev/packages/url_launcher



2.4.6 flutter_dotenv

`flutter_dotenv` è una libreria che permette di caricare variabili di configurazione da un file `.env` all'interno di un'applicazione Flutter. Consente di separare le configurazioni sensibili o dipendenti dall'ambiente (come chiavi API o endpoint) dal codice sorgente, rendendo più agevole la gestione di ambienti diversi senza modificare il codice dell'applicazione.

- **Versione utilizzata:** 6.0.0
- **Documentazione:** https://pub.dev/packages/flutter_dotenv

2.4.7 flutter_web_auth_2

`flutter_web_auth_2` è una libreria Flutter progettata per gestire flussi di autenticazione basati su protocolli web standard come OAuth 2.0 e OpenID Connect. Apre una sessione browser controllata, intercetta il redirect dell'URL di callback e restituisce il codice di autorizzazione all'applicazione, abilitando l'integrazione con provider di identità esterni senza esporre le credenziali nel codice nativo.

- **Versione utilizzata:** 5.0.2
- **Documentazione:** https://pub.dev/packages/flutter_web_auth_2

2.4.8 flutter_map

`flutter_map` è una libreria Flutter open-source per la visualizzazione di mappe interattive. Fornisce funzionalità per il rendering di layer multipli, gestione di marker, polilinee e poligoni, oltre al supporto per gesture come zoom e pan. È altamente configurabile e non dipende da servizi proprietari esterni come Google Maps.

- **Versione utilizzata:** 8.3.0
- **Documentazione:** https://pub.dev/packages/flutter_map

2.4.9 geolocator

`geolocator` è una libreria Flutter che consente di accedere ai servizi di geolocalizzazione del dispositivo. Permette di ottenere la posizione corrente, monitorare aggiornamenti di posizione in tempo reale e gestire i permessi richiesti dalle diverse piattaforme.

- **Versione utilizzata:** 14.0.2
- **Documentazione:** <https://pub.dev/packages/geolocator>



2.4.10 get_it

`get_it` è un service locator per Dart e Flutter utilizzato per la gestione delle dipendenze all'interno dell'applicazione. Consente di registrare e recuperare istanze di oggetti in modo centralizzato, facilitando l'implementazione di architetture modulari e testabili.

- **Versione utilizzata:** 9.2.1
- **Documentazione:** https://pub.dev/packages/get_it

2.4.11 latlong2

`latlong2` è una libreria Dart che fornisce strutture dati e utilities per la gestione di coordinate geografiche (latitudine e longitudine). Include funzionalità per il calcolo di distanze tra punti e altre operazioni geospaziali di base.

- **Versione utilizzata:** 0.10.1
- **Documentazione:** <https://pub.dev/packages/latlong2>

2.4.12 path

`path` è una libreria Dart che fornisce strumenti per la gestione e manipolazione dei percorsi file, indipendenti dalla piattaforma di esecuzione.

- **Versione utilizzata:** 1.9.1
- **Documentazione** <https://pub.dev/packages/path>

2.4.13 flutter_secure_storage

`flutter_secure_storage` è una libreria Flutter che consente il salvataggio di dati sensibili in modo sicuro e specifico per la piattaforma di esecuzione. I dati vengono salvati come coppie chiave-valore e possono essere criptati prima del salvataggio.

- **Versione utilizzata:** 10.0.0
- **Documentazione** https://pub.dev/packages/flutter_secure_storage

2.4.14 just_audio

`just_audio` è una libreria Flutter che fornisce strumenti per la creazione e utilizzo di player audio, e supporta molteplici formati di tracce audio.

- **Versione utilizzata:** 0.10.5
- **Documentazione** https://pub.dev/packages/just_audio



2.4.15 file_picker

`file_picker` è una libreria Flutter che permette l'utilizzo del file picker nativo per la selezione di uno o più file. La libreria permette anche di filtrare i file selezionabili dall'Utente^G in base al formato.

- **Versione utilizzata:** 11.0.2
- **Documentazione** https://pub.dev/packages/file_picker

2.4.16 url_launcher_platform_interface

`url_launcher_platform_interface` è una libreria flutter che permette estensioni platform-specific del plugin `url_launcher`.

- **Versione utilizzata:** 2.3.2
- **Documentazione** https://pub.dev/packages/url_launcher_platform_interface

2.4.17 command_it

`command_it` è una libreria Flutter che permette l'implementazione del pattern *Command*, tramite la creazione di wrap functions con gestione automatica dello stato.

- **Versione utilizzata:** 9.5.1
- **Documentazione** https://pub.dev/packages/command_it

2.4.18 listen_it

`listen_it` è una libreria Flutter che fornisce primitive reattive per la creazione di oggetti che notificano automaticamente eventuali ascoltatori in seguito a mutamenti del proprio stato.

- **Versione utilizzata:** 5.3.5
- **Documentazione** https://pub.dev/packages/listen_it

2.4.19 mocktail

`mocktail` è una libreria Dart che fornisce una API per la creazione di Mock senza doverli creare da zero.

- **Versione utilizzata:** 1.0.5
- **Documentazione** <https://pub.dev/packages/mocktail>



2.4.20 shared_preferences

`shared_preferences` è una libreria Flutter che permette l'utilizzo della memorizzazione persistente platform-specific per il salvataggio di dati semplici.

- **Versione utilizzata:** 2.5.5
- **Documentazione** https://pub.dev/packages/shared_preferences

2.4.21 connectivity_plus

`connectivity_plus` è una libreria Flutter che permette all'applicazione di conoscere i tipi di connessione disponibili in un dato momento, in modo da reagire di conseguenza.

- **Versione utilizzata:** 7.1.1
- **Documentazione** https://pub.dev/packages/connectivity_plus

2.4.22 audio_session

`audio_session` è una libreria Flutter per la gestione delle sessioni audio dell'applicazione. Permette una configurazione molto approfondita delle sessioni audio, permettendo di specificare caratteristiche quali il comportamento in presenza di audio appartenenti ad altre applicazioni o quali dispositivi possono riprodurre l'audio.

- **Versione utilizzata:** 0.2.3
- **Documentazione** https://pub.dev/packages/audio_session

2.4.23 python-ulid

`python-ulid` è una libreria Python per la generazione di identificatori univoci ULID (Universally Unique Lexicographically Sortable Identifier). Gli ULID combinano unicità e ordinamento temporale, risultando particolarmente utili in sistemi distribuiti dove è necessario generare ID ordinabili senza coordinamento centrale.

- **Versione utilizzata:** 3.1.0
- **Documentazione:** <https://pypi.org/project/python-ulid/>

2.4.24 rank-bm25

`rank-bm25` è una libreria Python che implementa l'algoritmo di ranking BM25 (Best Matching 25), utilizzato nel campo dell'information retrieval per valutare la rilevanza dei documenti rispetto a una query testuale. Fornisce diverse varianti dell'algoritmo BM25 e consente di costruire rapidamente sistemi di ricerca basati su punteggi statistici senza necessità di motori di ricerca completi.



- **Versione utilizzata:** 0.2.2
- **Documentazione:** <https://pypi.org/project/rank-bm25/>

2.4.25 boto3

boto3 è una libreria Python che fornisce strumenti per la creazione, configurazione e gestione dei servizi Amazon Web Services.

- **Versione utilizzata:** 1.43.5
- **Documentazione:** <https://docs.aws.amazon.com/boto3/latest/>

2.5. Tecnologie di analisi statica

2.5.1 SonarQube

SonarQube è una piattaforma open-source per l'analisi continua della qualità del codice. Esegue **Analisi Statica**^G su diversi linguaggi di programmazione per individuare bug, vulnerabilità, code smells e problemi di copertura dei test. Il funzionamento si basa su scanner che analizzano il codice sorgente e inviano i risultati a un server centrale, dove vengono aggregati e visualizzati tramite una dashboard dedicata.

- **Versione utilizzata:** 26.3.0
- **Documentazione:** <https://docs.sonarsource.com/sonarqube/>

2.6. Tecnologie di analisi dinamica

2.6.1 Pytest

Pytest è un **Framework**^G di testing per il linguaggio Python che consente di scrivere test in modo semplice e scalabile. Supporta test funzionali, unitari e di integrazione, offrendo funzionalità come fixture, parametrizzazione dei test e scoperta automatica dei test.

- **Versione utilizzata:** 9.0.2
- **Documentazione:** <https://docs.pytest.org/>

2.6.2 Moto

Moto è una libreria Python che consente di mockare i servizi AWS durante i test. Simula il comportamento delle API AWS, permettendo di testare codice che interagisce con servizi **Cloud**^G senza effettuare chiamate reali. Funziona intercettando le richieste e restituendo risposte simulate coerenti con quelle dei servizi AWS.

- **Versione utilizzata:** 5.1.23
- **Documentazione:** <https://docs.getmoto.org/>



2.7. Altro

2.7.1 Ollama

Ollama è una piattaforma che consente di eseguire e gestire LLM in locale tramite una semplice interfaccia. Permette il download, l'esecuzione e l'interazione con modelli di intelligenza artificiale direttamente in ambiente locale, senza necessità di servizi **Cloud^G** esterni. Il funzionamento si basa su un runtime che gestisce i modelli e espone endpoint per inferenza.

- **Versione utilizzata:** 0.18.3
- **Documentazione:** <https://ollama.com/docs>

2.7.2 Docker

Docker è una piattaforma di containerizzazione che consente di creare, distribuire ed eseguire applicazioni all'interno di ambienti isolati detti container. I container includono tutto il necessario per eseguire un'applicazione, garantendo portabilità e consistenza tra ambienti diversi. Nel progetto è utilizzato a supporto delle pipeline di **CI/CD^G** e per testare in locale servizi come DynamoDB e S3.

- **Documentazione:** <https://docs.docker.com/>

3. Architettura

3.1. Architettura logica

Il sistema software è progettato secondo i principi di un'**Architettura^G** a Microservizi, integrando internamente il paradigma dell'**Architettura^G** Esagonale (o Ports and Adapters). Questa scelta impone una netta separazione tra la logica di dominio e le tecnologie di infrastruttura, garantendo che il "core" applicativo rimanga isolato da influenze esterne. L'obiettivo cardine è massimizzare l'indipendenza della logica di business, favorendo l'intercambiabilità dei componenti (database, servizi **Cloud^G**, interfacce), la facilità di test automatizzati e una scalabilità granulare dei singoli moduli funzionali (**Separation of Concerns^G**).

L'interazione tra l'ecosistema mobile e l'infrastruttura di servizio avviene tramite interfacce API ben definite, che fungono da **Contratto^G** formale tra i due mondi. Tale disaccoppiamento permette ai microservizi di evolvere indipendentemente dal client, garantendo al contempo una comunicazione sicura e agnostica rispetto ai linguaggi di programmazione utilizzati.



3.1.1 Livello Client (Frontend Mobile)

Il client mobile è sviluppato seguendo il pattern architetturale Model-View-ViewModel (MVVM). Il codice è suddiviso nei seguenti strati logici:

- **Presentation Layer (View & ViewModel):** La View, costituita dai diversi widget Flutter, ha la responsabilità esclusiva di renderizzare l'interfaccia e catturare gli input **Utente^G**. Il ViewModel funge da mediatore: gestisce lo stato della View notificandola di cambiamenti e incapsula la logica di presentazione tramite il pattern Command. Quest'ultimo permette di gestire operazioni asincrone, monitorare i caricamenti e centralizzare la gestione degli errori senza sporcare il codice dell'interfaccia.
- **Domain Layer (Business Logic^G):** Rappresentato dai modelli di dominio, che definiscono le entità base su cui opera l'intera applicazione. In quanto modelli di dominio, sono completamente indipendenti da qualsiasi tecnologia di persistenza o protocollo di comunicazione, gestendo esclusivamente la business logic dell'applicazione.
- **Data Layer^G (Repository^G & Service):** Incaricato dell'acquisizione dei dati, il data layer utilizza classi Repository per trasformare i dati grezzi in arrivo dal backend in modelli di dominio, gestendo la logica di caching e di gestione degli errori. Le classi Service, invece, si fanno carico della gestione delle comunicazioni con fonti di dati esterne, esponendo metodi per inviare richieste HTTP al backend e ricevere le risposte sottoforma di JSON.

3.1.2 Livello di Servizio (Backend)

L'**Architettura^G** logica dei microservizi in **Cloud^G** si fonda sui principi dell'**Architettura^G** Esagonale, organizzando l'ambiente computazionale in tre componenti principali totalmente disaccoppiati:

- **Inbound Adapters (Entry Points):** Costituiscono i punti di ingresso del microservizio (es. API Gateway, Lambda Trigger). Hanno la responsabilità di intercettare le richieste esterne, gestire le autorizzazioni e validare il formato del **Payload^G** in ingresso, traducendo le chiamate esterne in comandi comprensibili per il nucleo applicativo.
- **Domain Core (Business Logic^G):** Rappresenta l'esagono interno dove risiede la logica di business pura. Implementato tramite funzioni indipendenti (AWS Lambda), questo strato è progettato per essere totalmente isolato dal protocollo di trasporto (HTTP) e dai dettagli della persistenza. Comunica con l'esterno esclusivamente attraverso interfacce definite (Porte), garantendo che le regole di dominio non vengano mai contaminate da specifiche tecniche o **Driver^G** di terze parti.



- **Outbound Adapters (Infrastructure):** Rappresentano le implementazioni concrete delle porte di uscita. Questi adattatori gestiscono l'interfacciamento fisico con i database (documentali o relazionali), i servizi di autenticazione (come AWS Cognito) o gli storage di file grezzi. Grazie a questo isolamento, è possibile sostituire o aggiornare la tecnologia di persistenza senza dover modificare la logica di business centrale.

3.2. Design pattern utilizzati

3.2.1 Model-view-viewmodel

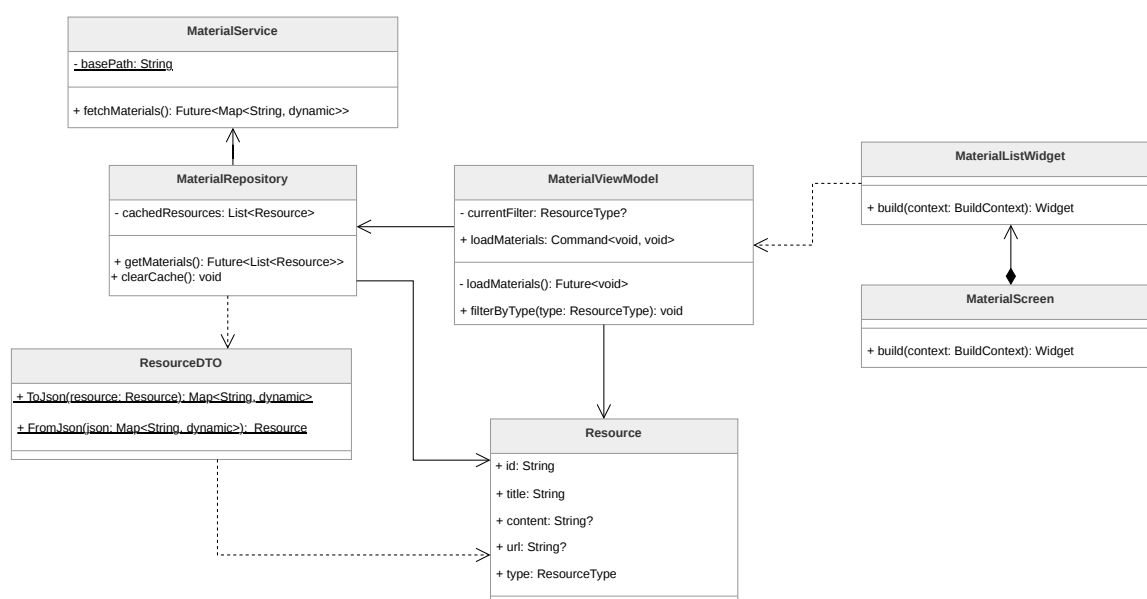


Figura 1: Rappresentazione del pattern MVVM nel modulo del materiale informativo

Descrizione del pattern:

Il pattern MVVM prevede la separazione tra la resa grafica, la **Presentation Logic^G** e la **Business Logic^G** in tre componenti principali:

- **Model:** contiene il codice responsabile per la **Business Logic^G** dell'applicazione.
- **View:** contiene la parte responsabile della resa grafica del software e della cattura delle interazioni dell'**Utente^G**, a cui reagisce invocando i metodi o i comandi messi a disposizione dal *ViewModel*.
- **ViewModel:** mantiene e gestisce lo stato della *View* e coordina le interazioni tra quest'ultima e il *Model*. La *View* può modificare il proprio stato esclusivamente attraverso l'interazione con i metodi e i **Command** esposti dal *ViewModel*.



Nel contesto del **Framework^G** Flutter, l'implementazione di questo pattern viene riadattata per assecondare la natura dichiarativa dell'interfaccia. Nello specifico, il `ViewModel` è tipicamente implementato sfruttando una classe o un mixin proprietario chiamato `ChangeNotifier`, il quale consente di notificare esplicitamente la View dei cambiamenti di stato interno. La View, composta a sua volta da `Widget`, ascolta reattivamente tali notifiche potendo ricostruire e aggiornare solo le specifiche porzioni grafiche necessarie.

Ragioni per l'utilizzo:

L'utilizzo del pattern garantisce notevoli vantaggi poiché permette una più semplice ed efficace gestione delle varie parti dell'applicazione, limitando il forte accoppiamento (*coupling*) tra l'interfaccia utente visiva e la logica di business o di presentazione sottostante. Questo netto disaccoppiamento migliora la manutenibilità del codice dell'applicativo e agevola considerevolmente lo sviluppo e l'esecuzione degli unit test, in quanto permette di testare i `ViewModel` e i `Model` isolandoli dal ciclo di vita visivo tipico del sistema operativo e del **Framework^G**.

Riferimenti nel progetto:

Il pattern MVVM è ampiamente utilizzato per strutturare l'interfaccia utente dell'applicazione e la logica ad essa associata, separando la resa grafica dalla gestione dello stato e dei dati. Di seguito sono elencati i punti principali dove viene impiegato; all'interno di tali sezioni sono fornite spiegazioni dettagliate su come il pattern interagisce con il sistema circostante:

- Autenticazione → `AuthViewModel`
- Chatbot → `ChatbotViewModel`
- Diario → `DiaryViewModel`
- Contatti Fidati → `TrustedContactViewModel`
- Materiale Informativo → `MaterialViewModel`
- Luoghi Sicuri → `SafePlaceViewModel`
- Impostazioni → `DeadManViewModel`



3.2.2 Command

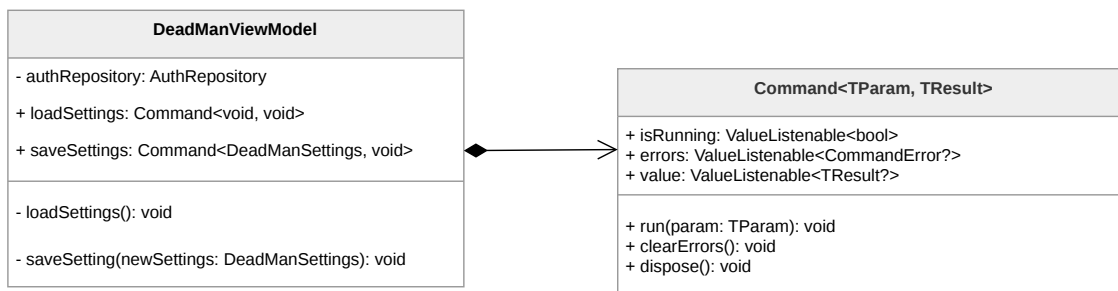


Figura 2: Rappresentazione del pattern Command

Descrizione del pattern:

Il pattern Command è un design pattern comportamentale che incapsula una richiesta o un'azione come un oggetto autonomo. Nel frontend dell'applicazione, questo pattern è implementato tramite il pacchetto `command_it`, che permette di standardizzare l'esecuzione della logica asincrona all'interno dei ViewModel.

A differenza di una semplice funzione, il comando così implementato agisce come un contenitore logico che gestisce automaticamente il **Ciclo di vita**^G di un'operazione: dall'invocazione alla gestione dei risultati, fino alla cattura di eventuali eccezioni. Ogni comando espone tre proprietà osservabili tramite `ValueListenable`:

- `isRunning`: un booleano che indica se l'operazione è attualmente in esecuzione.
- `errors`: contiene eventuali errori catturati durante l'esecuzione.
- `value`: contiene il risultato dell'operazione una volta completata con successo.

Ragioni per l'utilizzo:

L'adozione di `command_it` risponde alla necessità di eliminare il "boilerplate code" (codice ripetitivo) tipico della gestione dello stato asincrono in Flutter. Le ragioni principali includono:

- **Standardizzazione dello Stato**: evita la proliferazione manuale di variabili come `isLoading` o `errorMessage` per ogni singola azione nel ViewModel, accentrando tutto nell'oggetto Command.
- **Prevenzione di Esecuzioni Concorrenti**: il pattern impedisce automaticamente l'avvio accidentale di più istanze della stessa operazione (es. pressioni ripetute su un tasto di invio) finché la precedente non è terminata.



- **Disaccoppiamento UI-Logica:** la View non deve implementare blocchi **try-catch** o logiche di feedback: si limita a invocare il comando e "osservare" reattivamente i suoi cambiamenti di stato per mostrare indicatori di caricamento o dialoghi di errore.

Riferimenti nel progetto:

Il pattern viene implementato sistematicamente in tutti i ViewModel dell'applicazione per la gestione delle interazioni **Utente^G** e delle operazioni asincrone.

3.2.3 Observer

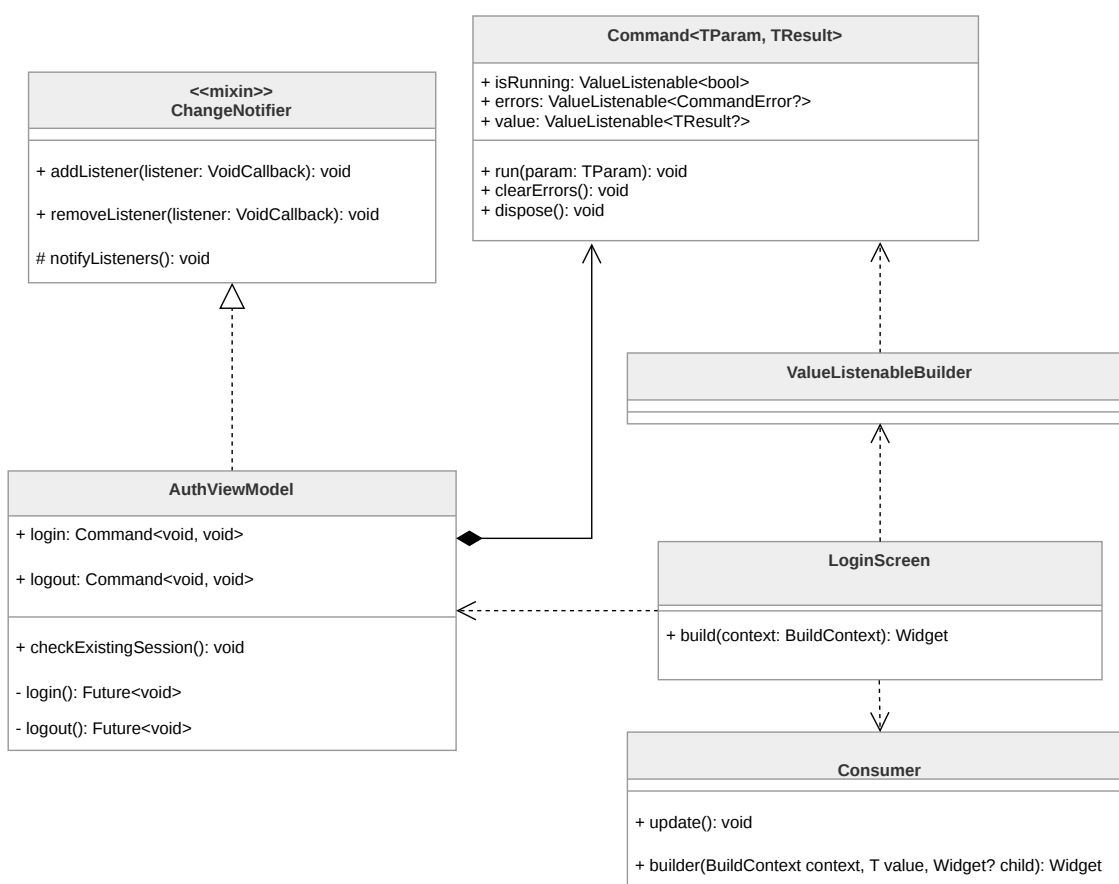


Figura 3: Rappresentazione del pattern Observer applicato nel modulo dell'autenticazione

Descrizione del pattern:

Il pattern Observer definisce un meccanismo di comunicazione in cui un oggetto, chiamato *Subject* (o *Publisher*), mantiene una lista di dipendenti, chiamati *Observers* (o *Subscriber*), e li notifica automaticamente di ogni cambiamento di stato. Nel sistema sviluppato, questo



pattern è stato implementato con due diversi livelli di granularità per gestire la reattività dell'interfaccia **Utente**^G:

- **Livello Macro (ChangeNotifier e Consumer)**: Il ViewModel agisce come Publisher globale della schermata espandendo il **ChangeNotifier**. Quando il ViewModel invoca il metodo **notifyListeners()**, segnala che l'intero stato della logica di presentazione è mutato. Il widget **Consumer** funge da Subscriber, ricostruendo i rami della View che dipendono da quel ViewModel. Questo approccio è utilizzato per aggiornamenti di stato generali che influenzano più componenti contemporaneamente (per esempio lo stato di autenticazione).
- **Livello Micro (Command e ValueListenableBuilder)**: Per una gestione più granulare, il sistema sfrutta l'integrazione tra il pattern Command e l'interfaccia **ValueListenable**. Ogni azione asincrona (es. **login**, **loadContacts**) è incapsulata in un oggetto **Command**, che agisce come un Publisher specializzato. Esso espone tre flussi di dati osservabili separatamente: **isRunning**, **errors** e **value**. La View utilizza widget **ValueListenableBuilder** per "mettersi in ascolto" solo di una specifica proprietà del comando. Ciò permette, ad esempio, di mostrare un indicatore di caricamento reagendo solo alla variazione di **isRunning**, senza richiedere la notifica di aggiornamento dell'intero ViewModel.

Ragioni per l'utilizzo:

L'adozione di questa doppia implementazione risponde a esigenze di efficienza prestazionale e pulizia del codice. L'utilizzo di **ValueListenableBuilder** accoppiato ai **Command** riduce drasticamente il numero di "rebuild" inutili dell'interfaccia: la View può reagire al cambiamento del risultato di una singola operazione asincrona senza dover processare nuovamente lo stato globale del modulo. Questo garantisce una UI fluida anche in presenza di logiche complesse o flussi di dati frequenti. Inoltre, questo approccio disaccoppia la gestione degli errori e del caricamento (loading state) dalla logica di business principale, centralizzandola negli oggetti Command osservabili.

Riferimenti nel progetto:

Essendo strettamente interconnesso al MVVM, il pattern Observer è utilizzato per la gestione dell'intera UI. Di seguito i punti principali dove i ViewModel agiscono da *Publisher* per le rispettive schermate:

- Autenticazione → **AuthViewModel**
- Chatbot → **ChatbotViewModel**
- Diario → **DiaryViewModel**
- Contatti Fidati → **TrustedContactViewModel**



- Materiale Informativo → **MaterialViewModel**
- Luoghi Sicuri → **SafePlaceViewModel**
- Impostazioni → **DeadManViewModel**

3.2.4 Proxy

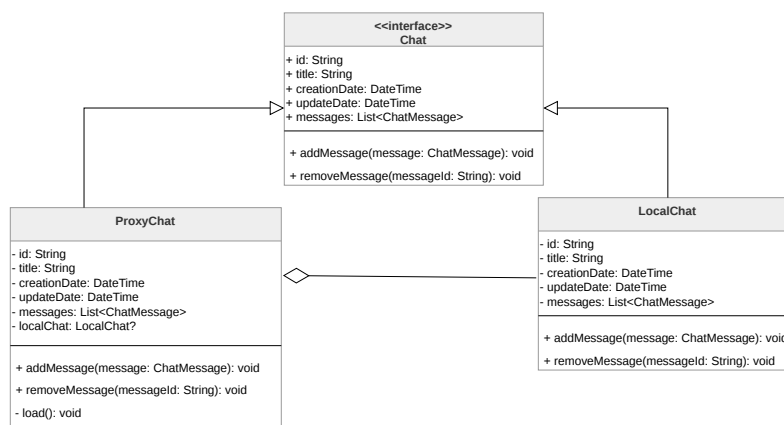


Figura 4: Rappresentazione del pattern Proxy applicato alla gestione delle chat

Descrizione del pattern:

Il pattern Proxy permette la creazione di un oggetto *Proxy* che controlla l'accesso all'oggetto reale. L'oggetto *Proxy* implementa la medesima interfaccia dell'oggetto reale, rendendolo perfettamente intercambiabile agli occhi del codice che lo utilizza, e ha il compito di delegare le richieste a un'istanza dell'oggetto reale solo quando necessario.

Ragioni per l'utilizzo:

L'utilizzo di questo pattern permette di implementare il caricamento ritardato di strutture dati potenzialmente voluminose; nello specifico, le classi che fungono da Proxy operano come segnaposti leggeri contenenti solo metadati essenziali, rimandando il caricamento completo dei contenuti (come i messaggi di una singola chat) al momento del loro effettivo accesso. Questo approccio permette di ridurre sensibilmente l'utilizzo di banda, limitando la quantità di dati recuperati dal **Backend^G** a quelli strettamente necessari, garantendo al contempo una maggiore fluidità nel caricamento degli elenchi e un consumo di risorse sul dispositivo più efficiente.

Riferimenti nel progetto:

Il pattern Proxy è utilizzato principalmente per la gestione dei modelli di dati persistenti:

- Chatbot → **ProxyChat**
- Diari Personali → **ProxyNote**



3.2.5 Singleton

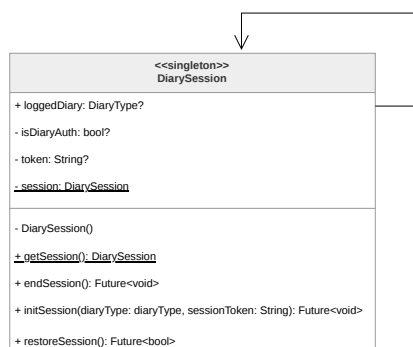


Figura 5: Rappresentazione del pattern Singleton applicato alla sessione del diario

Descrizione del pattern:

Il pattern Singleton garantisce che una classe abbia una sola istanza attiva in ogni momento e fornisce un punto di accesso globale a tale istanza. Questa proprietà si ottiene rendendo privato il costruttore della classe, impedendone l'istanziatura diretta dall'esterno, e mettendo a disposizione un metodo statico o una proprietà (*getter*) che restituisce l'unica istanza disponibile.

Nell'applicazione, questo pattern viene implementato seguendo gli standard del linguaggio Dart, utilizzando un costruttore privato e una variabile statica per mantenere il riferimento all'unica istanza creata al primo accesso.

Ragioni per l'utilizzo:

L'impiego del Singleton è fondamentale per gestire componenti che devono mantenere uno stato unico e coerente in tutto il sistema. Nello specifico, garantisce l'esistenza di una sola sessione attiva per il diario, impedendo che l'**Utente^G** possa accedere contemporaneamente alla versione criptata e a quella fittizia. Questo approccio previene la condivisione accidentale di dati sensibili e assicura che ogni operazione di lettura o scrittura avvenga sempre nel contesto dell'archivio corretto, eliminando alla radice potenziali conflitti di sincronizzazione.

Riferimenti nel progetto:

Il pattern Singleton è utilizzato per il coordinamento dei componenti di sessione:

- Diari Personali → **DiarySession**



3.2.6 Dependency Injection e Service Locator

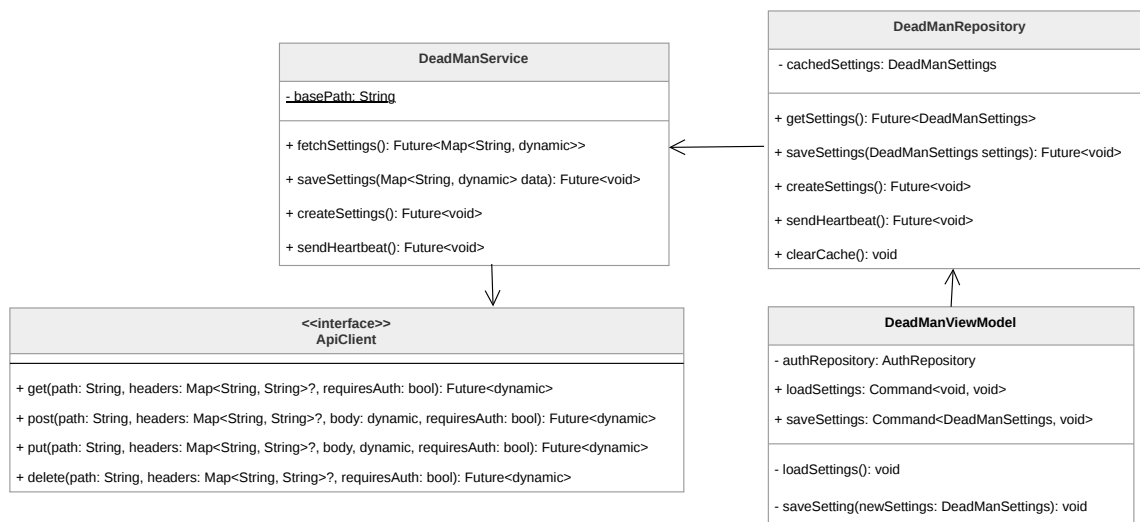


Figura 6: Rappresentazione del pattern Dependency Injection nel modulo del Dead Man' switch

Descrizione del pattern:

Il sistema adotta il pattern Dependency Injection (DI) supportato dal pattern **Service Locator**, implementato tramite la libreria `get_it`. Questo approccio permette di separare la creazione delle dipendenze dal loro effettivo utilizzo.

L'**Architettura^G** si basa su un punto di configurazione centralizzato (`setupLocator`) dove vengono registrate tutte le istanze del sistema. La DI viene poi attuata tramite *Constructor Injection*: i componenti (come i ViewModel) non creano mai i propri **Repository^G** o Service internamente, ma dichiarano le dipendenze nel proprio costruttore. È il Service Locator a farsi carico di "iniettare" l'istanza corretta al momento della creazione, prelevandola dal registro globale.

Ragioni per l'utilizzo:

L'integrazione di `get_it` e della DI offre tre vantaggi fondamentali per l'applicazione:

- **Gestione del Ciclo di vita^G**: Permette di differenziare tra *Singletons* (istanze uniche come `DeadManRepository` o `ApiClient`), che mantengono lo stato per tutta la durata dell'app, e *Factories* (come i ViewModel), che vengono creati ex-novo ogni volta che si accede a una schermata per garantire uno stato pulito.
- **Lazy Loading**: Grazie alla registrazione di tipo *Lazy*, le risorse (es. `MaterialService`) vengono istanziate solo nel momento del primo effettivo utilizzo, ottimizzando il consumo di memoria e i tempi di avvio del software.



- **Testabilità e Mocking:** Centralizzare la creazione dei componenti permette di sostituire istantaneamente le implementazioni reali con versioni fittizie (Mock) durante i test, semplicemente modificando la registrazione nel locator senza toccare il codice della View o della logica di business.

Riferimenti nel progetto:

Il pattern è applicato diffusamente in tutta l'applicazione. Di seguito alcune classi che integrano attivamente questo approccio:

- Autenticazione → **AuthViewModel**
- Chatbot → **ChatbotViewModel**
- Contatti Fidati → **TrustedContactViewModel**

3.2.7 Adapter

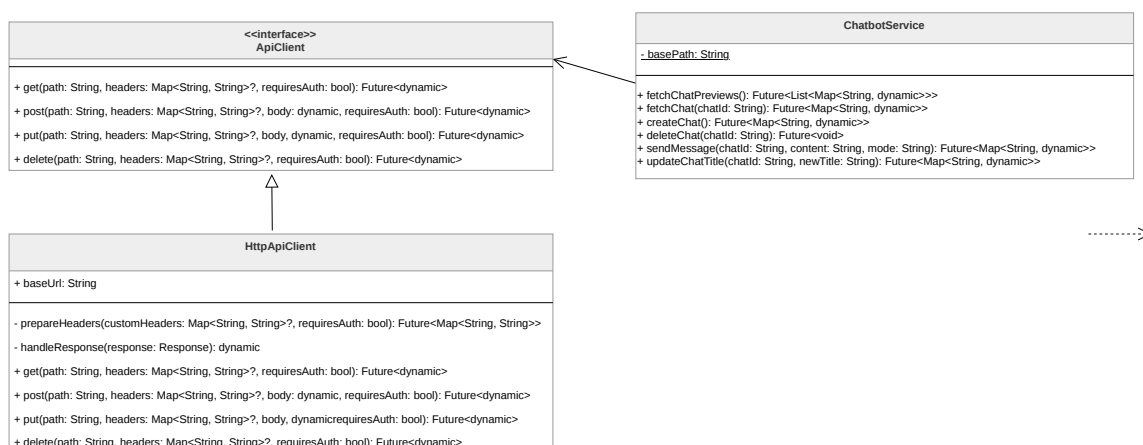


Figura 7: Rappresentazione del pattern Adapter all'ApiClient

Descrizione del pattern:

Il pattern Adapter è un design pattern strutturale che permette a interfacce incompatibili di lavorare insieme. Agisce come un convertitore che traduce le chiamate del client nel formato compreso da una classe o libreria esterna. Nell'**Architettura^G** del sistema, questo pattern è fondamentale per implementare i principi dell'**Architettura^G** Esagonale, dove il "core" dell'applicazione definisce dei contratti (Porte) e gli Adapter forniscono l'implementazione concreta per interfacciarsi con il mondo esterno.

Ragioni per l'utilizzo:

L'integrazione di questo pattern permette di isolare il codice sorgente dalle dipendenze di terze parti. Utilizzando un Adapter per la comunicazione di rete o per i servizi di



sistema, si evita di "inquinare" i Service o i **Repository^G** con dettagli implementativi di librerie specifiche. Se in futuro si decidesse di cambiare il client HTTP (ad esempio passando dalla libreria **http** a **dio**) o il fornitore di un servizio **Cloud^G**, basterebbe creare un nuovo Adapter che rispetti l'interfaccia predefinita, senza dover modificare una singola riga di logica di business.

Riferimenti nel progetto:

Il pattern è applicato in modo sistematico nella gestione dell'infrastruttura di rete e dei servizi hardware del dispositivo, oltre che nelle AWS Lambda che utilizzano l'**Architettura^G** esagonale:

- NetworkAdapter → **ApiClient**
- Location Service → **LocationService**
-

3.3. Architettura di deployment

L'**Architettura^G** di **Deployment^G** del sistema si divide in due componenti principali: l'applicazione Client (eseguita sul dispositivo dell'**Utente^G**) e l'infrastruttura di **Backend^G** (eseguita in **Cloud^G**). Il sistema si basa su un paradigma **Serverless^G** ed **Event-Driven^G**, per garantire scalabilità, elevata disponibilità e abbattimento dei costi legati alla gestione manuale dei server.

3.3.1 Client Mobile

L'applicazione viene rilasciata come eseguibile nativo sui dispositivi mobili degli utenti (Android/iOS). Essendo sviluppata tramite il **Framework^G** **Flutter**, il codice scritto in Dart viene compilato in formato binario specifico per l'**Architettura^G** hardware del dispositivo ospite. Il Client si interfaccia in modo del tutto indipendente ai servizi di **Backend^G** unicamente tramite la rete Internet, sfruttando protocolli di comunicazione sicuri.

3.3.2 Infrastruttura AWS (Serverless Cloud)

L'intero ecosistema di **Backend^G** è ospitato e gestito all'interno di **Amazon Web Services (AWS)**. Questa scelta architetturale consente:

- **Scalabilità dinamica^G**: le risorse allocate (il calcolo e lo storage) scalano o si accendono/spengono automaticamente in base al volume di richieste ricevute dai Client;



- **Isolamento e Sicurezza:** ogni funzione ha una specifica responsabilità e policy di accesso minima verso le altre risorse, limitando l'esposizione al minimo indispensabile;
- **Gestione del servizio (Zero Ops^G):** l'uso di infrastrutture completamente gestite e la persistenza dei dati automatizzata tramite **Amazon DynamoDB** e **Amazon S3** libera il team dagli oneri di manutenzione sistemistica.

Il flusso operativo del sistema e la connessione tra i componenti AWS si articolano nei seguenti layer di **Deployment^G**:

- **Amazon API Gateway:** costituisce l'unico punto di ingresso espositivo e si occupa di orchestrare e bilanciare le richieste HTTP provenienti dal Client.
- **Amazon Cognito:** agisce come filtro primario in stretta collaborazione con l'API Gateway, validando i token **Utente^G** e negando le richieste non autorizzate.
- **AWS Lambda:** rappresenta il cuore elaborativo decentralizzato; l'infrastruttura crea dinamicamente container di esecuzione che accolgono le richieste smistate e processano la **Business Logic^G** interna.
- **Amazon DynamoDB e Amazon S3:** compongono lo strato di persistenza profondo ospitando, in alta disponibilità, i dati relazionali/JSON e i contenuti multimediali grezzi sollevando il livello server della necessità di memoria sul disco.

3.3.3 Orchestrazione Event-Driven

Oltre alle comunicazioni sincrone HTTP per le API *REST*, l'**Architettura^G** di **Deployment^G** prevede un'orchestrazione asincrona fondamentale per le funzionalità dell'applicazione (es. le notifiche del sistema *Dead Man's Switch*). I componenti comunicano internamente tramite eventi gestiti da **Amazon EventBridge**, che funge da **Message Bus^G** e permette:

- **Comunicazione asincrona e disaccoppiata:** riducendo la dipendenza operativa tra i servizi;
- L'esecuzione ritardata e pianificata di check sui dati salvati nel database;
- L'instradamento di Eventi critici e allerte verso il servizio **Amazon SES** per l'invio automatizzato di e-mail.

3.3.4 Infrastructure as Code (IaC)

Il **Deployment^G** materiale di tutte le risorse in ambiente AWS non è eseguito manualmente, ma viene automatizzato in modo riproducibile attraverso l'impiego di **AWS SAM**



(Serverless^G Application Model) e AWS CloudFormation. L'infrastruttura logica e fisica è versionata sotto forma di file **YAML^G**, aderendo interamente al paradigma dell'*Infrastructure as Code^G*.

3.4. Architettura Front End

3.4.1 Autenticazione

La funzionalità di autenticazione è stata implementata seguendo il pattern architetturale MVVM raccomandato dalle linee guida di Flutter, adattato al flusso OAuth 2.0 con OpenID Connect descritto nelle specifiche. La struttura separa nettamente le responsabilità tra i layer: il Service gestisce la comunicazione con gli endpoint di AWS Cognito e il flusso OAuth, il **Repository^G** mantiene lo stato dell'**Utente^G** autenticato e orchestra la traduzione dei dati grezzi in oggetti di dominio tramite i DTO, il ViewModel espone lo stato e le azioni alla View tramite i Command del pattern MVVM. Per la modellazione di un **Utente^G** viene utilizzato il domain model **User**.

L'istanza unica dell'**Utente^G** autenticato viene gestita tramite il Provider di Flutter a livello di app per mantenere una sola istanza attiva per tutta la durata della sessione.

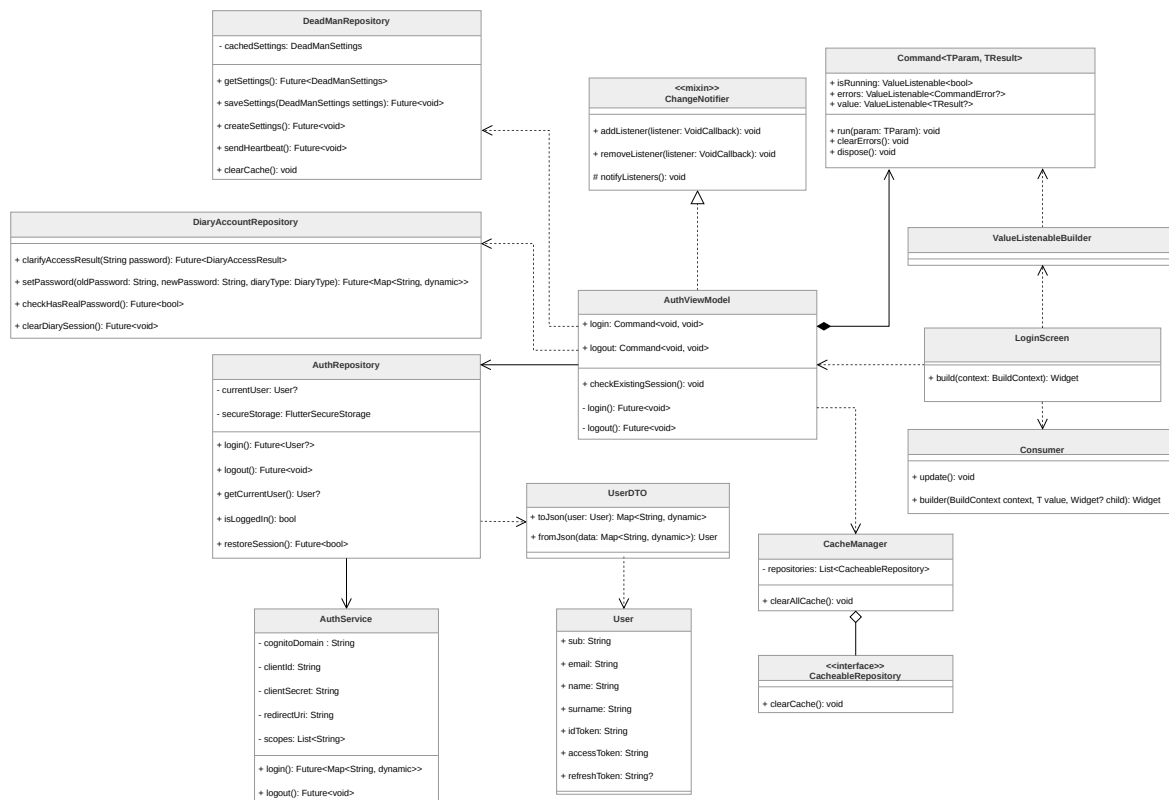


Figura 8: Diagramma delle classi relativo all'autenticazione



LoginScreen

Widget principale che rappresenta la schermata di autenticazione dell'applicazione. Nel contesto del pattern MVVM, svolge il ruolo di View, occupandosi esclusivamente della costruzione dell'interfaccia Utente tramite il metodo `build(BuildContext context)`. Mantiene un riferimento al `AuthViewModel`, dal quale osserva lo stato dell'autenticazione (avvalendosi di componenti come `ValueListenableBuilder` e `Consumer`) e a cui delega le azioni dell'**Utente^G**.

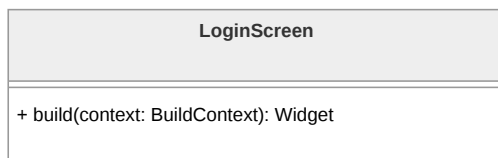


Figura 9: Diagramma della classe `LoginScreen`

User

Rappresenta il modello di dominio dell'**Utente^G** autenticato all'interno dell'applicazione. Contiene le informazioni essenziali legate all'identità e alla sessione, come l'identificativo univoco e i token di accesso, ed è utilizzata dal resto del sistema per gestire lo stato dell'autenticazione in modo indipendente dal formato dei dati provenienti dall'esterno.

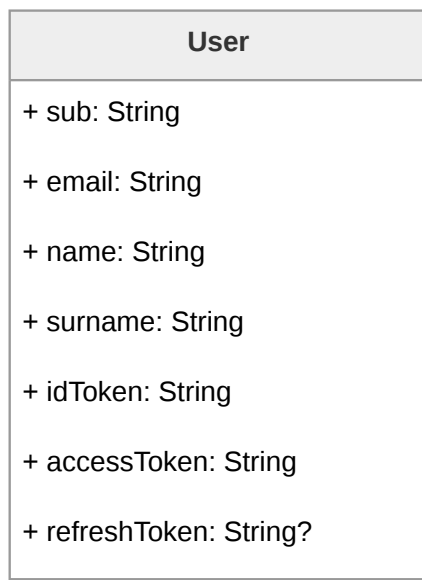


Figura 10: Diagramma della classe `User`



UserDTO

Ha il compito di gestire la conversione tra il modello di dominio **User** e la rappresentazione serializzata dei dati. Attraverso i metodi **toJson** e **fromJson**, consente rispettivamente di trasformare un oggetto **User** in formato JSON e di ricostruirlo a partire da dati serializzati.

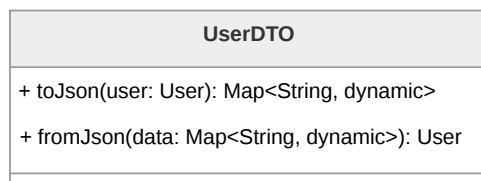


Figura 11: Diagramma della classe **UserDTO**

AuthViewModel

Gestisce lo stato dell'autenticazione e coordina le operazioni richieste dalla View interagendo con il **AuthRepository**. Le funzionalità principali, come il login e il logout, sono esposte tramite il Command pattern. La logica concreta è incapsulata nei metodi che vengono passati come azioni ai rispettivi Command, mentre il metodo **checkExistingSession()** consente di verificare la presenza di una sessione già attiva al momento dell'avvio. Inoltre, funge da orchestratore: al login si interfaccia con il **DeadManRepository** per inizializzare le impostazioni di sicurezza, mentre al logout coordina la pulizia profonda dei dati interagendo con **CacheManager** e **DiaryAccountRepository**.

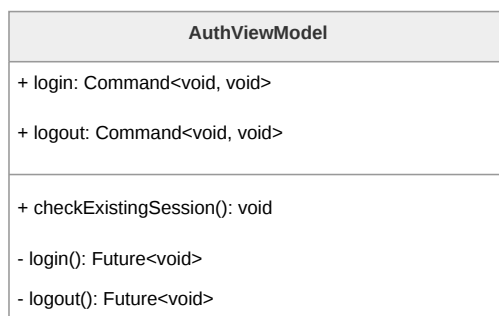


Figura 12: Diagramma della classe **AuthViewModel**

AuthRepository

Funge da intermediario tra il **AuthViewModel** e il livello di accesso ai dati, incapsulando la logica necessaria per gestire l'autenticazione. Mantiene lo stato dell'**Utente^G** corrente tramite **currentUser**, gestisce la persistenza sicura dei token tramite **FlutterSecureStorage** e delega le operazioni di comunicazione remota all'**AuthService**. Il **Repository^G** espone metodi per effettuare il login, il logout, recuperare l'**Utente^G** corrente e verificare o ripristinare una sessione attiva.

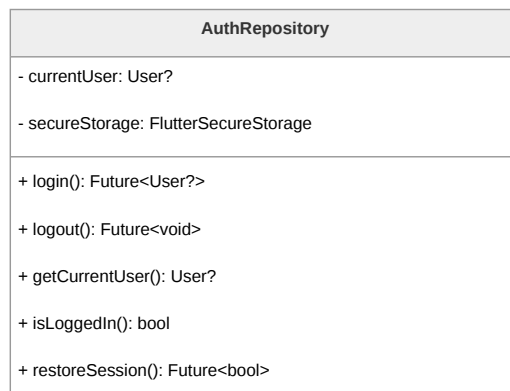


Figura 13: Diagramma della classe **AuthRepository**

AuthService

È responsabile della comunicazione con il sistema di autenticazione esterno AWS Cognito. Contiene le informazioni di configurazione necessarie, tra cui dominio, credenziali del client, URI di redirect e scope richiesti. Si occupa di eseguire le chiamate di rete per il login e il logout, delegando l'interazione e l'apertura del browser di sistema al pacchetto **flutter_web_auth_2**.

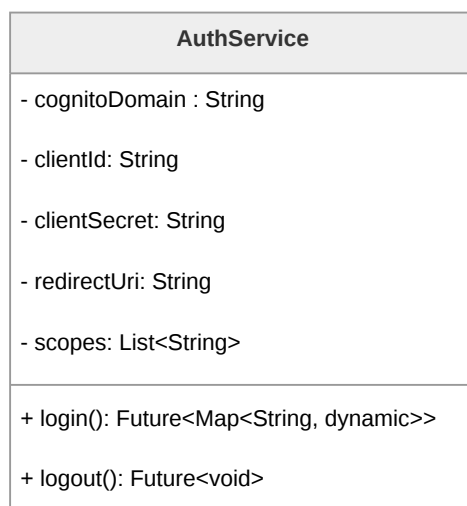


Figura 14: Diagramma della classe **AuthService**

3.4.2 Chatbot

La funzionalità del chatbot permette all'**Utente^G** di comunicare in tempo reale con un modello di linguaggio (LLM) e di gestire lo storico delle conversazioni. L'**Architettura^G**



segue rigorosamente il pattern MVVM e i principi di **Clean Architecture**^G. La struttura si avvale del pattern strutturale **Proxy** per ottimizzare il caricamento delle cronologie e di un approccio **Optimistic UI** per garantire reattività. Il sistema distingue nettamente un **Data Layer**^G (Service, **Repository**^G, API Client), un Domain Layer (interfaccia **Chat**, **ProxyChat**, **LocalChat**, **ChatMessage**) e un **UI Layer**^G (View e ViewModel).

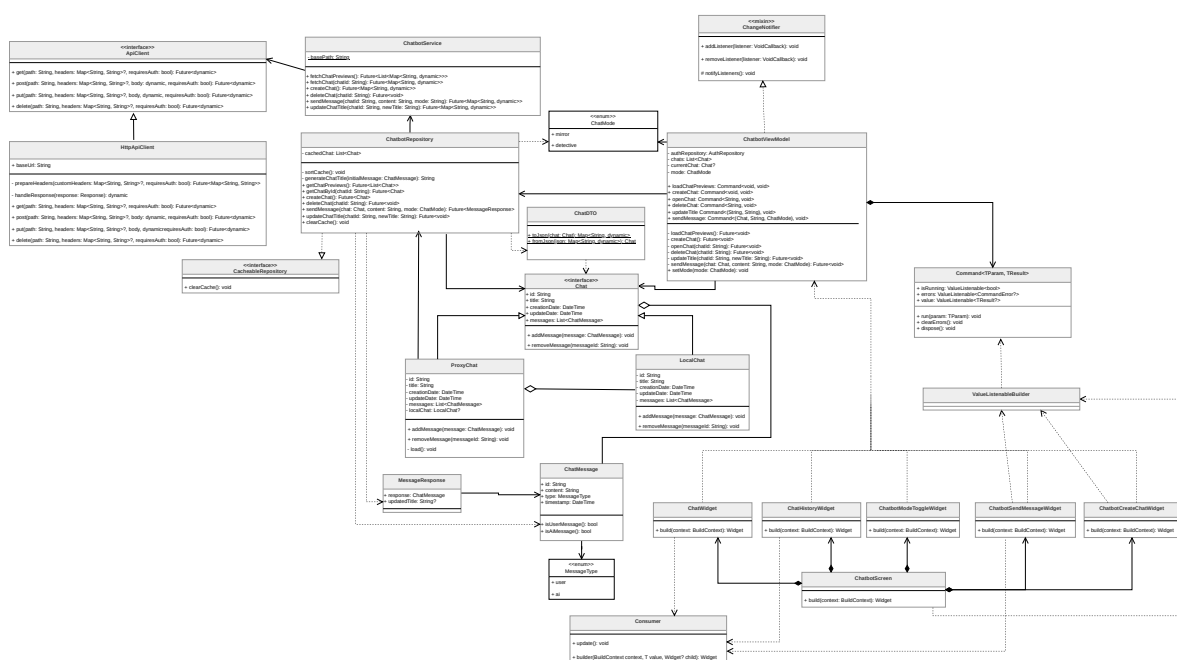


Figura 15: Diagramma delle classi relativo alle funzionalità del Chatbot

ChatbotScreen

Rappresenta la schermata principale del Chatbot all'interno del *Presentation Layer*. Funge da orchestratore visivo, componendo widget specializzati come **ChatWidget**, **ChatHistoryWidget** e **ChatbotSendMessageWidget**. Contiene esclusivamente la logica di rendering necessaria per disaccoppiare la presentazione dalla logica di business gestita dal ViewModel.

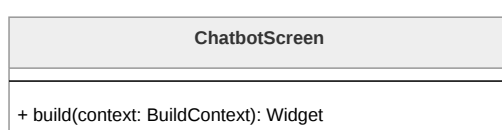


Figura 16: Diagramma della classe **ChatbotScreen**



ChatWidget

La classe **ChatWidget** è responsabile della visualizzazione della conversazione attiva. Utilizza internamente il widget **Consumer** per osservare il **ChatbotViewModel** e reagire ai cambiamenti dello stato applicativo, garantendo l'aggiornamento automatico della lista dei messaggi e dei feedback di caricamento.

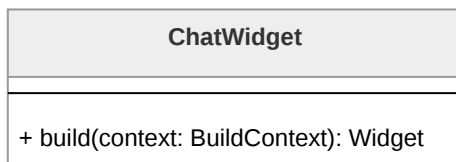


Figura 17: Diagramma della classe **ChatWidget**

ChatHistoryWidget

ChatHistoryWidget mostra l'elenco delle anteprime delle conversazioni passate. Grazie all'utilizzo del pattern **Proxy**, il widget è in grado di visualizzare i metadati delle chat (titolo, data) senza dover caricare l'intera cronologia dei messaggi. Tramite un **Consumer**, aggiorna la vista ogni volta che la lista delle anteprime nel ViewModel viene aggiornata.

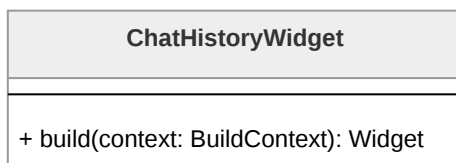


Figura 18: Diagramma della classe **ChatHistoryWidget**

ChatbotModeToggleWidget

Consente all'**Utente**^G di selezionare il contesto operativo del chatbot (**ChatMode**). Le modalità disponibili, "Specchio" e "Detective", influenzano il *system prompting* applicato lato **Backend**^G durante la generazione della risposta da parte di AWS Bedrock.

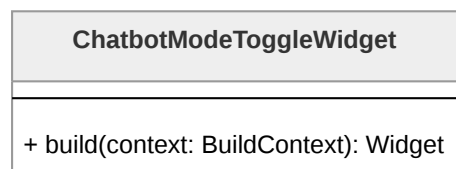


Figura 19: Diagramma della classe **ChatbotModeToggleWidget**



ChatbotSendMessageWidget

Gestisce l'input testuale e l'invio dei messaggi. Delega l'azione al ViewModel invocando il relativo Command. Il widget utilizza **ValueListenableBuilder** per monitorare lo stato di esecuzione dell'invio e mostrare un indicatore di caricamento (rotellina) durante l'attesa della risposta sincrona dal LLM.

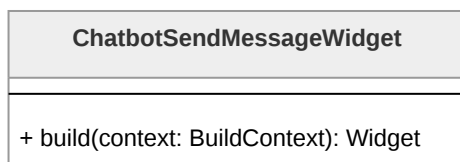


Figura 20: Diagramma della classe **ChatbotSendMessageWidget**

ChatbotCreateChatWidget

Widget dedicato all'avvio di nuove sessioni di chat. Comunica l'intenzione al ViewModel, che prepara lo stato per una nuova conversazione gestendo la creazione in modo *lazy*: la chat viene effettivamente creata sul server solo all'invio del primo messaggio.

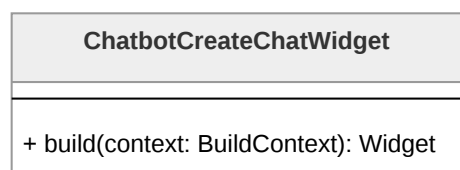


Figura 21: Diagramma della classe **ChatbotCreateChatWidget**

ChatbotViewModel

ChatbotViewModel è il nucleo della logica di presentazione. Orchestrando i vari **Command** (**sendMessage**, **openChat**, **loadChatPreviews**, **deleteChat**), gestisce la reattività della View e mantiene in memoria la modalità operativa (**ChatMode**). Implementa il mixin **ChangeNotifier** per notificare i widget osservatori ad ogni aggiornamento dello stato.

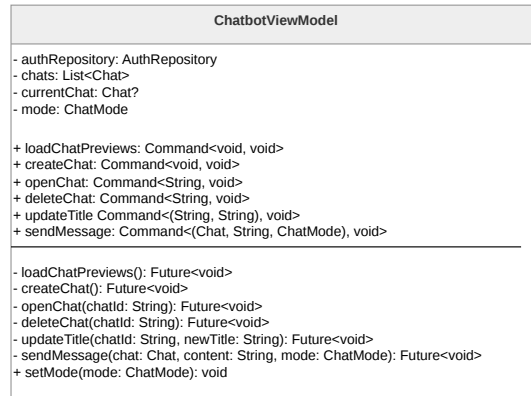


Figura 22: Diagramma della classe **ChatbotViewModel**

Chat

Interfaccia fondamentale che definisce il **Contratto^G** per una conversazione. Permette al sistema di trattare in modo polimorfico sia le anteprime leggere (**ProxyChat**) che le conversazioni complete di messaggi (**LocalChat**).

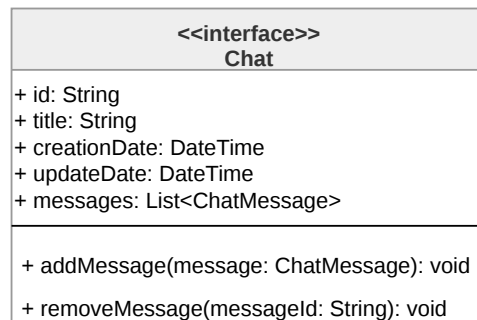
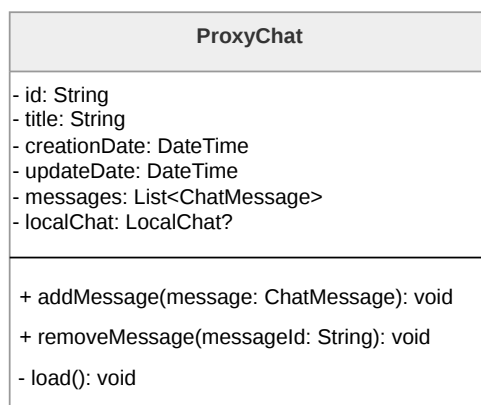


Figura 23: Diagramma della classe **Chat**

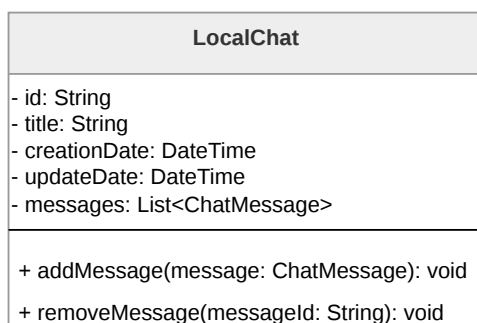
ProxyChat

Implementazione del pattern Proxy che agisce come segnaposto per una conversazione storica. Contiene solo i metadati essenziali e implementa il metodo **load()**, che interagisce con il **Repository^G** per istanziare una **LocalChat** e caricarne i messaggi solo quando l'**Utente^G** decide di aprire effettivamente la conversazione (*Lazy Loading*).

Figura 24: Diagramma della classe **ProxyChat**

LocalChat

Rappresenta l'oggetto reale nel pattern Proxy. Mantiene l'elenco completo dei **ChatMessage** scambiati e fornisce i metodi per la manipolazione della cronologia in memoria RAM durante la sessione attiva.

Figura 25: Diagramma della classe **LocalChat**

ChatMessage

Classe di dominio che modella il singolo messaggio, specificando il contenuto, il timestamp e il ruolo dell'emittente tramite l'enumerazione **MessageType**.

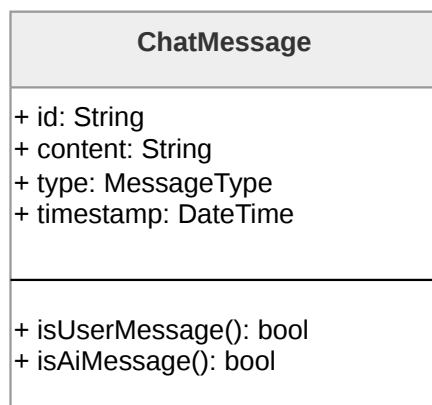


Figura 26: Diagramma della classe **ChatMessage**

ChatMode

Enumerazione che definisce i contesti di risposta del chatbot. Viene inoltrata dal View-Model fino al Service per permettere al **Backend^G** di applicare il corretto *system prompt* su Amazon Bedrock.

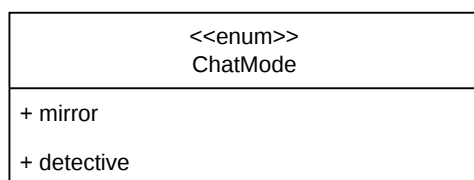


Figura 27: Diagramma della classe **ChatMode**

MessageResponse

Incapsula la risposta prodotta dall'LLM lato server. Contiene il corpo del messaggio e i metadati necessari per aggiornare la conversazione corrente nel ViewModel.

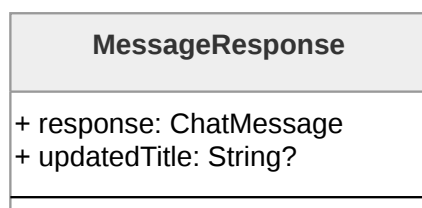


Figura 28: Diagramma della classe **MessageResponse**



ChatbotRepository

Agisce come orchestratore dei dati e implementa una cache puramente **in-memory** (**cachedPreviews**) per garantire privacy e sicurezza. Implementa strategie di **Optimistic UI** per l'invio e l'eliminazione dei messaggi, aggiornando immediatamente lo stato locale prima della conferma del server. Utilizza il **ChatbotService** per le operazioni remote e il **ChatDTO** per la mappatura dei dati.

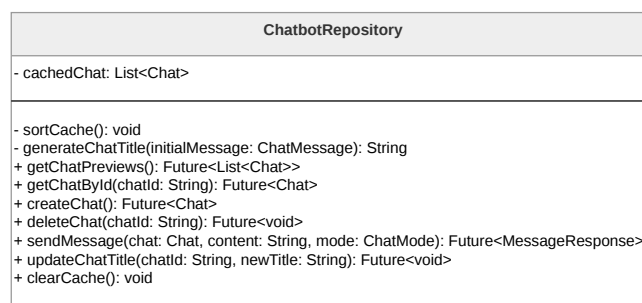


Figura 29: Diagramma della classe **ChatbotRepository**

ChatbotService

Responsabile della comunicazione con le **API REST^G** di AWS. Si occupa di invocare gli endpoint per il recupero delle chat, l'invio dei messaggi e la gestione della cronologia, restituendo risposte grezze al **Repository^G**.

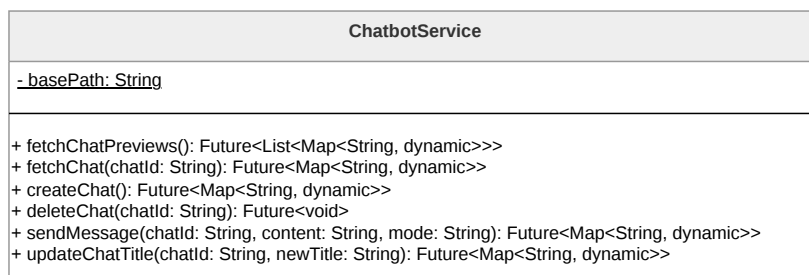
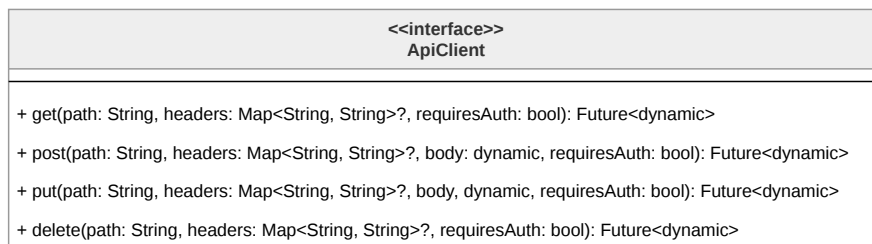


Figura 30: Diagramma della classe **ChatbotService**

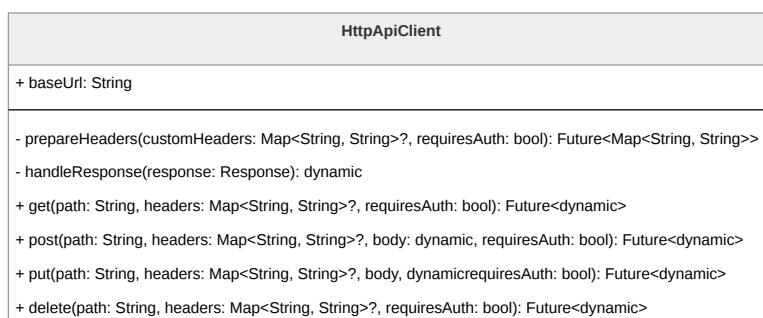
ApiClient

Interfaccia astratta che definisce il **Contratto^G** per le operazioni HTTP (**get**, **post**, **put**, **delete**). Astrae il client di rete permettendo una facile sostituzione o simulazione (mocking) del livello di comunicazione.

Figura 31: Diagramma dell'interfaccia **ApiClient**

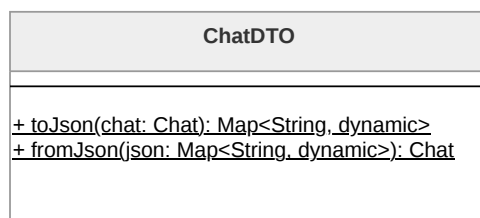
HttpApiClient

Implementazione concreta dell'interfaccia **ApiClient**. Gestisce la comunicazione effettiva tramite il pacchetto **http**, occupandosi dell'inserimento dei token di autenticazione negli header di ogni richiesta e della gestione globale delle eccezioni di rete.

Figura 32: Diagramma della classe **HttpApiClient**

ChatDTO

Gestisce la traduzione tra i modelli di dominio e le rappresentazioni JSON scambiate tramite l'API. Isola il resto del sistema dai dettagli del formato di serializzazione esterno.

Figura 33: Diagramma della classe **ChatDTO**

3.4.3 Diari

La funzionalità di diario criptato e diario fittizio permette all'**Utente^G** di utilizzare due archivi separati in cui memorizzare note, contenenti testo, immagini e/o contenuti audio,

[illegible]

DiaryAccessScreen

DiaryAccessScreen
+ build(context: BuildContext): Widget

PasswordFormWidget

51



tipo **Consumer**, osserva il **DiaryAccessViewModel** per reagire a tentativi di accesso falliti, errori di rete o stati di blocco temporaneo dovuto al superamento del limite di tentativi.

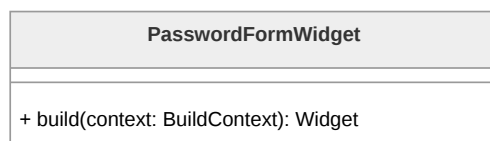


Figura 36: Diagramma della classe **PasswordFormWidget**

DiaryAccessViewModel

Coordina le operazioni di autenticazione tra UI e **Data Layer**^G. Implementa il Command **login** per validare le credenziali tramite il **Repository**^G. Una volta ottenuto l'accesso, istruisce il sistema sulla natura della sessione attiva (**fake_diary** o **real_diary**). Essendo un **ChangeNotifier**, notifica i widget interessati ad ogni cambiamento di stato.

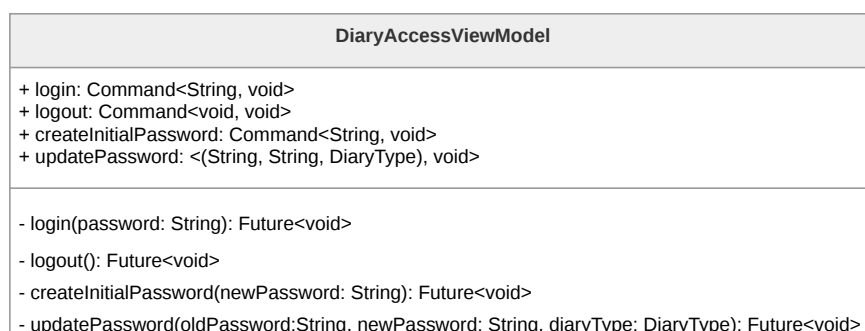


Figura 37: Diagramma della classe **DiaryAccessViewModel**

DiaryAccountRepository

Funge da strato di astrazione per la gestione dell'account del diario. Si occupa di invocare il metodo **clarifyAccessResult** per validare la password tramite il Service e trasforma le risposte grezze in oggetti **DiaryAccessResult**. È responsabile dell'inizializzazione della **DiarySession** globale in caso di successo.

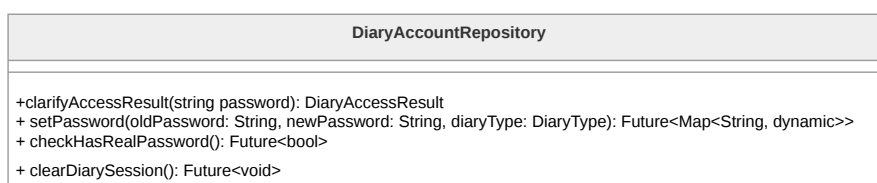


Figura 38: Diagramma della classe **DiaryAccountRepository**

DiaryAccountService

Rappresenta lo strato infrastrutturale responsabile della comunicazione con le API remote



per la **Validazione^G** delle password e la gestione dei parametri di sicurezza del diario. Restituisce dati grezzi al **Repository^G** per la successiva elaborazione.

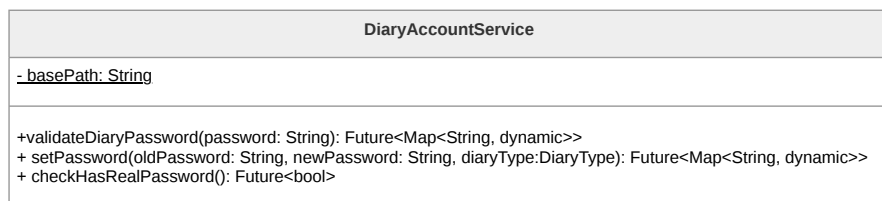


Figura 39: Diagramma della classe **DiaryAccountService**

DiaryAccessResult

Enumerazione che classifica l'esito di un tentativo di sblocco. Include stati per l'accesso garantito (**real_diary**, **fake_diary**), il diniego (**error**) o il blocco per rate-limiting (**too_many_attempts**).

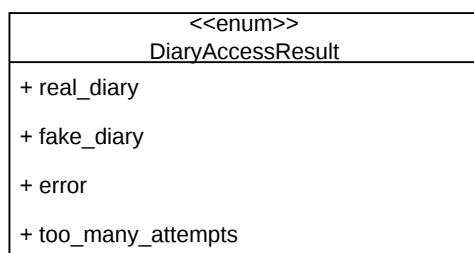


Figura 40: Diagramma della classe **DiaryAccessResult**

DiaryScreen

Gestisce l'interfaccia principale del diario sbloccato. Per garantire la *Plausible Deniability*, la struttura visiva è identica per entrambi i tipi di diario, includendo i widget **NoteListWidget** e **NoteActionsWidget**. La gestione dello stato operativo è delegata a **DiaryViewModel**.

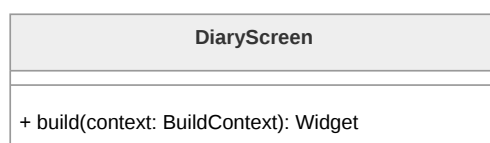


Figura 41: Diagramma della classe **DiaryScreen**

NoteListWidget

Visualizza l'elenco delle note caricate per la sessione corrente. Utilizza il **Consumer** per



osservare il **DiaryViewModel** e aggiornare la lista in tempo reale a seguito di aggiunte o eliminazioni. Supporta il caricamento pigro delle note grazie al pattern Proxy.

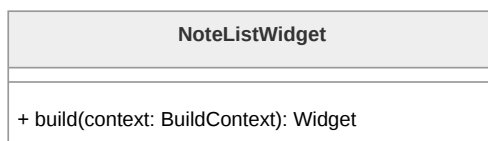


Figura 42: Diagramma della classe **NoteListWidget**

NoteActionsWidget

Espone le interazioni per la creazione di nuove note o l'attivazione della modalità di editing. Delega l'esecuzione dei comandi al ViewModel, mantenendo il disaccoppiamento tra UI e dominio.

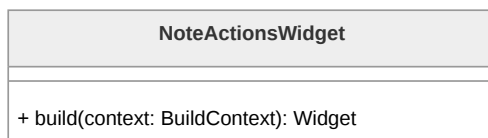


Figura 43: Diagramma della classe **NoteActionsWidget**

NoteEditorWidget

Gestisce l'interfaccia di editing granulare della nota selezionata. Permette la manipolazione dei singoli **NoteElement** (testo, audio, immagini, file) e implementa logiche di *Optimistic UI* per riflettere immediatamente le modifiche locali prima della persistenza sul server.

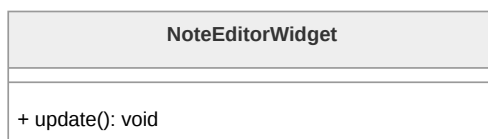


Figura 44: Diagramma della classe **NoteEditorWidget**

DiaryViewModel

Nucleo della logica di presentazione per la gestione del diario. Mantiene in memoria RAM la lista delle note correnti e gestisce i comandi CRUD (**addNote**, **updateNote**, **deleteNote**). Utilizza il **NoteRepository** per la persistenza dei dati e dei file multimediali, notificando la UI tramite **ChangeNotifier**.

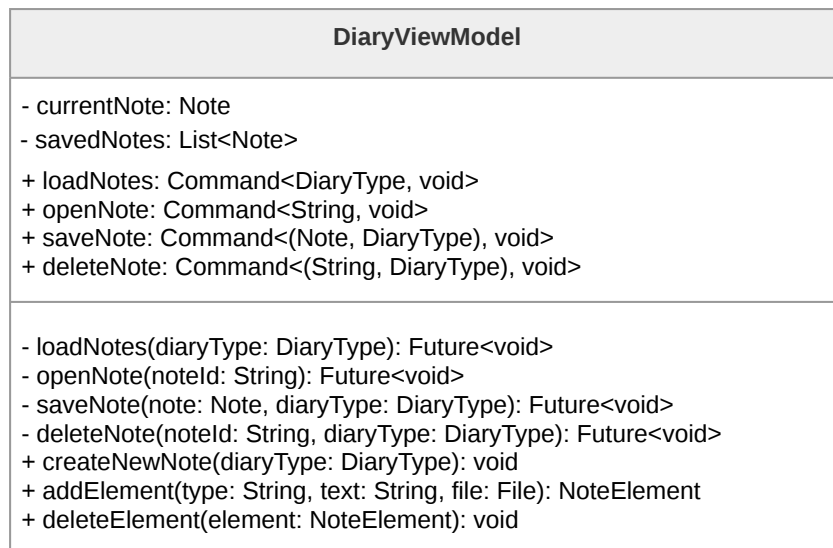


Figura 45: Diagramma della classe **DiaryViewModel**

DiarySession

Classe Singleton responsabile del mantenimento dello stato della sessione attiva esclusivamente in memoria RAM. Tiene traccia del **DiaryType** e del token di accesso, garantendo che le informazioni sensibili vengano perse alla chiusura del **Processo**^G dell'applicazione per massimizzare la sicurezza.

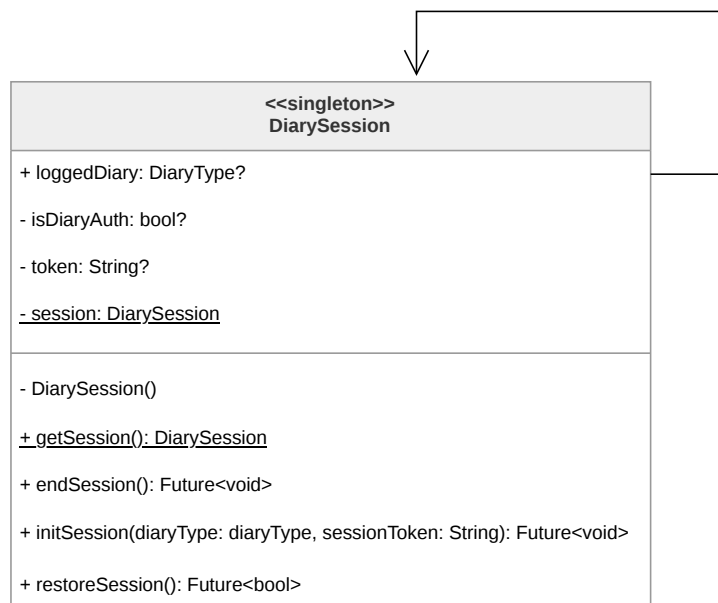


Figura 46: Diagramma della classe **DiarySession**



DiaryType

Enumerazione che definisce le due tipologie di archivio disponibili: **fake_diary** e **real_diary**. È un parametro fondamentale utilizzato per instradare correttamente le richieste verso gli endpoint del server.

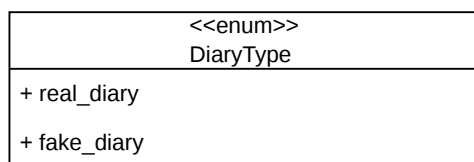


Figura 47: Diagramma della classe **DiaryType**

Note

Interfaccia che definisce il **Contratto^G** per una nota di diario. Supporta il pattern Proxy per ottimizzare le risorse di sistema durante la visualizzazione di elenchi di note voluminosi.

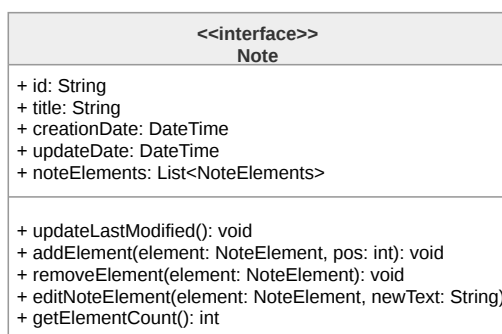
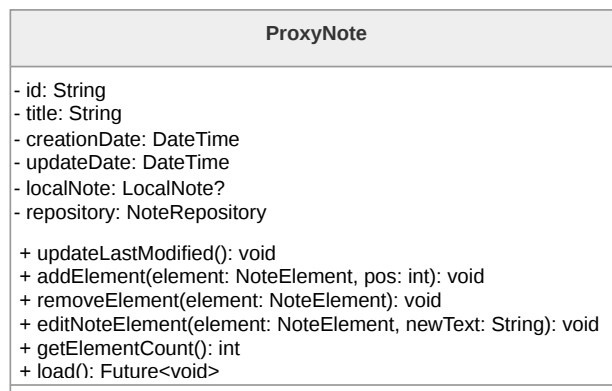


Figura 48: Diagramma della classe **Note**

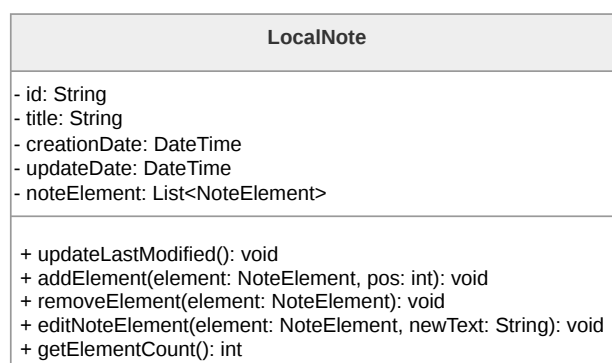
ProxyNote

Realizza il pattern Proxy ritardando il caricamento completo della cronologia degli elementi di una nota. Mantiene i metadati di base e delega le operazioni a un oggetto **LocalNote** solo in caso di accesso effettivo ai contenuti (*Lazy Loading*).

Figura 49: Diagramma della classe **ProxyNote**

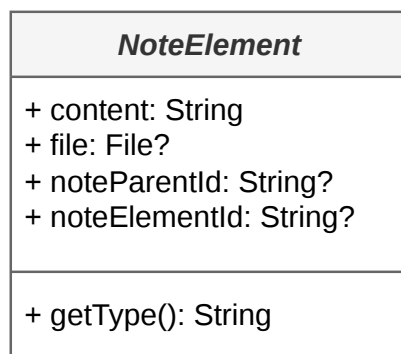
LocalNote

Implementazione reale dell'interfaccia **Note**. Rappresenta l'oggetto caricato in memoria che contiene la lista polimorfica di **NoteElement** che compongono il corpo della nota.

Figura 50: Diagramma della classe **LocalNote**

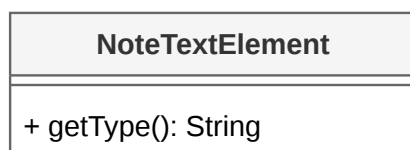
NoteElement

Classe base astratta che modella un blocco di contenuto all'interno della nota. Definisce il metodo **getType()** fondamentale per il rendering e la serializzazione polimorfica degli elementi.

Figura 51: Diagramma della classe **NoteElement**

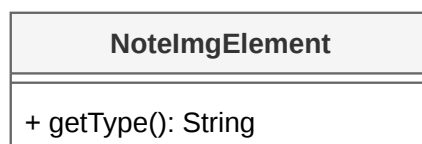
NoteTextElement

Specializzazione di **NoteElement** per la gestione di contenuti testuali all'interno dell'editor a blocchi.

Figura 52: Diagramma della classe **NoteTextElement**

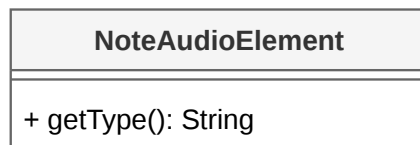
NoteImageElement

Specializzazione di **NoteElement** per i contenuti grafici. Gestisce il riferimento locale dell'immagine per l'Optimistic UI e l'URL remoto post-caricamento.

Figura 53: Diagramma della classe **NoteImageElement**

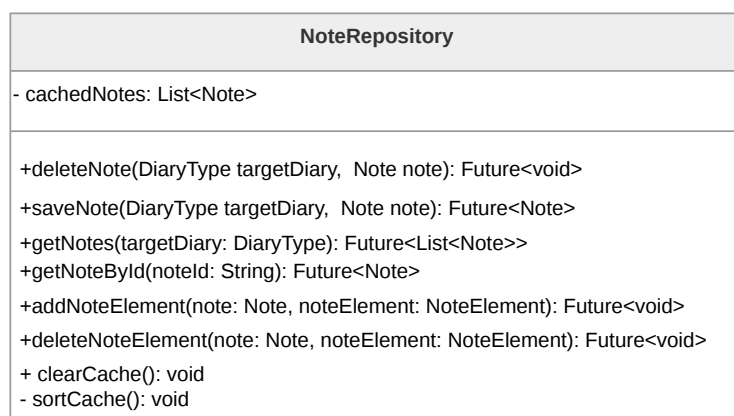
NoteAudioElement

Specializzazione di **NoteElement** per i messaggi vocali e le registrazioni audio, contenente i metadati necessari alla riproduzione.

Figura 54: Diagramma della classe **NoteAudioElement**

NoteRepository

Orchestra l'accesso ai dati delle note e dei file multimediali. Implementa logiche di cache in-memory (**cachedNotes**) e gestisce l'upload immediato dei media (**uploadFile**) tramite il Service, mantenendo lo stato locale aggiornato prima del completamento dell'operazione remota.

Figura 55: Diagramma della classe **NoteRepository**

NoteService

Livello infrastrutturale per la comunicazione con gli endpoint delle note. Gestisce operazioni CRUD remote e l'invio/ricezione di byte per i file multimediali. Restituisce i dati grezzi al **Repository**^G per la trasformazione in modelli di dominio.

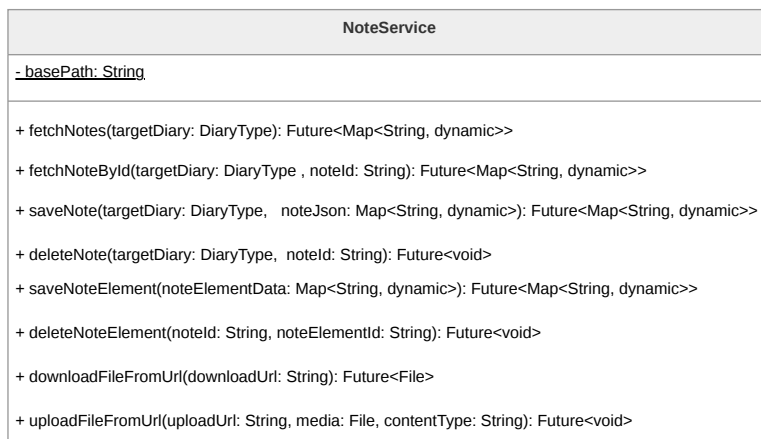


Figura 56: Diagramma della classe **NoteService**

NoteDTO

La classe **NoteDTO** è un Data Transfer Object responsabile della serializzazione e deserializzazione delle note in formato JSON. Per la gestione del corpo della nota, delega la trasformazione della lista di elementi a **NoteElementDTO**, ricomponendo l'oggetto **Note** completo per il Domain Layer.

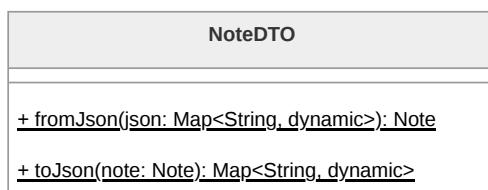


Figura 57: Diagramma della classe **NoteDTO**

NoteElementDTO

La classe **NoteElementDTO** implementa la logica di deserializzazione polimorfica per i blocchi multimediali. Funge da Factory: analizzando il campo identificativo del tipo nel JSON, esegue uno *switch* per istanziare la sottoclasse corretta di **NoteElement** (**NoteTextElement**, **NoteAudioElement**, **NoteImageElement** o **NoteFileElement**).

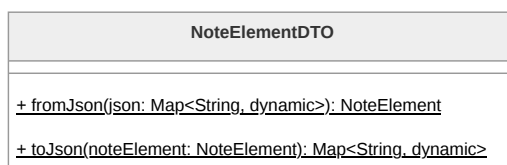


Figura 58: Diagramma della classe **NoteElementDTO**



3.4.4 Contatti Fidati

La funzionalità dei Contatti Fidati permette all'**Utente^G** di gestire una lista di persone di fiducia da avvisare in caso di inattività tramite il sistema di *Dead Man's Switch* o in caso di emergenza tramite segnale SOS. Analogamente al resto dell'applicazione, la funzionalità è implementata secondo il pattern MVVM e le linee guida architetturali di Flutter, distinguendo un **Data Layer^G**, composto da Service e **Repository^G**, e un **UI Layer^G**, composto da View e ViewModel.

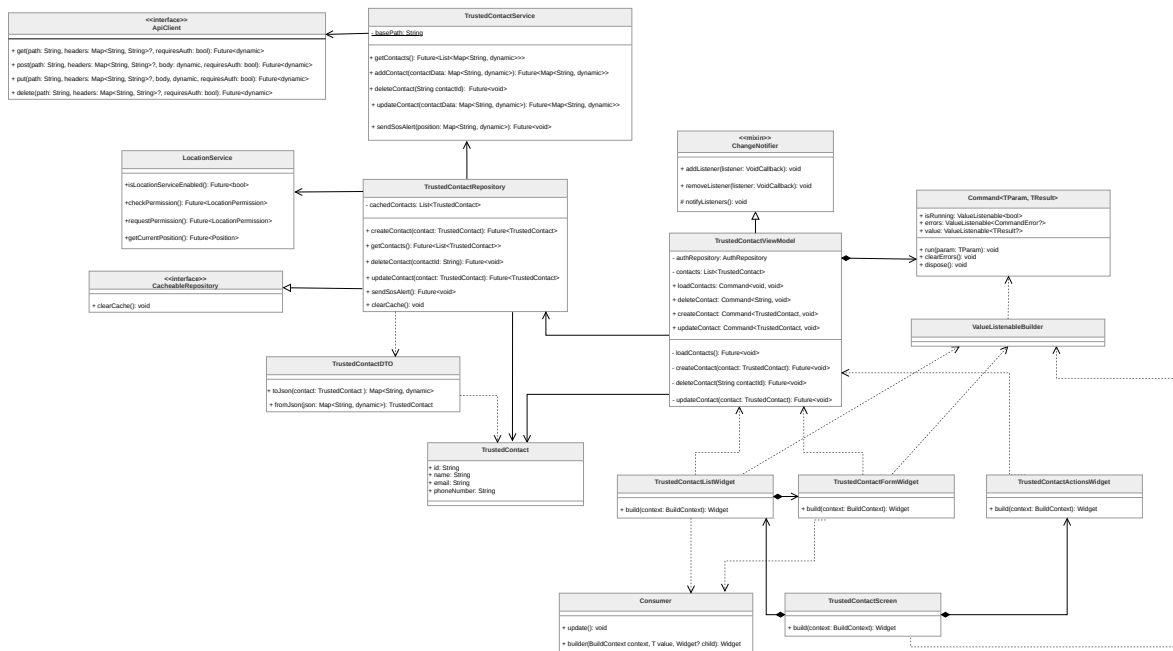


Figura 59: Diagramma delle classi relativo alle funzionalità dei Contatti Fidati

TrustedContactScreen

La classe **TrustedContactScreen** rappresenta la schermata principale dedicata alla gestione dei contatti fidati all'interno del *Presentation Layer*. Essa funge da compositore, unendo i vari widget presenti e ordinandoli secondo il layout stabilito, delegando la logica di stato ai componenti sottostanti.

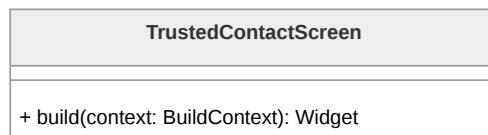


Figura 60: Diagramma della classe **TrustedContactScreen**

TrustedContactListWidget

La classe **TrustedContactListWidget** è responsabile della visualizzazione dell'elenco dei



contatti attualmente salvati. Attraverso l'utilizzo del componente **Consumer**, osserva il ViewModel per reagire in modo reattivo ai cambiamenti della lista dei contatti e aggiornare dinamicamente l'interfaccia grafica senza ricostruire l'intera schermata.

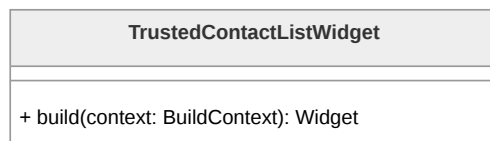


Figura 61: Diagramma della classe **TrustedContactListWidget**

TrustedContactFormWidget

TrustedContactFormWidget gestisce i campi di input necessari per l'inserimento o la modifica dei dati di un contatto fidato. Il widget integra componenti di osservazione come **Consumer** per monitorare lo stato di **Validazione^G** e gli errori provenienti dal ViewModel, e **ValueListenableBuilder** per gestire lo stato di esecuzione dei Command, garantendo un feedback visivo durante le operazioni asincrone.

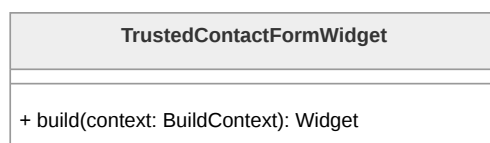


Figura 62: Diagramma della classe **TrustedContactFormWidget**

TrustedContactActionsWidget

Questo widget raggruppa i bottoni e le azioni specifiche per la gestione dei contatti fidati, come l'aggiunta di nuovi elementi o l'attivazione di procedure di modifica. Comunica le intenzioni dell'**Utente^G** al ViewModel invocando i relativi Command e reagisce al loro stato interno per gestire correttamente la disponibilità delle interazioni durante i processi di caricamento.



Figura 63: Diagramma della classe **TrustedContactActionsWidget**

TrustedContactViewModel

TrustedContactViewModel rappresenta il componente centrale per la gestione dello stato della funzionalità. Esso mantiene la lista dei contatti (**contacts**) e orchestra le operazioni asincrone attraverso l'uso del pattern Command (**loadContacts**, **createContact**,



`updateContact`, `deleteContact`). La gestione degli stati di caricamento e degli eventuali fallimenti è incapsulata direttamente all'interno degli oggetti Command stessi, che espongono tali informazioni alla View in modo reattivo. Il ViewModel notifica i cambiamenti generali ai widget in ascolto tramite `ChangeNotifier`.

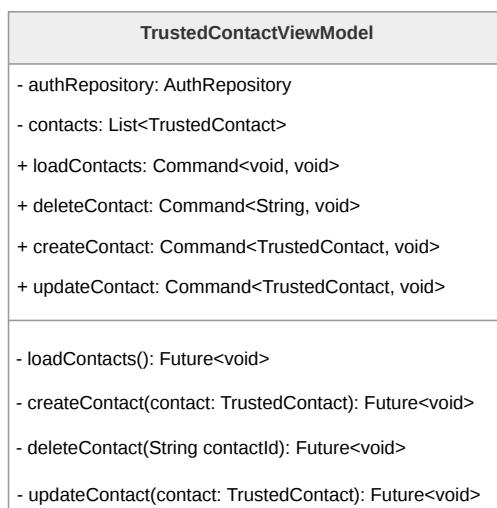


Figura 64: Diagramma della classe `TrustedContactViewModel`

TrustedContact

`TrustedContact` è la classe di dominio che modella il singolo contatto fidato. Contiene le informazioni anagrafiche e di recapito essenziali: identificativo unico (`id`), `name`, `email` e `phoneNumber`.



Figura 65: Diagramma della classe `TrustedContact`

TrustedContactRepository

`TrustedContactRepository` astrae la sorgente dei dati e implementa le logiche di caching tramite `cachedContacts`. Per l'eliminazione dei contatti adotta una strategia di *Optimistic UI* con rollback in caso di errore. Inoltre, orchestra l'invio del segnale SOS tramite il metodo `sendSosAlert()`, interagendo con il `LocationService` per acquisire la posizione geografica prima di inoltrare la richiesta al Service.

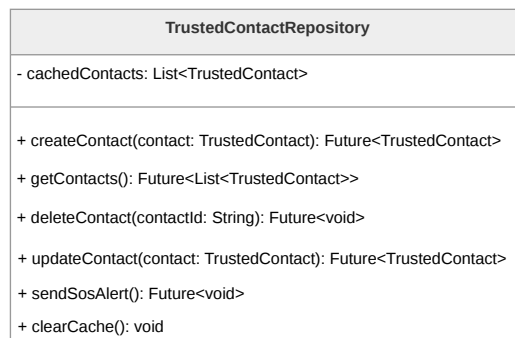


Figura 66: Diagramma della classe **TrustedContactRepository**

TrustedContactService

TrustedContactService si occupa delle chiamate API verso il **Backend^G**. Oltre alle operazioni CRUD standard (**getContacts**, **addContact**, **deleteContact**, **updateContact**), implementa il metodo **sendSosAlert(position)** per l'invio dei dati di emergenza e della posizione geografica ai server.

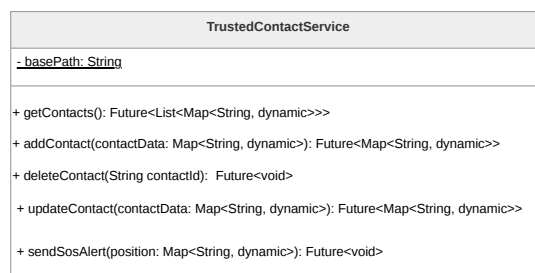


Figura 67: Diagramma della classe **TrustedContactService**

TrustedContactDTO

TrustedContactDTO gestisce la serializzazione e deserializzazione dei dati tramite i metodi **toJson** e **fromJson**. Garantisce che i dati scambiati con le API siano correttamente mappati da e verso gli oggetti di dominio **TrustedContact**.



Figura 68: Diagramma della classe **TrustedContactDTO**



3.4.5 Luoghi Sicuri

La funzionalità dei Luoghi Sicuri permette all'**Utente^G** di visualizzare geograficamente, tramite un'interfaccia cartografica, i punti di interesse rilevanti per la sicurezza personale (es. ospedali, centri anti-violenza, forze dell'ordine). La funzionalità è architettata secondo il pattern MVVM e aderisce ai principi della **Clean Architecture^G**. Il sistema distingue un **Data Layer^G**, responsabile del fetching e del caching locale dei dati provenienti da un Bucket S3 dell'infrastruttura AWS, e un **UI Layer^G** reattivo, implementato sfruttando la libreria **flutter_map** per il rendering cartografico basato su OpenStreetMap.

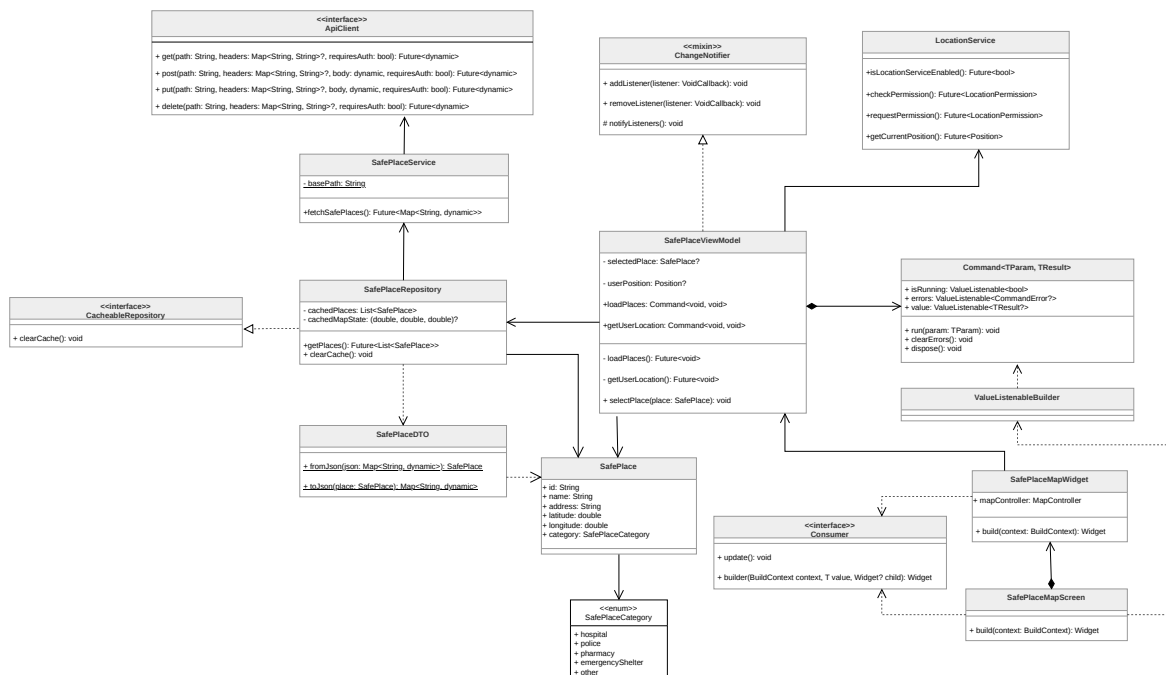


Figura 69: Diagramma delle classi complessivo della funzionalità Luoghi Sicuri

SafePlaceMapScreen La classe **SafePlaceMapScreen** rappresenta la schermata principale nel *Presentation Layer*. Funge da orchestratore (compositore), organizzando il layout della pagina e delegando il rendering cartografico al widget figlio specializzato. Si occupa inoltre della gestione dell'interfaccia passiva di dettaglio (tramite *BottomSheet* testuali, per evitare *context switch* non sicuri) e di coordinare l'ascolto reattivo granulare (tramite **ValueListenableBuilder**) degli stati asincroni, garantendo la *graceful degradation* (es. mostrando un avviso se il recupero della posizione GPS fallisce, senza bloccare la mappa).

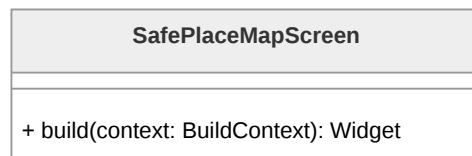


Figura 70: Diagramma della classe **SafePlaceMapScreen**

SafePlaceMapWidget Il componente **SafePlaceMapWidget** incapsula la logica di interazione diretta con la mappa digitale. Implementando l'interfaccia **Consumer**, si mette in ascolto dei cambiamenti di stato globali propagati dal ViewModel. Al suo interno, utilizza i componenti forniti da **flutter_map** (**TileLayer** e **MarkerLayer**) per renderizzare la cartografia e mappare le istanze di **SafePlace** in **Marker** visivi. Inoltre, si occupa di intercettare gli eventi di interazione (pan e zoom) per salvare costantemente lo stato della sessione nel ViewModel.

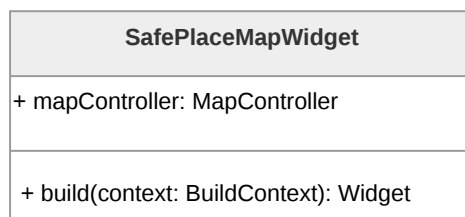


Figura 71: Diagramma della classe **SafePlaceMapWidget**

SafePlaceViewModel Il **SafePlaceViewModel** agisce come nucleo della gestione dello stato (*State Management*^G). Eredita da **ChangeNotifier** per la notifica degli stati sincroni e passivi (come la selezione di un luogo tramite la variabile **selectedPlace**). Per la gestione asincrona delle operazioni di rete e hardware, adotta il pattern Command esponendo le proprietà **loadPlaces** e **getUserLocation**. Questi comandi incapsulano internamente i **ValueListenable** per gli stati di caricamento (**isRunning**) ed errore (**errors**), garantendo una reattività granulare della View senza causare *rebuild* dell'intera schermata.

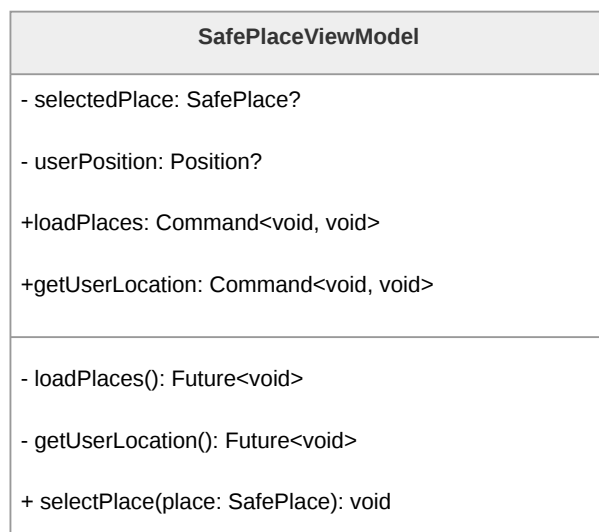


Figura 72: Diagramma della classe **SafePlaceViewModel**

SafePlace rappresenta l'entità di dominio fondamentale che modella un punto di interesse all'interno del sistema. La classe incapsula gli attributi identificativi del luogo, tra cui il nome e l'indirizzo testuale, oltre alle proprietà numeriche **latitude** e **longitude**. Queste ultime sono essenziali per il posizionamento spaziale dei **Marker** sulla superficie cartografica gestita da **flutter_map**. La classe mantiene inoltre un riferimento alla propria categoria di appartenenza, delegando ad essa la semantica tipologica del luogo.

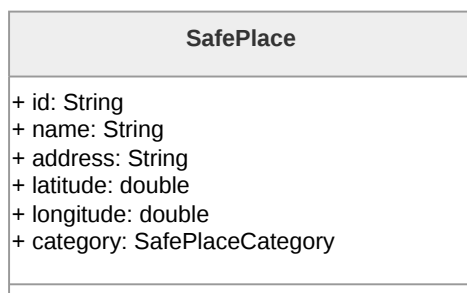


Figura 73: Diagramma della classe **SafePlace**

SafePlaceCategory L'enumerazione **SafePlaceCategory** definisce la classificazione tipologica dei luoghi sicuri gestiti dall'applicazione.

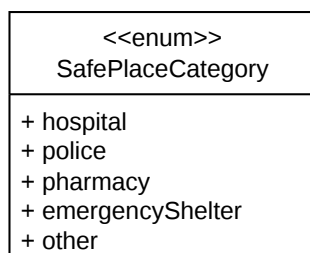


Figura 74: Diagramma della classe **SafePlaceCategory**

SafePlaceRepository Il **SafePlaceRepository** funge da *Single Source of Truth* per la sessione corrente, estendendo l'interfaccia **CacheableRepository**. Oltre a convertire i dati grezzi in oggetti di dominio tramite il DTO, incapsula una logica di memorizzazione in cache (**cachedPlaces**). Salva inoltre le coordinate e lo zoom correnti (**cachedMapState**) per ripristinare il contesto geografico tra diverse navigazioni nella schermata, ottimizzando le prestazioni e abbattendo i costi associati alle chiamate di rete superflue verso il **Backend^G Serverless^G**.

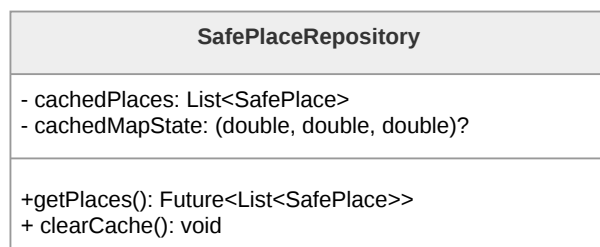


Figura 75: Diagramma della classe **SafePlaceRepository**

LocationService Appartenente al livello infrastrutturale, **LocationService** astrae l'interazione con l'hardware GPS del dispositivo mobile. Offre i metodi essenziali per controllare l'abilitazione del sensore (**isLocationServiceEnabled**) e per richiedere in modo sicuro i permessi all'**Utente^G** (**checkPermission**, **requestPermission**), permettendo al ViewModel di gestire preventivamente gli accessi negati.

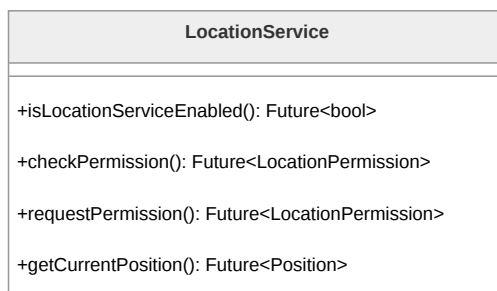


Figura 76: Diagramma della classe **LocationService**

SafePlaceService **SafePlaceService** è incaricato delle comunicazioni HTTP esterne. Sfruttando **ApiClient**, inoltra le richieste per ottenere il dataset dei luoghi sicuri tramite una funzione Lambda esposta da API Gateway. Architetturealmente, l'infrastruttura **Backend^G** preleva il documento JSON statico da un Bucket S3, minimizzando i tempi di latenza.

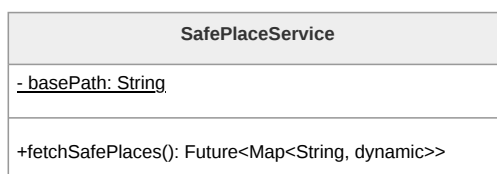


Figura 77: Diagramma della classe **SafePlaceService**

SafePlaceDTO La classe **SafePlaceDTO** (*Data Transfer Object*) isola la logica di serializzazione e deserializzazione (**fromJson**, **toJson**). Mappa le chiavi della **Payload^G** JSON scaricata dal **Backend^G** nei tipi primitivi Dart attesi dal dominio, disaccoppiando la persistenza esterna dalla **Business Logic^G** interna.

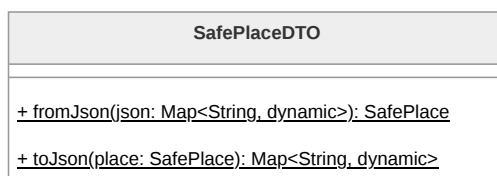


Figura 78: Diagramma della classe **SafePlaceDTO**

3.4.6 Dead Man's Switch

La funzionalità del *Dead Man's Switch* rappresenta un sistema di sicurezza critico che invia avvisi automatici in caso di inattività prolungata dell'**Utente^G**. L'**Architettura^G** frontend è focalizzata sulla gestione sicura e reattiva della configurazione di tale sistema.



Il frontend non interagisce direttamente con servizi di posta elettronica o code di schedulazione, ma demanda la gestione dei timer e l'invio delle email al **Backend^G**, garantendo l'affidabilità del servizio anche ad applicazione chiusa. Il conteggio dell'inattività viene azzerato tramite un segnale di *heartbeat*, il cui innesco primario è delegato a componenti esterne (come l'avvenuto login). Analogamente al resto dell'applicazione, la funzionalità implementa il pattern MVVM e le linee guida della *Clean Architecture^G*.

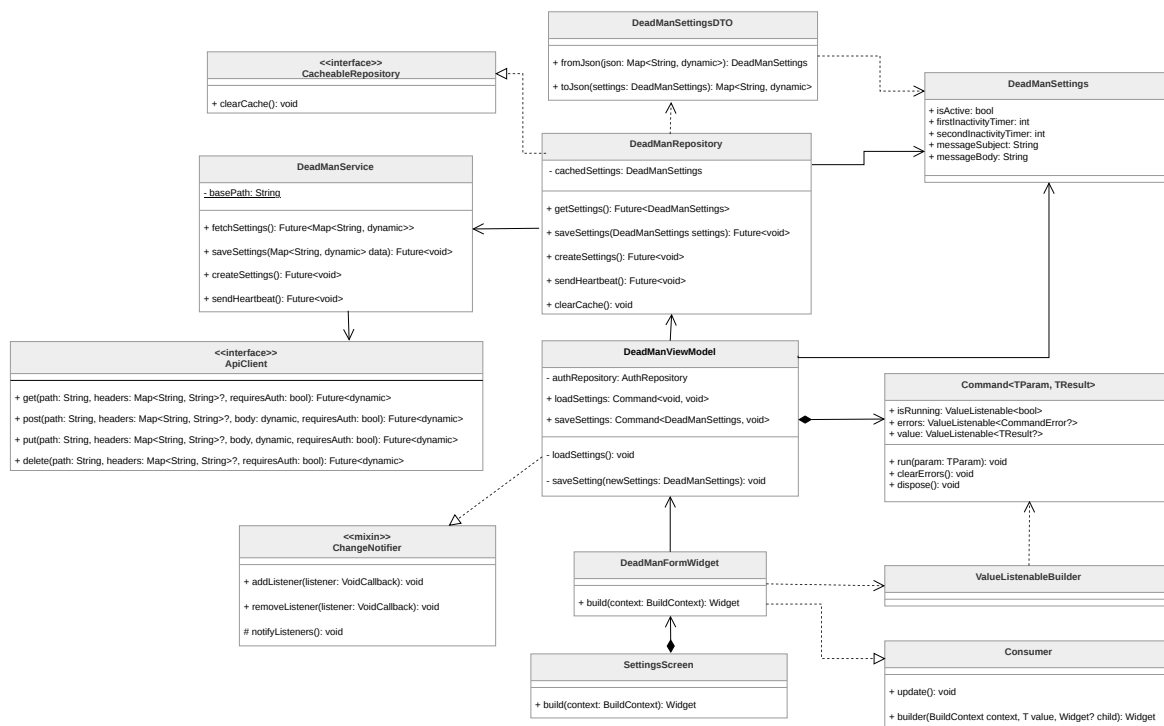


Figura 79: Diagramma delle classi relativo alla funzionalità del Dead Man's Switch

SettingsScreen La classe **SettingsScreen** rappresenta la schermata principale che ospita il pannello di controllo per la sicurezza. Funge da compositore, strutturando il layout della pagina e integrando i form interattivi, delegando al **DeadManViewModel** la reattività agli stati asincroni e la **Persistence Logic^G**.

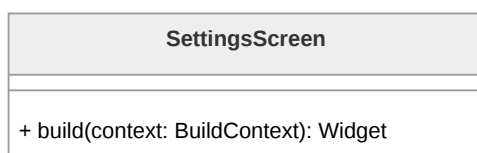


Figura 80: Diagramma della classe **SettingsScreen**

DeadManFormWidget Questo componente incapsula gli elementi interattivi dell'interfaccia, come lo switch per l'attivazione del servizio, i campi di input per i timer e le



aree di testo per i messaggi di emergenza. È implementato come widget con stato locale (*StatefulWidget*) per gestire in modo efficiente lo stato transitorio della form durante la digitazione, evitando onerosi *rebuild* del ViewModel. Si occupa inoltre di disabilitare visivamente e bloccare i campi di input quando l'**Utente^G** sceglie di disattivare il servizio.



Figura 81: Diagramma della classe **DeadManFormWidget**

DeadManViewModel **DeadManViewModel** orchestra la logica di business relativa alla configurazione del servizio. Abbandonando la gestione manuale di flag di caricamento generici, adotta il pattern Command esponendo le proprietà **loadSettings** e **saveSettings**. Questi comandi incapsulano autonomamente gli stati asincroni dell'operazione, permettendo alla View di reagirvi in modo granulare. Riceve i dati dalla form solo al momento del salvataggio, delegando al **Repository^G** la comunicazione dei nuovi parametri temporali e testuali.

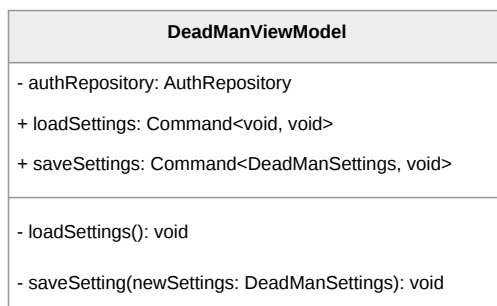


Figura 82: Diagramma della classe **DeadManViewModel**

DeadManSettings **DeadManSettings** è l'entità di dominio che modella le preferenze del *Dead Man's Switch*. Contiene gli attributi necessari a configurare la logica di allarme: uno stato booleano di attivazione (**isActive**), i valori temporali che definiscono le due distinte soglie di inattività (**firstInactivityTimer** e **secondInactivityTimer**), nonché l'oggetto (**messageSubject**) e il corpo (**messageBody**) del messaggio testuale.

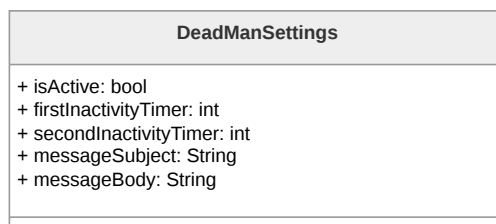


Figura 83: Diagramma della classe **DeadManSettings**

DeadManRepository Il **DeadManRepository** astrae la persistenza e il recupero delle configurazioni, agendo come *Single Source of Truth* temporanea in quanto implementazione di **CacheableRepository**. Utilizza **DeadManService** per recuperare o inviare le preferenze e per instradare richieste accessorie come la creazione iniziale (**createSettings**) o la segnalazione di attività vitale (**sendHeartbeat**). Si occupa inoltre di convertire le risposte grezze di rete in oggetti di dominio sicuri tramite l'apposito DTO.

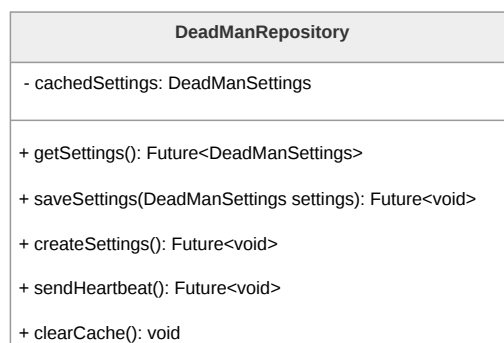


Figura 84: Diagramma della classe **DeadManRepository**

DeadManService Appartenente al **Data Layer^G**, **DeadManService** si occupa della comunicazione HTTP/REST con le API del **Backend^G**. Fornisce i metodi necessari per effettuare le operazioni CRUD sulle configurazioni (**fetchSettings**, **saveSettings**, **createSettings**) e per l'invio effettivo del segnale vitale dell'**Utente^G** verso i server AWS (**sendHeartbeat**).

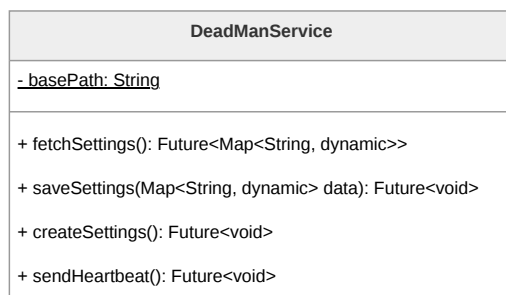


Figura 85: Diagramma della classe **DeadManService**

DeadManSettingsDTO Il **DeadManSettingsDTO** (*Data Transfer Object*) gestisce la logica di serializzazione bidirezionale. Consente di mappare coerentemente i campi della **Payload^G** JSON in istanze del dominio, assicurando che la comunicazione tra l'app Flutter e il **Backend^G** rispetti un rigido **Contratto^G** dati per i nuovi parametri di configurazione temporali e testuali.

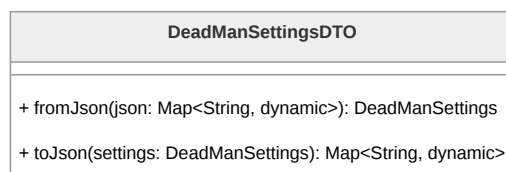


Figura 86: Diagramma della classe **DeadManSettingsDTO**

3.4.7 Materiale Informativo

La funzionalità del Materiale Informativo consente all'**Utente^G** di consultare risorse educative, normative di legge e contatti utili per la sicurezza personale. A livello architetturale, questa sezione condivide la medesima impostazione ibrida e ottimizzata della Mappa dei Luoghi Sicuri. Aderisce rigorosamente al pattern MVVM e ai principi della **Clean Architecture^G**, prevedendo un disaccoppiamento netto tra la logica di fetching dei dati statici per massimizzare la scalabilità), il caching in locale e la logica di presentazione reattiva.

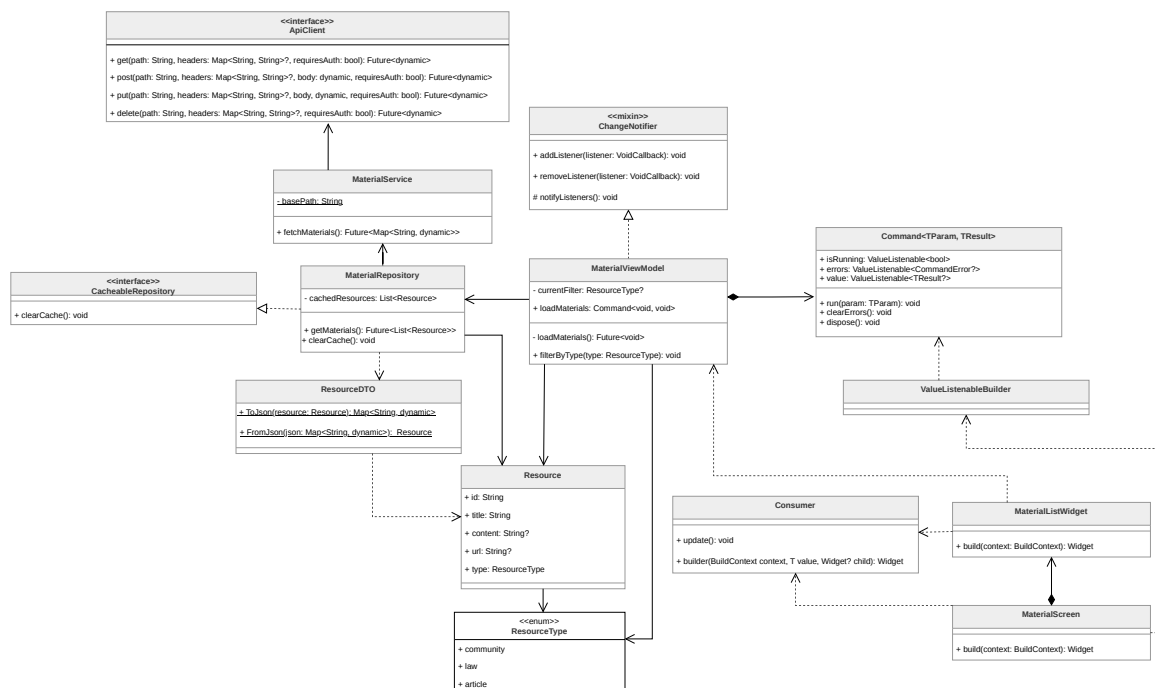


Figura 87: Diagramma delle classi complessivo della funzionalità Materiale Informativo

MaterialScreen La classe **MaterialScreen** rappresenta il punto d'ingresso per la fruizione delle risorse all'interno del *Presentation Layer*. Agisce come compositore del layout principale: organizza la barra di navigazione, delega la visualizzazione degli stati asincroni (caricamento, errore) e ospita i componenti di interazione rapida, che permettono all'**Utente**^G di selezionare agilmente le categorie di interesse senza bloccare l'interfaccia.

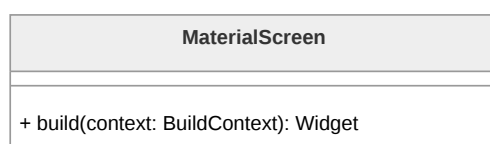


Figura 88: Diagramma della classe **MaterialScreen**

MaterialListWidget Il **MaterialListWidget** è incaricato della renderizzazione della lista di risorse. Sfruttando un approccio ibrido, si mette in ascolto tramite **Consumer** della lista filtrata fornita dal ViewModel. La visualizzazione adotta strategie UX differenziate in base alla natura del dato: per i contenuti testuali interni (**content**) utilizza il componente **ExpansionTile**, permettendo la lettura inline senza perdere il contesto della lista; per i riferimenti esterni (**url**) utilizza il pacchetto nativo **url_launcher**, delegando l'apertura del link al browser di sistema per garantire la massima sicurezza ed evitare *context switch* non controllati all'interno dell'app.

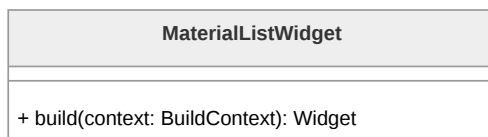


Figura 89: Diagramma della classe **MaterialListWidget**

MaterialViewModel Il **MaterialViewModel** gestisce la **Presentation Logic^G** e lo stato dell'interfaccia. Espone il comando asincrono **loadMaterials** (basato sul pattern Command) per governare il primo fetch dei dati tramite **ValueListenable**. La caratteristica peculiare di questo ViewModel è la gestione totalmente client-side del filtraggio tramite la variabile **currentFilter** e il metodo sincrono **filterByType**: operando sulla lista già mantenuta in memoria, le transizioni di interfaccia risultano istantanee, azzerando la latenza di rete e ottimizzando i costi infrastrutturali.

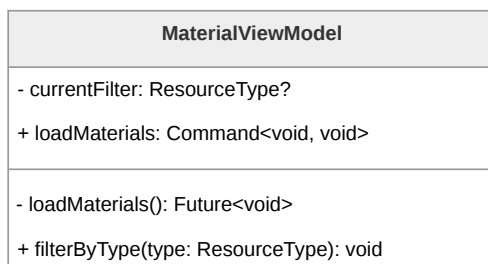


Figura 90: Diagramma della classe **MaterialViewModel**

Resource **Resource** modella la singola entità di dominio del materiale informativo. È progettata per essere flessibile, contenendo campi obbligatori di identificazione (**id**, **title**, **type**) e campi opzionali mutualmente esclusivi nella logica applicativa: **content** (per testi enciclopedici brevi da mostrare internamente) e **url** (per reindirizzamenti verso leggi ufficiali o siti istituzionali).

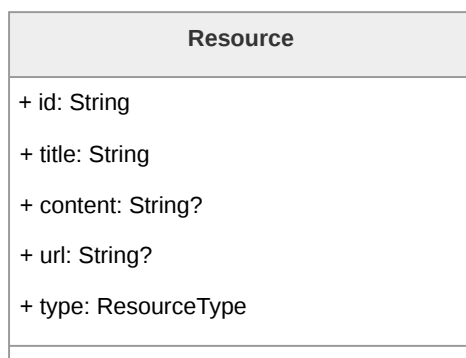


Figura 91: Diagramma della classe **Resource**

ResourceType **ResourceType** è l'enumerazione di dominio che definisce le possibili nature della risorsa (es. **community**, **law**, **article**). Questa astrazione tipizzata è cruciale non solo per la *Business Logic*^G, ma anche per il ViewModel, che la utilizza come chiave discriminante per l'applicazione dei filtri locali.

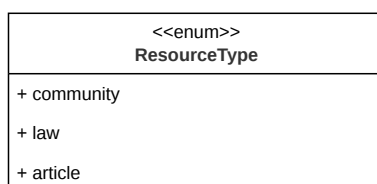


Figura 92: Diagramma della classe **ResourceType**

MaterialRepository Il **MaterialRepository** assolve al ruolo di *Single Source of Truth* per l'accesso ai documenti. Implementando l'interfaccia **CacheableRepository**, mantiene in memoria la lista completa dei materiali recuperati (**cachedResources**). Questa scelta architetturale disaccoppia le operazioni di filtraggio visivo dalla rete, permettendo all'applicazione di fornire una reattività immediata ai cambi di categoria operati dall'**Utente**^G.

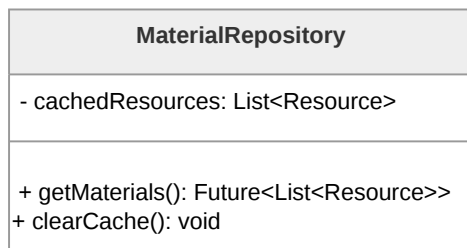


Figura 93: Diagramma della classe **MaterialRepository**

MaterialService Appartenente al livello infrastrutturale, il **MaterialService** gestisce le invocazioni HTTP tramite **ApiClient**. Il metodo **fetchMaterials** inoltra la richiesta al **Backend^G** **AWS Serverless^G**, il quale si occupa di esporre e recapitare il **Payload^G** JSON statico custodito all'interno di un Bucket S3, configurazione ideale per minimizzare il caricamento iniziale e mantenere i costi scalabili.

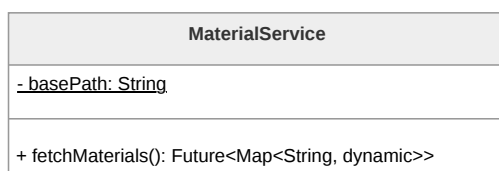


Figura 94: Diagramma della classe **MaterialService**

ResourceDTO La classe **ResourceDTO** (*Data Transfer Object*) gestisce il confine di serializzazione. Mappa in modo difensivo le chiavi del JSON scaricato dal **Backend^G** (gestendo correttamente la presenza o l'assenza dei campi opzionali **content** o **url**) e restituisce al **Repository^G** oggetti **Resource** perfettamente istanziati e tipizzati per il dominio interno.

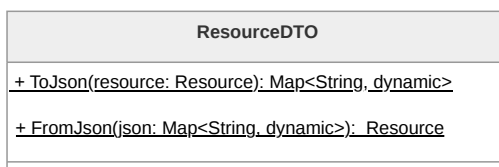


Figura 95: Diagramma della classe **ResourceDTO**

3.4.8 Main App, Home e Sistema SOS

Il nucleo orchestrale dell'applicazione è definito dall'integrazione tra la radice del sistema, la dashboard di controllo e il modulo di emergenza. Questa sezione governa il **Ciclo di vita^G** della sessione, il routing globale e i meccanismi di sicurezza immediata, garantendo che le funzionalità critiche siano sempre accessibili all'**Utente^G**.

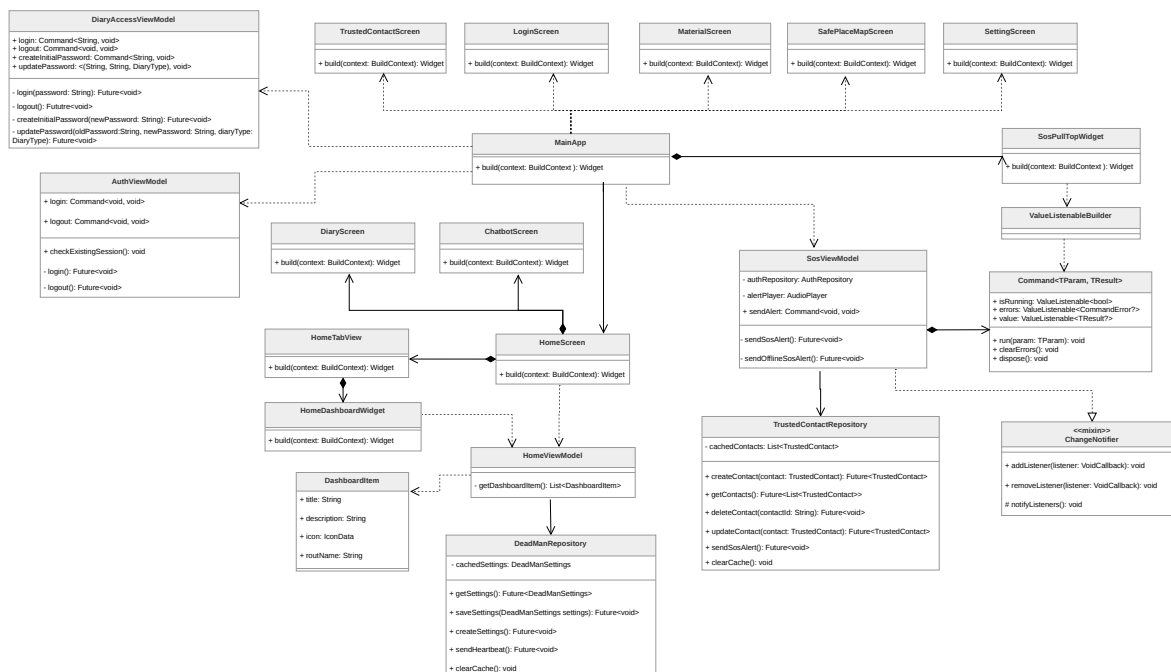


Figura 96: Diagramma delle classi complessivo di Main App, Home e SOS

MainAppWidget La classe **MainAppWidget** costituisce l'*Entry Point* dell'applicazione Flutter. Si occupa dell'inizializzazione dei provider globali (**AuthViewModel**, **SosViewModel**, **DiaryAccessViewModel**) tramite un **MultiProvider**. Architetturealmente, definisce una strategia di navigazione ibrida: utilizza rotte nominate per le pagine verticali (es. **/login**, **/settings**) e sfrutta il parametro **builder** di **MaterialApp** per iniettare il **SosPullTopWidget** in uno stack globale. Questa scelta assicura che il trigger di emergenza risieda in uno strato superiore dell'interfaccia, rendendolo disponibile sopra ogni altra schermata.

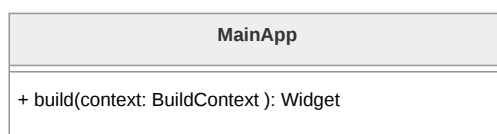


Figura 97: Diagramma della classe **MainAppWidget**

HomeScreen La **HomeScreen** implementa la navigazione principale dell'applicativo tramite un pattern a schede (*Tab Navigation*). Coordina un widget **PageView** per lo scorrimento delle sezioni (Chatbot, Home, Diario) e una **NavigationBar** per la selezione rapida. Per preservare l'integrità dei dati sensibili, la classe integra logiche di controllo dell'autenticazione: se l'**Utente** non è loggato, le tab protette vengono sostituite dinamicamente da un **AuthPlaceholderScreen**, impedendo accessi non autorizzati senza interrompere il flusso di navigazione.

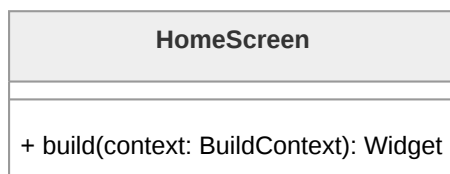


Figura 98: Diagramma della classe **HomeScreen**

HomeTabView La **HomeTabView** rappresenta il contenuto specifico della tab centrale dell'applicazione. Agisce come un contenitore specializzato che definisce il layout della dashboard, includendo l'*AppBar* con i punti di accesso al profilo e alle impostazioni, e i messaggi di benvenuto personalizzati. Al suo interno, delega il caricamento e la visualizzazione dei moduli interattivi al widget figlio **HomeDashboardWidget**.

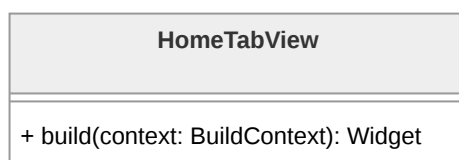


Figura 99: Diagramma della classe **HomeTabView**

HomeDashboardWidget Il **HomeDashboardWidget** è incaricato della renderizzazione visiva dei punti di accesso ai vari moduli dell'app (Materiali, Mappa, Contatti). Ogni modulo viene presentato tramite una card interattiva che incapsula la logica di navigazione verso la relativa rotta nominata.

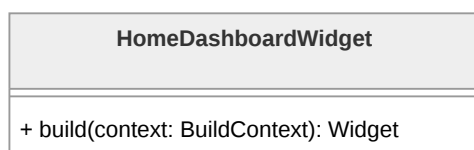


Figura 100: Diagramma della classe **HomeDashboardWidget**

HomeViewModel Il **HomeViewModel** centralizza la **Presentation Logic**^G della dashboard. Attraverso il metodo `getDashboardItem()`, popola staticamente una lista di oggetti di dominio **DashboardItem**, definendone l'iconografia, il titolo descrittivo e la rotta di destinazione. Questo approccio permette di gestire l'**Architettura**^G della homepage in modo centralizzato, facilitando l'aggiunta o la rimozione di moduli funzionali senza modificare la struttura dei widget.

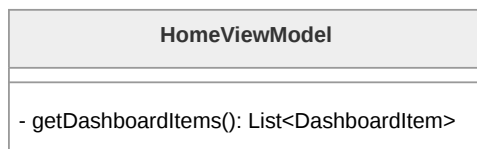


Figura 101: Diagramma della classe **HomeViewModel**

SosPullTopWidget Il **SosPullTopWidget** rappresenta l'interfaccia di attivazione globale del sistema di emergenza. Essendo iniettato alla radice dell'applicazione, rimane persistente e accessibile tramite un'interazione di trascinamento (*pull*) o pressione. Comunica direttamente con il **SosViewModel** per innescare immediatamente le procedure di soccorso indipendentemente dalla schermata in cui si trova l'**Utente**^G.



Figura 102: Diagramma della classe **SosPullTopWidget**

SosViewModel Il **SosViewModel** orchestra la logica critica di invio delle segnalazioni. Implementando il pattern Command, espone il metodo **sendAlert** che avvia una strategia di *fallback* basata sulla connettività del dispositivo. In modalità online, inoltra una richiesta al **Backend**^G per l'invio di email tramite AWS SES; in modalità offline, attiva un segnale acustico ad alta intensità tramite il componente **AudioPlayer**, garantendo una forma di allerta locale anche in assenza di rete.

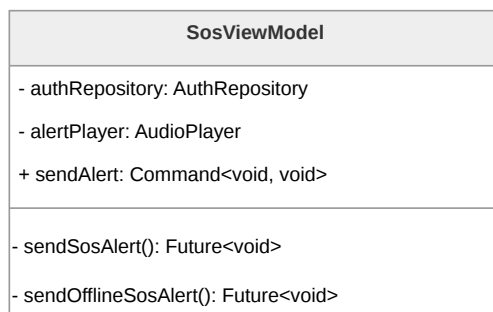


Figura 103: Diagramma della classe **SosViewModel**



3.5. Architettura Back End

3.5.1 Autenticazione

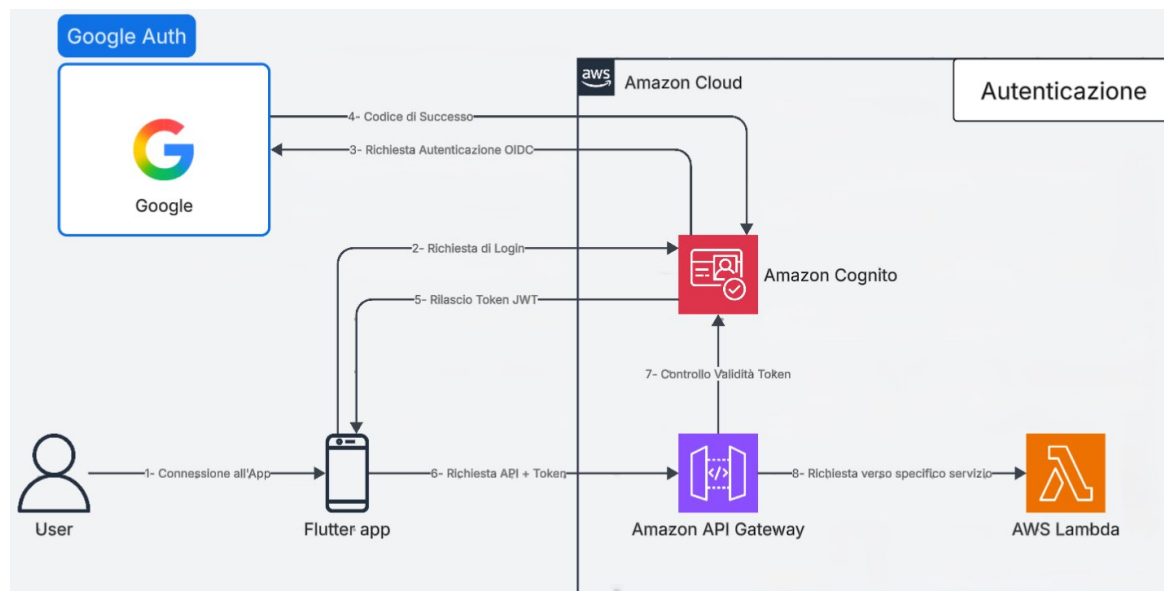


Figura 104: **Architettura^G** e Flusso di Autenticazione

Architettura^G e Flusso di Autenticazione: Modulo Federated Login

La presente sezione descrive l'**Architettura^G** logica e il flusso dei dati relativi al modulo di autenticazione e autorizzazione dell'applicazione. Il sistema adotta un modello di federated login basato su Amazon Cognito integrato con Google come Identity Provider (IdP), progettato per garantire un accesso sicuro e standardizzato alle risorse di **Backend^G** senza demandare la gestione crittografica delle credenziali all'applicazione client.

Iniziazione del Flusso e Autenticazione Federata

Il processo di autenticazione ha inizio quando l'**Utente^G** interagisce con l'applicazione client (Flutter) richiedendo l'accesso al sistema. L'applicazione delega interamente la gestione dell'identità inoltrando una richiesta di login ad Amazon Cognito. Cognito, fungendo da Service Provider, intercetta la richiesta e avvia una procedura di autenticazione basata sullo standard OpenID Connect (OIDC) delegandola ai server di Google. L'**Utente^G** completa la fase di riconoscimento interfacciandosi direttamente con



l'infrastruttura del provider esterno. A seguito della corretta verifica delle credenziali, Google restituisce ad Amazon Cognito un codice di autorizzazione attestante il successo dell'operazione e l'identità dell'**Utente**^G.

Emissione dei Token di Sessione

Acquisita la conferma dal provider federato, Amazon Cognito processa il codice di autorizzazione e genera un set di token crittografici aderenti allo standard JWT (JSON Web Token). Questi token vengono trasmessi in modo sicuro all'applicazione mobile, concludendo la fase di autenticazione. Da questo momento, il client dispone di una prova di identità verificata e utilizzerà l'Access Token JWT per autorizzare le successive chiamate di rete dirette ai servizi di **Backend**^G.

validazione Perimetrale e Instradamento Sicuro

Per accedere alle logiche applicative, l'app Flutter compone una richiesta HTTPS indirizzata all'Amazon API Gateway, includendo il token JWT nelle intestazioni di autorizzazione. L'API Gateway assume il ruolo di strato di sicurezza perimetrale (edge security). Prima di instradare il traffico, il Gateway utilizza un Cognito Authorizer nativo per eseguire un rigoroso controllo di validità sul token ricevuto. Tale procedura verifica l'autenticità della firma crittografica, l'integrità del **Payload**^G e la validità temporale della sessione direttamente contro le configurazioni dello User Pool di Cognito.

Questa specifica impostazione architetturale assicura che qualsiasi richiesta malevola, non autorizzata o provvista di credenziali scadute venga respinta al livello di rete. Solamente le richieste che superano con successo la validazione del token vengono instradate dall'API Gateway verso la specifica funzione AWS Lambda designata, garantendo che le risorse computazionali elaborino esclusivamente traffico certificato e sicuro.

3.5.2 Luoghi sicuri

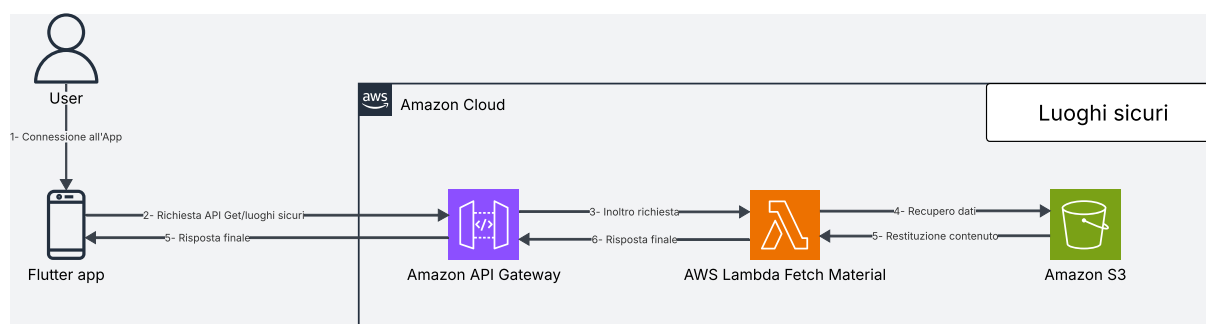


Figura 105: **Architettura**^G moduli Luoghi Sicuri

Architettura^G e Flusso di Elaborazione: Modulo Luoghi Sicuri

Il modulo "Luoghi Sicuri" è stato ingegnerizzato adottando una rigorosa separazione delle



responsabilità (*Separation of Concerns*^G) tra l'elaborazione lato client (Frontend) e l'infrastruttura **Cloud**^G (**Backend**^G). L'obiettivo primario è stato quello di minimizzare il carico computazionale server-side ed eliminare del tutto i costi legati ai calcoli geospaziali.

Il flusso di elaborazione si articola nei seguenti passaggi:

- **Ingestione della Richiesta:** L'**Utente**^G accede alla sezione della mappa tramite l'applicazione. Il client Flutter genera una richiesta HTTPS (metodo **GET** /safe_locations).
- **Instradamento verso Lambda:** API Gateway intercetta la richiesta, ne valida il contesto e la inoltra alla funzione AWS Lambda dedicata alla gestione dei luoghi sicuri.
- **Accesso ai dati e logica applicativa:** la Lambda esegue la logica applicativa necessaria e recupera i dati dal bucket S3 dedicato. Al suo interno sono infatti memorizzati sia i metadati descrittivi, sia le coordinate geospaziali (latitudine e longitudine) per ciascun punto di interesse.
- **Elaborazione e serializzazione:** la funzione Lambda elabora i dati estratti e li converte in un **Payload**^G JSON standardizzato e ottimizzato per il consumo lato client.
- **Risposta al client:** API Gateway inoltra la risposta HTTP 200 OK al client Flutter contenente l'elenco dei luoghi sicuri.

Delega della Logica Geospaziale al Livello Client (Edge Computing)

Al fine di ottimizzare ulteriormente i costi, l'intera logica di rendering visivo e di routing è delegata all'applicazione sul dispositivo dell'**Utente**^G. Il **Backend**^G AWS si configura esclusivamente come Data Provider di coordinate statiche. Alla ricezione del **Payload**^G JSON, l'applicativo Flutter si fa carico di istanziare la mappa nativa e posizionare i marker grafici (pin). L'azione di navigazione guidata (es. pulsante "Portami lì") sfrutta le Intents (su Android) o le URL Schemes (su iOS) del sistema operativo per invocare applicazioni di navigazione di terze parti (come Google Maps o Apple Maps). Questo approccio demanda il tracciamento GPS e il calcolo dinamico dell'itinerario al dispositivo locale, garantendo la scalabilità infinita del sistema AWS anche in condizioni di massiccio utilizzo concorrente.

API associate

- **GET /safe_locations/:** ritorna le informazioni di tutti i luoghi sicuri.



Nota di sicurezza: L'integrazione di un Authorizer Cognito per la verifica delle autorizzazioni è stata deliberatamente omessa per questa route. L'assenza dell'Authorizer in quest'area è difatti una scelta architetturale mirata a rimuovere complessità infrastrutturale non necessaria e ad abbattere i costi operativi associati al recupero di questi specifici dati.

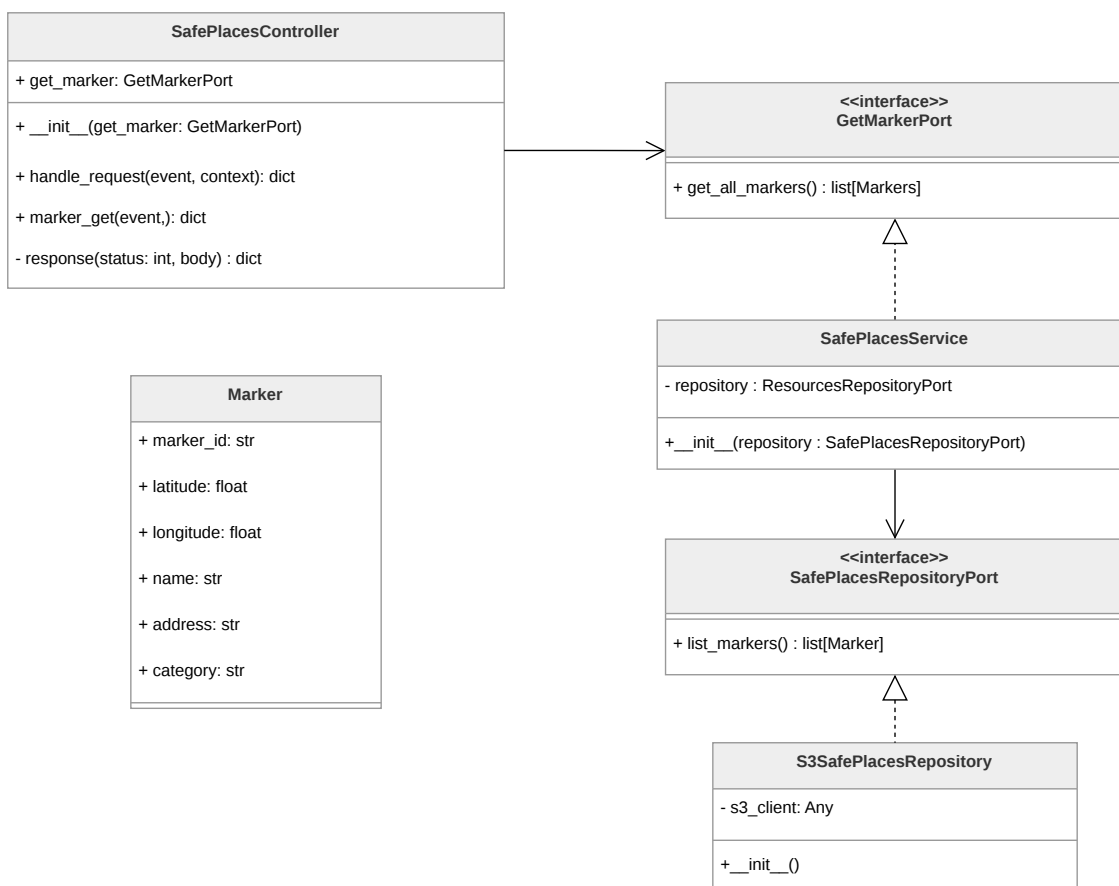


Figura 106: Componenti del microservizio luoghi sicuri

Architettura^G logica: Modulo Luoghi Sicuri Il microservizio **luoghi sicuri** viene utilizzato per eseguire il fetch dei **Marker** dall'istanza del bucket S3. È formato da tre componenti principali:

- **SafePlacesController**, che rappresenta l'application logic e corrisponde al controller del pattern esagonale. Riceve l'evento grezzo di API Gateway, lo instrada verso la port primaria e costruisce la risposta HTTP da restituire al client;
- **SafePlacesService**, che rappresenta la **Business Logic^G** e corrisponde alla **Business Logic^G** del pattern esagonale. Implementa la port primaria **GetMarkerPort**



e orchestra la comunicazione con il layer di persistenza tramite la port secondaria **SafePlacesRepositoryPort**;

- **S3SafePlacesRepository**, che rappresenta la **Persistence Logic^G** e corrisponde all'adapter secondario dell'**Architettura^G** esagonale. Contiene tutta la logica specifica per il recupero dei marker da un bucket S3 tramite il client boto3.

Marker

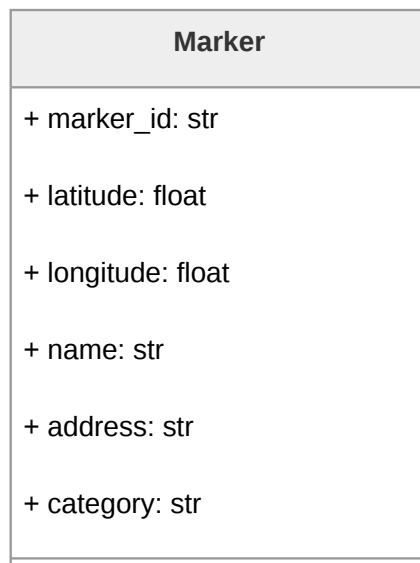


Figura 107: Diagramma della classe **Marker**

Rappresenta un punto geografico sicuro visualizzabile sulla mappa nel frontend dell'applicazione. Descrizione attributi:

Sommario

marker_id, che identifica univocamente il marker;

latitude e **longitude** definiscono la posizione geografica;

name indica il nome del marker;

address indica l'indirizzo;

category indica la categoria del marker;



SafePlacesController

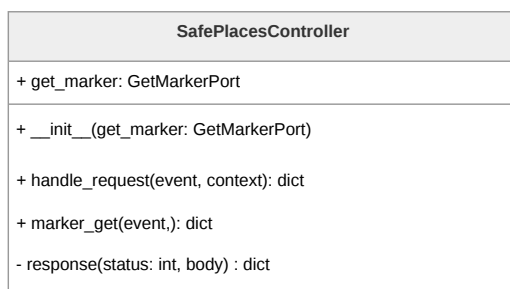


Figura 108: Diagramma della classe **SafePlacesController**

È il controller del pattern esagonale. Riceve la richiesta grezza da API Gateway, la instrada verso la port primaria e costruisce la risposta HTTP da restituire al client. Descrizione metodi:

- **marker_get(event)** fa il parsing della richiesta e chiama la funzione appropriata della specifica port primaria;
- **handle_request(event)** chiama la funzione appropriata del controller in base alla route della richiesta;
- **response(status, body)** costruisce il dizionario di risposta nel formato atteso da API Gateway.

GetMarkerPort

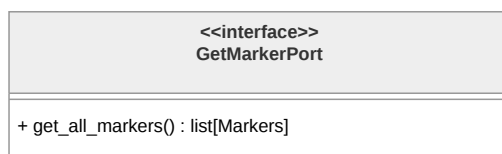


Figura 109: Diagramma dell'interfaccia **GetMarkerPort**

È la port primaria del pattern esagonale.

Il metodo **get_all_markers()** restituisce la lista completa dei marker disponibili senza esporre dettagli implementativi su come vengono recuperati.

SafePlacesService

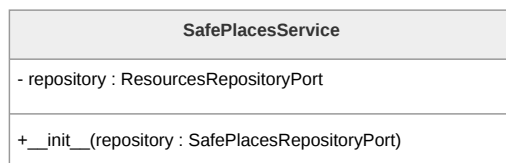


Figura 110: Diagramma della classe **SafePlacesService**

È la **Business Logic**^G del microservizio. Contiene l'application logic per il recupero dei luoghi sicuri e orchestra la comunicazione con il layer di persistenza tramite la outbound port **SafePlacesRepositoryPort**.

SafePlacesRepositoryPort

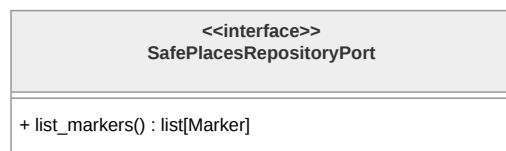


Figura 111: Diagramma dell'interfaccia **SafePlacesRepositoryPort**

È la outbound port del pattern esagonale orientata alla persistenza, utilizzata per restituire la lista di tutti i marker disponibili senza esporre dettagli su S3 o qualsiasi altro storage sottostante.

S3SafePlacesRepository



Figura 112: Diagramma della classe **S3SafePlacesRepository**

È l'adapter secondario che implementa **SafePlacesRepositoryPort**. Il costruttore **__init__()** inizializza il client S3.



3.5.3 Materiale informativo

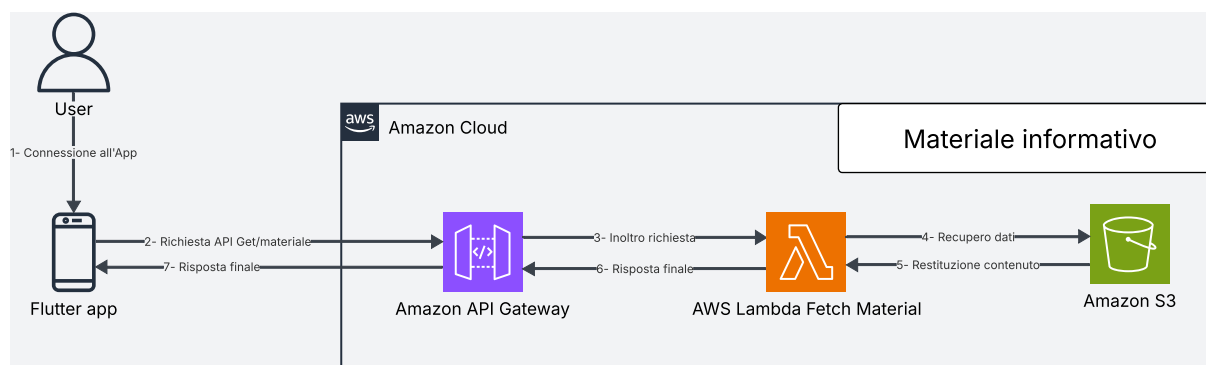


Figura 113: **Architettura^G** moduli Materiale Informativo

Architettura^G e Flusso di Elaborazione: Modulo Materiale Informativo

La sezione dedicata al "Materiale Informativo" è stata progettata privilegiando la massima efficienza e l'ottimizzazione dei costi operativi. Trattandosi di un modulo a prevalenza di lettura (Read-Heavy) e privo di logiche di business complesse per la manipolazione dei dati, l'**Architettura^G** prevede una funzione Lambda molto leggera, che si interfaccia con S3 esclusivamente per il fetch dei dati.

Il ciclo di elaborazione si innesca quando l'applicazione client necessita di recuperare il catalogo delle informazioni, generando una richiesta HTTPS (metodo **GET**) verso l'endpoint dedicato. L'API Gateway intercetta la chiamata e la inoltra alla funzione AWS Lambda responsabile del modulo.

La Lambda si occupa quindi di:

- recuperare i record relativi ai materiali informativi dal bucket S3;
- decodificare il contenuto per validarne l'integrità strutturale;
- serializzare il risultato aggiungendo gli header CORS necessari per renderlo consumabile dal frontend.

Il risultato elaborato viene restituito all'API Gateway, che lo inoltra al client Flutter tramite una risposta HTTP. Il ciclo sincrono si conclude con la consegna del catalogo informativo all'applicazione.

API associate

- **GET /learning/**: ritorna le *preview* di tutti i materiali di apprendimento.
- **GET /learning/{learning_id}**: ritorna il contenuto di uno specifico materiale di apprendimento.



Nota di sicurezza: In maniera analoga a quanto previsto per i luoghi sicuri, anche per l'accesso al materiale informativo si è optato per non implementare l'Authorizer Cognito. Questa decisione è dettata dalla volontà di snellire l'**Architettura^G**, riducendo le complessità superflue ed evitando di incorrere nei costi operativi aggiuntivi dell'infrastruttura di autenticazione.

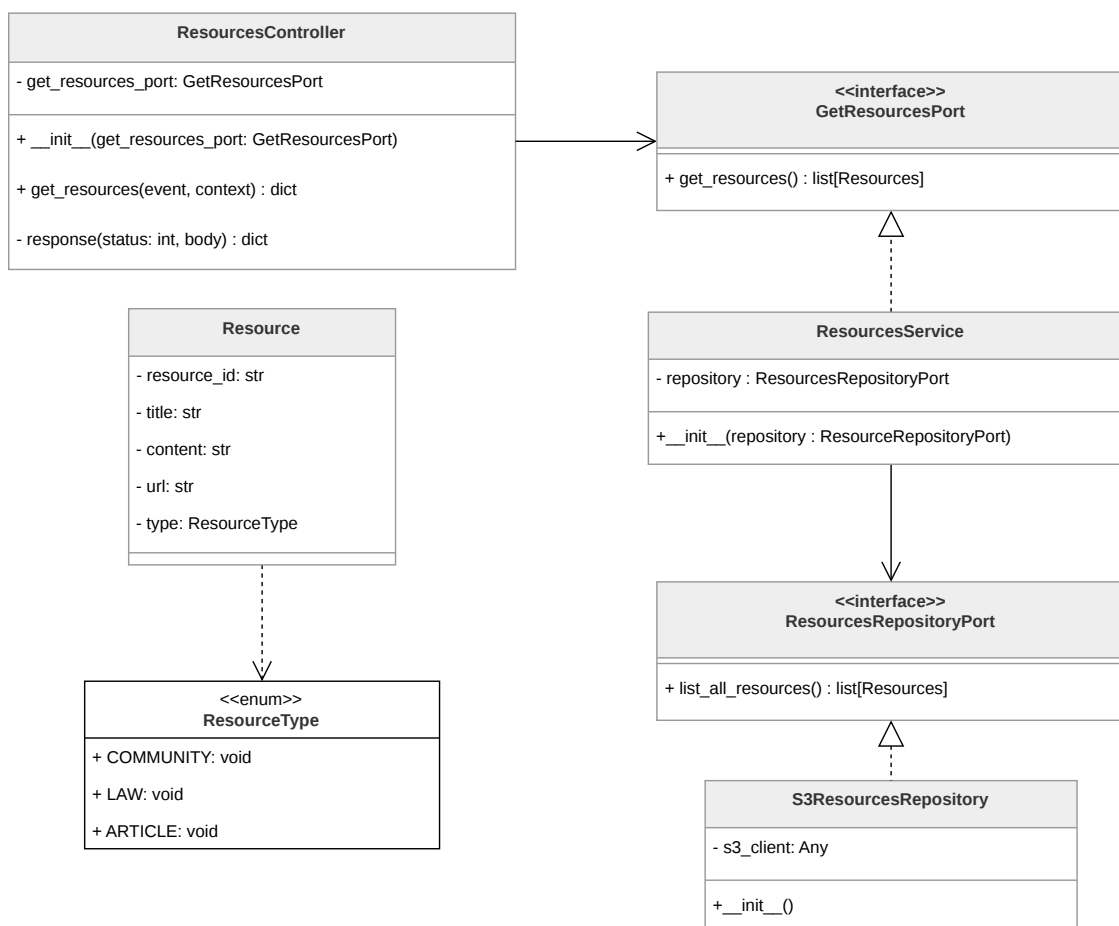


Figura 114: Componenti del microservizio materiale informativo

Architettura^G logica: Modulo Materiale Informativo Il microservizio **materiale informativo** viene utilizzato per eseguire il fetch delle **Resource** dall'istanza del bucket S3 in cui sono memorizzate le risorse informative da visualizzare nell'applicazione. È formato da tre componenti principali:

- **ResourcesController**, che rappresenta l'application logic e corrisponde al controller del pattern esagonale. Riceve l'evento grezzo di API Gateway, lo instrada verso la port primaria e costruisce la risposta HTTP da restituire al client;



- **ResourcesService**, che rappresenta la **Business Logic^G** e corrisponde alla **Business Logic^G** del pattern esagonale. Implementa la port primaria **GetResourcesPort** e orchestra la comunicazione con il layer di persistenza tramite la port secondaria **ResourcesRepositoryPort**;
- **S3ResourcesRepository**, che rappresenta la **Persistence Logic^G** e corrisponde all'adapter secondario del pattern esagonale. Contiene tutta la logica specifica per il recupero delle risorse da un bucket S3 tramite il client boto3.

Resource

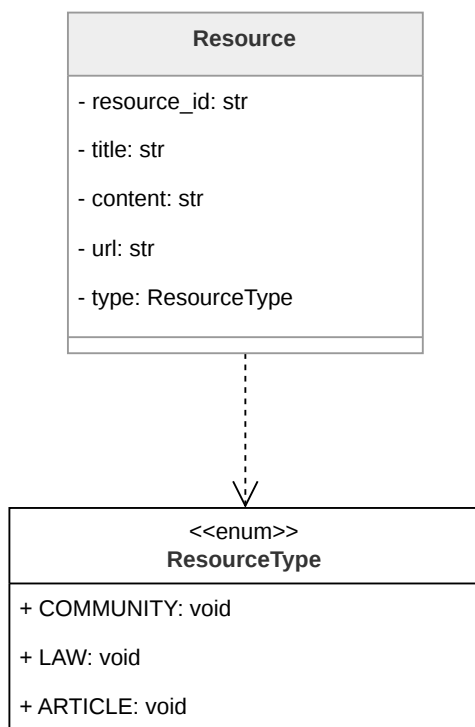


Figura 115: Diagramma della classe **Resource**

Rappresenta una risorsa informativa disponibile nell'applicazione. Descrizione attributi:

- **resource_id** identificativo della singola risorsa;
- **title** nome della singola risorsa;
- **content** il testo descrittivo;
- **url** il collegamento alla fonte esterna;



- **type** è di tipo **ResourceType** e categorizza la risorsa secondo una delle tipologie previste dal dominio.

L'enumerazione **ResourceType** definisce le tre categorie disponibili: **COMMUNITY**, **LAW** e **ARTICLE**.

ResourcesController

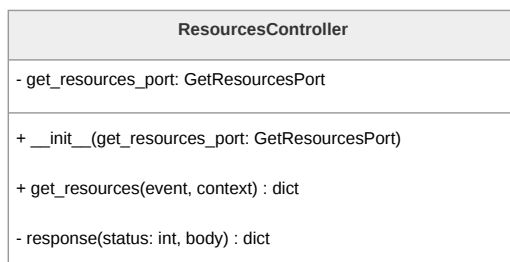


Figura 116: Diagramma della classe **ResourcesController**

È il controller del pattern esagonale. Riceve la richiesta grezza di API Gateway, la instrada verso la port primaria e costruisce la risposta HTTP da restituire al client. Descrizione metodi:

- **get_resources(event)** costituisce il punto di ingresso del controller e riceve i parametri nativi della Lambda;
- **response(status, body)** è un metodo di utilità che costruisce il dizionario di risposta nel formato atteso da API Gateway.

GetResourcesPort

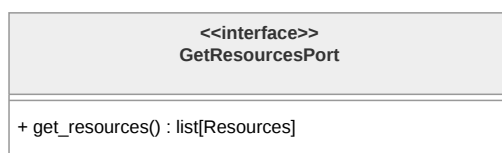


Figura 117: Diagramma della classe **GetResourcesPort**

È la inbound port del pattern esagonale. Descrive l'interfaccia per il recupero della lista completa delle risorse informative disponibili.



ResourcesService

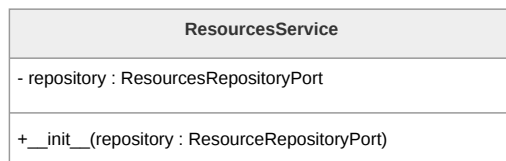


Figura 118: Diagramma della classe **ResourcesService**

È la **Business Logic**^G del microservizio e implementa la port primaria **GetResourcesPort**. Contiene la logica applicativa per il recupero del materiale informativo e orchestra la comunicazione con il persistence layer tramite la port secondaria **ResourcesRepositoryPort**.

ResourcesRepositoryPort



Figura 119: Diagramma della classe **ResourcesRepositoryPort**

È l'outbound port del pattern esagonale. Definisce l'interfaccia per la restituzione della lista completa delle risorse disponibili, senza esporre dettagli su S3.

S3ResourcesRepository

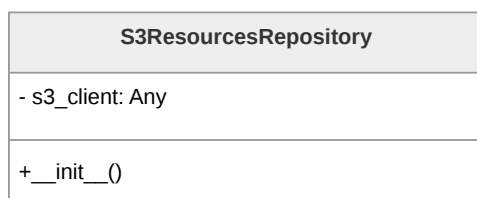


Figura 120: Diagramma della classe **S3ResourcesRepository**

È l'adapter secondario che implementa **ResourcesRepositoryPort**. Contiene tutta la logica specifica per il recupero delle risorse informative da un bucket S3.



3.5.4 Diario criptato e fittizio

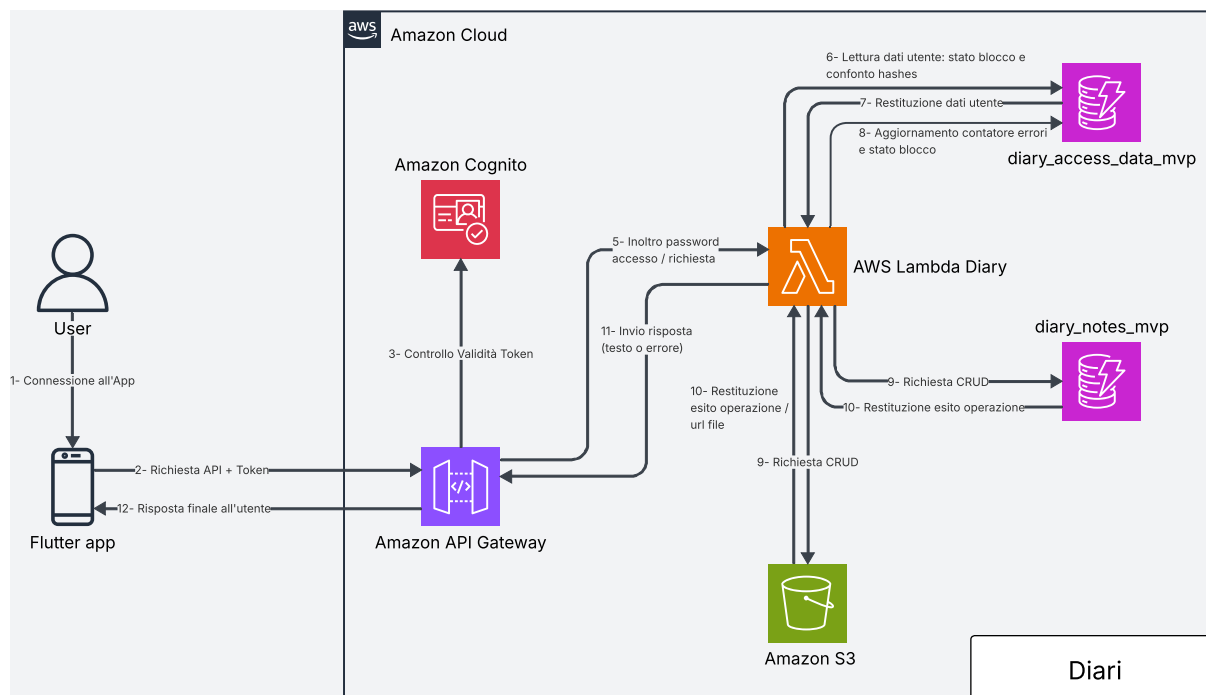


Figura 121: **Architettura^G** diari

Architettura^G e Flusso di Elaborazione: diario criptato e fittizio

La sezione dedicata al diario criptato e fittizio è progettata per garantire la sicurezza delle informazioni dell'**Utente^G** e la corretta gestione e archiviazione di diverse tipologie di dati, quali contenuti testuali, immagini e tracce audio. A differenza delle altre funzionalità, il diario è l'unico componente protetto da una password aggiuntiva, distinta da quella utilizzata per l'account principale.

Questo approccio consente all'applicazione di fornire accesso a risorse differenti in base alla password inserita, dato che i due diari sono protetti da credenziali separate. Poiché Amazon Cognito non supporta direttamente flussi di autenticazione di questo tipo, la gestione dell'accesso al diario avviene tramite la memorizzazione in DynamoDB degli hash delle due password. Quando l'**Utente^G** tenta l'accesso, viene generato l'hash della password inserita tramite Argon2id. Questo hash viene quindi confrontato con quello salvato nella tabella utenti. In caso di corrispondenza, la password fornita viene utilizzata per derivare una chiave crittografica, impiegata per decifrare le note del diario corrispondente. In entrambi i casi viene utilizzato Argon2id, ma con parametri di memory cost, time cost, salt length e parallelism diversi.

Una volta completato con successo l'accesso, viene creata una sessione lato **Backend^G**, così da rendere più efficienti le successive operazioni CRUD. La creazione della sessione consiste nella generazione di un token casuale che viene inviato in risposta al client. Tale



token ha durata limitata e viene memorizzato nella tabella DynamoDB relativa all'accesso al diario. In caso di un numero di tentativi di accesso errati superiore alla soglia consentita, viene applicato un meccanismo di timeout incrementale che blocca temporaneamente ulteriori tentativi.

L'archiviazione delle note avviene su due servizi AWS distinti. I contenuti testuali sono memorizzati in DynamoDB, che mantiene le informazioni per ciascun **Utente^G** e per ciascun diario. I file binari, come immagini e tracce audio, sono invece salvati in bucket S3. Il recupero di tali risorse avviene tramite URL *presigned*, al fine di ridurre la quantità di dati trasferiti sulla rete.

Il flusso di elaborazione è dunque il seguente:

- **Invio della Richiesta di Autenticazione:** L'**Utente^G** inserisce la password del diario tramite app. Il client genera una richiesta HTTPS (metodo POST /diary/auth/) includendo la password nel body della richiesta.
- **verifica delle Credenziali:** API Gateway verifica che l'**Utente^G** sia autenticato all'applicazione. Dopodiché inoltra la richiesta alla funzione Lambda, che calcola l'hash della password ricevuta e lo confronta con gli hash memorizzati in DynamoDB. In caso di corrispondenza, viene identificato il tipo di diario associato.
- **Derivazione della Chiave Crittografica:** La funzione Lambda utilizza la password fornita per derivare una chiave crittografica tramite Argon2id, impiegando un salt dedicato. Tale chiave viene utilizzata per le successive operazioni di decrittazione.
- **Creazione della Sessione:** A seguito dell'autenticazione, il **Backend^G** genera una sessione associata all'**Utente^G**, che consente di effettuare operazioni sul diario senza ripetere il processo di autenticazione per ogni richiesta.
- **Recupero delle Note:** Quando l'**Utente^G** richiede di recuperare la lista di note di uno dei diari (metodo GET /diary/{diary_type}/), API Gateway invia la richiesta alla Lambda, che recupera le *preview* dal database DynamoDB e le restituisce al client.
- **Recupero di una Nota:** Per ottenere il contenuto completo di una nota (metodo GET /diary/{diary_type}/{note_id}/), la Lambda recupera i dati cifrati da DynamoDB e gli eventuali riferimenti a file su S3, decrittando il contenuto tramite la chiave derivata e restituendolo al client.
- **Gestione dei Contenuti Binari:** In presenza di immagini o tracce audio, la Lambda genera URL *presigned* per gli oggetti memorizzati su S3, permettendo al client di accedere direttamente alle risorse senza transitare attraverso il **Backend^G**.



- **Operazioni di Scrittura:** Per la creazione o modifica di note (metodi POST /diary/{diary_type}/ e PUT /diary/{diary_type}/{note_id}/), la Lambda cifra i contenuti tramite la chiave derivata e li memorizza in DynamoDB, salvando eventuali file binari su S3.
- **Eliminazione delle Note:** In caso di richiesta di eliminazione (metodo DELETE /diary/{diary_type}/{note_id}/), la Lambda rimuove i dati associati dal database e, se presenti, elimina anche i file binari corrispondenti dai bucket S3.

API associate

- POST /diary/auth/: invia la password per l'autenticazione; ritorna l'esito dell'autenticazione e il tipo di diario associato.
- GET /diary/{diary_type}/: ritorna le *preview* di tutte le note del diario specificato.
- POST /diary/{diary_type}/: crea una nuova nota per l'**Utente^G** nel diario specificato.
- GET /diary/{diary_type}/{note_id}/: ritorna il contenuto completo della nota specificata.
- PUT /diary/{diary_type}/{note_id}/: aggiorna la nota specificata nel database.
- DELETE /diary/{diary_type}/{note_id}/: rimuove la nota specificata.

3.5.5 Chatbot

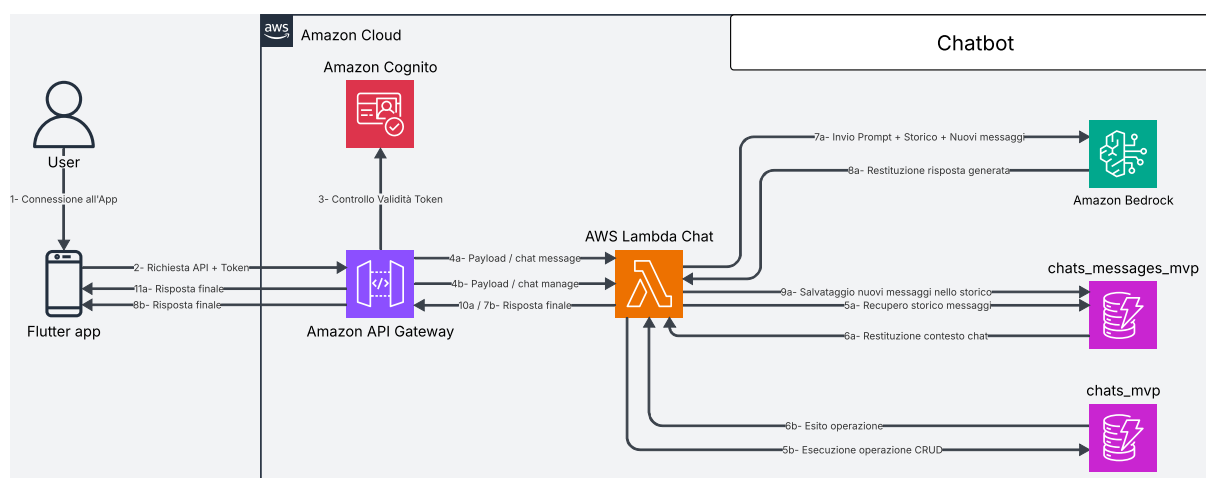


Figura 122: Architettura^G chatbot



Architettura^G e Flusso di Elaborazione: chatbot (detective delle relazioni e specchio intelligente)

L'Architettura^G di Backend^G dedicata al chatbot è progettata per gestire conversazioni persistenti tra Utente^G e modello linguistico, garantendo scalabilità, efficienza nell'accesso ai dati e qualità delle risposte generate.

API Gateway rappresenta il punto di ingresso per tutte le richieste client e si occupa della verifica dell'autenticazione dell'Utente^G. Una volta validata la richiesta, questa viene inoltrata a una funzione Lambda che implementa la logica applicativa del chatbot, inclusa la gestione delle chat e l'interazione con il modello linguistico.

Le conversazioni sono strutturate come entità persistenti (chat), ciascuna identificata da un `chat_id` univoco all'interno dell'intero sistema. Per garantire una maggiore modularità e facilitare il recupero delle informazioni, chat e messaggi sono memorizzati in tabelle DynamoDB separate. Ogni chat contiene una sequenza di messaggi (Utente^G e assistente artificiale), anch'essi memorizzati in DynamoDB. Per ottimizzare le prestazioni e ridurre il carico di rete, le operazioni di recupero dello storico restituiscono esclusivamente informazioni di sintesi (ad esempio titolo e metadata), mentre il contenuto completo dei messaggi viene fornito solo su richiesta esplicita.

Quando l'Utente^G invia un messaggio, il sistema non si limita a inoltrarlo direttamente al modello, ma recupera le informazioni più rilevanti dalla chat in corso. In particolare, la Lambda recupera il contesto rilevante della conversazione dalla base di dati (DynamoDB), costruendo un prompt arricchito che include la cronologia significativa dei messaggi. Per estrarre tali informazioni, la Lambda utilizza tecniche di ricerca lessicale basate su BM25, evitando di concatenare l'intera conversazione al testo inviato dall'Utente^G. Questo approccio consente di migliorare la coerenza e la pertinenza delle risposte generate dal modello.

Il prompt così costruito viene inviato a Bedrock, che restituisce la risposta generata dal LLM. La risposta, insieme al messaggio dell'Utente^G, viene quindi memorizzata in DynamoDB come parte della chat. Nel caso in cui il messaggio inviato sia il primo di una nuova chat, la Lambda imposta il titolo della chat alle prime tre parole del messaggio dell'Utente^G.

Il flusso di elaborazione è dunque il seguente:

- **Invio della Richiesta:** L'Utente^G interagisce con il chatbot tramite app, generando una richiesta HTTPS verso uno degli endpoint disponibili (ad esempio `POST /chats/{chat_id}/messages`).
- **Autenticazione:** API Gateway verifica che l'Utente^G sia autenticato all'applicazione. In caso positivo, la richiesta viene inoltrata alla funzione Lambda responsabile della logica del chatbot.
- **Gestione della Chat:** La Lambda identifica la chat di riferimento tramite `chat_id`. Se necessario, recupera i metadati o verifica l'esistenza della chat in DynamoDB.



- **Recupero del Contesto (RAG):** Prima di inviare il messaggio al modello, la Lambda esegue una query su DynamoDB per ottenere i messaggi precedenti rilevanti della conversazione, costruendo il contesto da includere nel prompt.
- **Costruzione del Prompt:** Il messaggio dell'**Utente^G** viene combinato con il contesto recuperato, formando un prompt strutturato da inviare al modello linguistico.
- **Invocazione del Modello:** La Lambda invia il prompt a Bedrock, che elabora la richiesta e restituisce una risposta generata.
- **Persistenza dei Messaggi:** La Lambda salva sia il messaggio dell'**Utente^G** sia la risposta del modello in DynamoDB, associandoli alla chat corrente.
- **Aggiornamento dei Metadati:** Se necessario, la Lambda aggiorna le informazioni della chat (ad esempio titolo o timestamp dell'ultima attività).
- **Restituzione della Risposta:** La risposta generata dal modello, insieme ai metadati aggiornati, viene restituita al client.
- **Recupero dello Storico:** Quando l'**Utente^G** richiede la lista delle chat (metodo GET /chats/), la Lambda restituisce solo le *preview* delle chat, senza includere l'intero contenuto dei messaggi.
- **Recupero di una Chat:** Per ottenere il contenuto completo di una chat (metodo GET /chats/{chat_id}), la Lambda recupera tutti i messaggi associati da DynamoDB e li restituisce al client.
- **Operazioni CRUD:** Le operazioni di creazione, aggiornamento ed eliminazione delle chat vengono gestite direttamente dalla Lambda tramite interazioni con DynamoDB.

API associate

- **GET /chats/**
 - **Descrizione:** Recupera la lista delle chat associate all'**Utente^G** autenticato, restituendo solo le informazioni principali, senza i messaggi.
 - **Parametri:**
 - * **user_id:** identificativo dell'**Utente^G**.
 - **Response:**
 - * 200 OK: richiesta completata con successo.
 - * Corpo della risposta:



```
{
  "chats": [
    {
      "chat_id": "01KPX72J3Y5MFM9982J3TFTH7H",
      "title": "Supporto emotivo",
      "created_at": "2026-02-22T15:22:27.615449+00:00",
      "updated_at": "2026-04-22T18:13:05.452786+00:00",
      "messages": []
    },
    {
      "chat_id": "01KPX8AB4Z9QTR672L8VYH3D2C",
      "title": "Violenza domestica",
      "created_at": "2026-03-05T09:11:42.123456+00:00",
      "updated_at": "2026-04-24T16:47:19.987654+00:00",
      "messages": []
    }
  ]
}
```

– **Errori:**

- * 400 Bad Request: parametri mancanti o non validi.
- * 401 Unauthorized: **Utente^G** non autenticato.
- * 500 Internal Server Error: errore interno del server.

● **POST /chats/**

– **Descrizione:** Crea una nuova chat associata all'**Utente^G** autenticato, iniziata senza messaggi.

– **Parametri:**

- * **title:** titolo della chat.

– **Response:**

- * 201 Created: chat creata con successo.
- * Corpo della risposta:

```
{
  "user_id": "3f8a1c2e-7b4d-4f9a-9c21-5d7e6a8b2c90",
  "chat_id": "01KQ52JAEYD7ZDH894CN9HEH05",
  "title": "La mia nuovissima chat",
  "created_at": "2026-04-26T14:19:48.574747+00:00",
  "updated_at": "2026-04-26T14:19:48.574747+00:00",
  "messages": []
}
```



– **Errori:**

- * 400 Bad Request: body mancante o non valido.
- * 500 Internal Server Error: errore interno del server.

• **GET /chats/{chat_id}**

– **Descrizione:** Recupera una chat specifica identificata da `chat_id`, includendo tutti i messaggi associati.

– **Parametri:**

- * `chat_id`: identificativo della chat.

– **Response:**

- * 200 OK: richiesta completata con successo.

- * Corpo della risposta:

```
{
  "user_id": "3f8a1c2e-7b4d-4f9a-9c21-5d7e6a8b2c90",
  "chat_id": "01KPTWJ5CZ9QM4VH8CSHV451ZV",
  "title": "La mia chat",
  "created_at": "2026-04-22T15:22:27.615449+00:00",
  "updated_at": "2026-04-25T17:26:12.748959+00:00",
  "messages": [
    {
      "chat_id": "01KPTWJ5CZ9QM4VH8CSHV451ZV",
      "message_id": "01MSG001",
      "text": "Ciao, come ti chiami?",
      "sender": "user",
      "created_at": "2026-04-25T17:25:00.000000+00:00"
    },
    {
      "chat_id": "01KPTWJ5CZ9QM4VH8CSHV451ZV",
      "message_id": "01MSG002",
      "text": "Sono Chatbot, come posso aiutarti?",
      "sender": "ai",
      "created_at": "2026-04-25T17:25:01.000000+00:00"
    }
  ]
}
```

– **Errori:**

- * 404 Not Found: chat non trovata.



- * 400 Bad Request: parametri non validi.
 - * 500 Internal Server Error: errore interno del server.
- **PUT /chats/{chat_id}**
 - **Descrizione:** Aggiorna le informazioni di una chat esistente identificata da chat_id.
 - **Parametri:**
 - * chat_id: identificativo della chat.
 - * title: nuovo titolo della chat.
 - * user_id: identificativo dell'Utente^G.
 - **Response:**
 - * 200 OK: aggiornamento completato con successo.
 - * Corpo della risposta:

```

{
  "user_id": "3f8a1c2e-7b4d-4f9a-9c21-5d7e6a8b2c90",
  "chat_id": "01KPTWJ5CZ9QM4VH8CSHV451ZV",
  "title": "Nuovo titolo aggiornato",
  "created_at": "2026-04-22T15:22:27.615449+00:00",
  "updated_at": "2026-04-26T14:26:38.684047+00:00",
  "messages": [...]
}

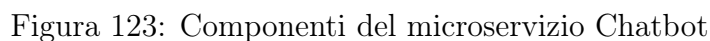
```
 - **Errori:**
 - * 400 Bad Request: body non valido o parametri mancanti.
 - * 404 Not Found: chat non trovata.
 - * 500 Internal Server Error: errore interno del server.
 - **DELETE /chats/{chat_id}**
 - **Descrizione:** Elimina una chat esistente identificata da chat_id insieme ai relativi messaggi.
 - **Parametri:**
 - * chat_id: identificativo della chat.
 - **Response:**
 - * 204 No Content: eliminazione completata con successo.
 - * Corpo della risposta: vuoto.
 - **Errori:**
 - * 404 Not Found: chat non trovata.



- * 400 Bad Request: parametri non validi.
 - * 500 Internal Server Error: errore interno del server.
- **POST /chats/{chat_id}/messages**
 - **Descrizione:** Inserisce un nuovo messaggio all'interno di una chat esistente e attiva la generazione della risposta da parte del modello LLM, secondo la modalità specificata.
 - **Parametri:**
 - * **chat_id:** identificativo della chat.
 - * **message:** contenuto del messaggio dell'**Utente^G**.
 - * **response_mode** (body, opzionale): modalità di generazione della risposta del chatbot: "mirror" o "detective".
 - **Response:**
 - * 200 OK: messaggio elaborato con successo e risposta generata.
 - * Corpo della risposta:

```
{
  "response": "questa è la risposta del chatbot",
  "input_message_id": "01KQ536RE4QX5XSCZVC42XQJ49",
  "response_message_id": "01KQ536RGM0Q86WPV8TNGZ5J1V"
}
```
 - **Errori:**
 - * 400 Bad Request: messaggio mancante o non valido.
 - * 404 Not Found: chat non trovata.
 - * 500 Internal Server Error: errore interno del servizio LLM o del **Bac-kend^G**.

Architettura^G logica del chatbot



La classe **Chat** rappresenta il modello di dominio di una conversazione tra **Utente^G** e chatbot. Contiene i metadati identificativi (**user_id**, **chat_id**), il titolo della conversazione, le date di creazione e aggiornamento e l'elenco completo dei messaggi scambiati. Questa classe costituisce l'entità centrale del dominio applicativo e viene utilizzata sia dai servizi di persistenza che da quelli di elaborazione tramite LLM.





ChatMessage

ChatMessage modella un singolo messaggio all'interno di una conversazione. Include l'identificativo univoco del messaggio (**id**), il riferimento alla chat di appartenenza (**chat_id**), il contenuto testuale (**content**), il mittente (**sender**, che può assumere il valore **"user"** o **"ai"**) e la data di creazione. Questa classe permette di mantenere traccia completa dello storico conversazionale necessario per il funzionamento del chatbot.

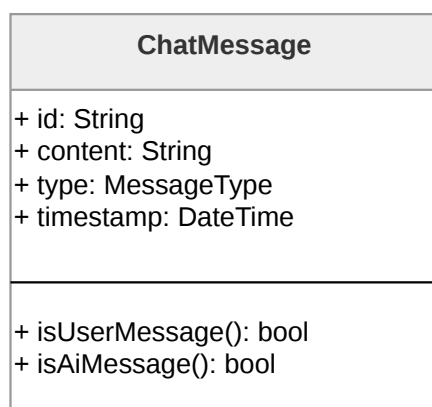


Figura 125: Diagramma della classe **ChatMessage**

ChatbotLLMPort

L'interfaccia **ChatbotLLMPort** definisce il **Contratto**^G per gli adapter che si interfacciano con LLM. Espone il metodo **process_prompt**, che accetta un prompt testuale e una modalità di risposta, restituendo la stringa generata dal modello. Questa astrazione permette di disaccoppiare la logica applicativa dall'implementazione specifica del provider LLM utilizzato.



Figura 126: Diagramma dell'interfaccia **ChatbotLLMPort**

BedrockLLMAdapter



BedrockLLMAdapter implementa l'interfaccia **ChatbotLLMPort** fornendo l'integrazione concreta con Amazon Bedrock. Questa classe incapsula la logica di comunicazione con il servizio **Cloud^G**, traducendo le richieste dell'applicazione in chiamate API specifiche per il modello di linguaggio selezionato.

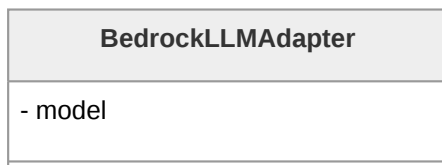


Figura 127: Diagramma della classe **BedrockLLMAdapter**

ChatbotLLMService

ChatbotLLMService coordina la generazione delle risposte del chatbot interagendo con un'implementazione di **ChatbotLLMPort**. Il metodo **get_message_response** orchestra l'intero processo: recupera i messaggi rilevanti dalla cronologia della conversazione tramite l'algoritmo BM25, costruisce il prompt contestualizzato attraverso **_build_prompt** combinando il messaggio **Utente^G** con il contesto recuperato e il prompt di sistema appropriato alla modalità selezionata, e infine delega al modello la generazione della risposta. Questa classe è quindi responsabile di implementare i diversi meccanismi di risposta sulla base di **response_mode**, realizzando così le funzionalità di risposta "Detective delle relazioni" e "Specchio intelligente".

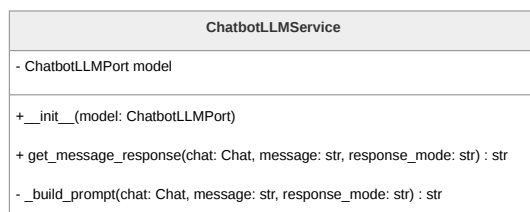


Figura 128: Diagramma della classe **ChatbotLLMService**

ChatsRepositoryPort

L'interfaccia **ChatsRepositoryPort** definisce il **Contratto^G** per la persistenza delle conversazioni. Espone metodi per tutte le operazioni CRUD: creazione di nuove chat (**create_chat**), recupero singolo (**get_chat**) e multiplo (**list_chats**), eliminazione (**delete_chat**) e aggiunta di messaggi (**add_message**). Questa astrazione garantisce l'indipendenza del dominio dall'implementazione specifica del layer di persistenza.

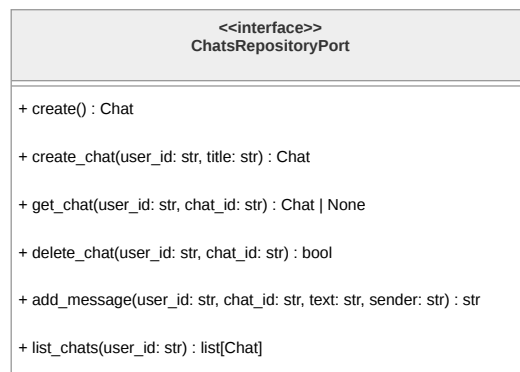


Figura 129: Diagramma dell'interfaccia **ChatsRepositoryPort**

DynamoChatAdapter

DynamoChatAdapter implementa **ChatsRepositoryPort** fornendo la persistenza concreta su DynamoDB. Questa classe gestisce l'interazione con due tabelle: **chats_mvp** per i metadati delle conversazioni e **chats_messages_mvp** per i messaggi. Implementa la logica di traduzione tra gli oggetti di dominio e le strutture dati di DynamoDB, inclusa la generazione di identificativi ULID e la gestione delle operazioni batch per l'eliminazione dei messaggi. L'adapter incapsula completamente i dettagli tecnici di AWS, esponendo al dominio un'interfaccia pulita e tecnologicamente agnostica.

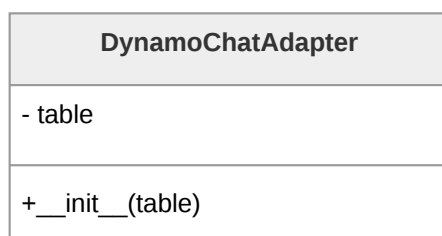
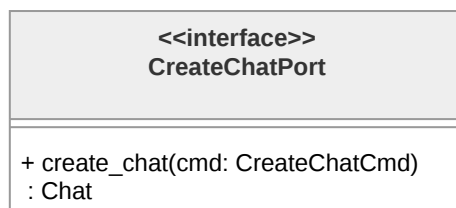


Figura 130: Diagramma della classe **DynamoChatAdapter**

CreateChatPort

L'interfaccia **CreateChatPort** definisce la porta applicativa per la creazione di nuove conversazioni. Espone il metodo **create_chat**, che accetta un comando **CreateChatCmd** e restituisce l'oggetto **Chat** creato. Questa interfaccia rappresenta uno degli use case primari del sistema, separando l'intento dell'operazione dalla sua implementazione.

Figura 131: Diagramma dell'interfaccia **CreateChatPort**

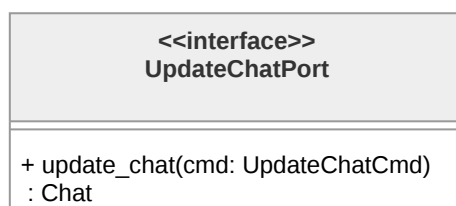
GetChatPort

GetChatPort definisce le operazioni di lettura delle conversazioni. Include il metodo **get_chat** per il recupero di una singola chat dato il suo identificativo e **get_chat_list** per ottenere l'elenco di tutte le conversazioni appartenenti a un **Utente^G**. Entrambi i metodi operano attraverso comandi dedicati, mantenendo la separazione tra richiesta ed esecuzione.

Figura 132: Diagramma dell'interfaccia **GetChatPort**

UpdateChatPort

L'interfaccia **UpdateChatPort** gestisce le operazioni di modifica delle conversazioni esistenti. Il metodo **update_chat** accetta un **UpdateChatCmd** e restituisce la chat aggiornata, permettendo la modifica di metadati quali il titolo della conversazione.

Figura 133: Diagramma dell'interfaccia **UpdateChatPort**



DeleteChatPort

DeleteChatPort definisce l'operazione di eliminazione di una conversazione. Il metodo **delete_chat** riceve un **DeleteChatCmd** contenente gli identificativi necessari e procede con la rimozione della chat e di tutti i messaggi associati dalle rispettive tabelle DynamoDB.

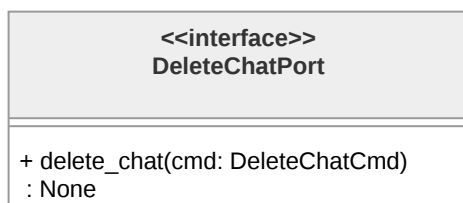


Figura 134: Diagramma dell'interfaccia **DeleteChatPort**

ChatMessagePort

L'interfaccia **ChatMessagePort** astrae l'operazione di aggiunta di messaggi a una conversazione esistente. Il metodo **add_chat_message** accetta un **AddChatMessageCmd** e restituisce l'identificativo del messaggio appena creato, permettendo al chiamante di tracciare il risultato dell'operazione.

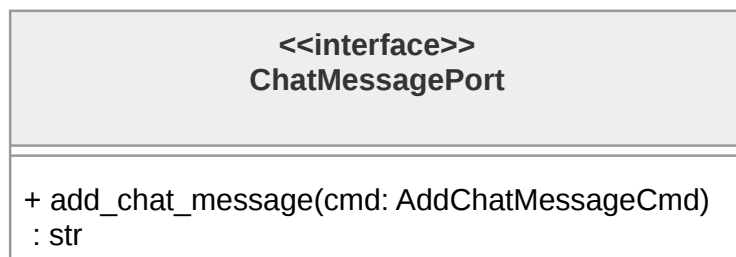


Figura 135: Diagramma dell'interfaccia **ChatMessagePort**

ChatbotCRUDService

ChatbotCRUDService implementa tutte le porte applicative relative alle operazioni CRUD sulle conversazioni: **CreateChatPort**, **GetChatPort**, **UpdateChatPort**, **DeleteChatPort** e **ChatMessagePort**. Questo servizio funge da facade unificato per tutte le operazioni di gestione delle chat, delegando al **Repository**^G la persistenza effettiva dei dati. Mantiene un riferimento a **ChatsRepositoryPort**, rispettando il principio di inversione delle dipendenze dell'**Architettura**^G esagonale.

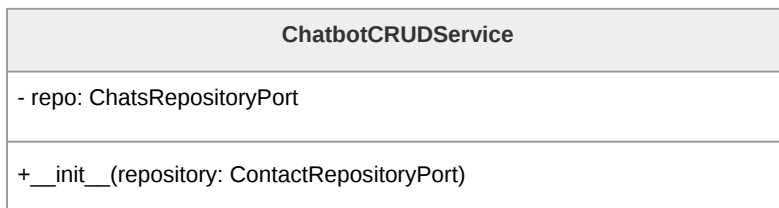


Figura 136: Diagramma della classe **ChatbotCRUDService**

CreateChatCmd

CreateChatCmd incapsula i parametri necessari per la creazione di una nuova conversazione: l'identificativo dell'**Utente^G** (**user_id**) e il titolo della chat (**title**).

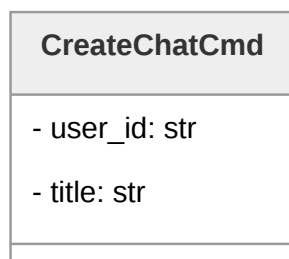


Figura 137: Diagramma della classe **CreateChatCmd**

GetChatCmd

GetChatCmd trasporta i parametri per il recupero di una specifica conversazione: l'identificativo dell'**Utente^G** e quello della chat.

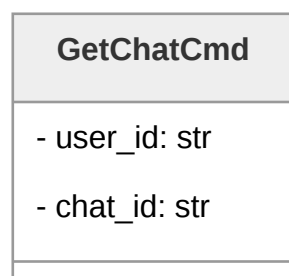


Figura 138: Diagramma della classe **GetChatCmd**

GetChatListCmd



GetChatListCmd contiene l'identificativo dell'**Utente^G** per il quale si richiede l'elenco completo delle conversazioni. Questo comando rappresenta l'operazione di recupero multiplo, fondamentale per la visualizzazione dello storico delle chat nell'interfaccia **Utente^G**.



Figura 139: Diagramma della classe **GetChatListCmd**

UpdateChatCmd

UpdateChatCmd incapsula i parametri per la modifica di una conversazione esistente: gli identificativi di **Utente^G** e chat, e il nuovo titolo da assegnare.

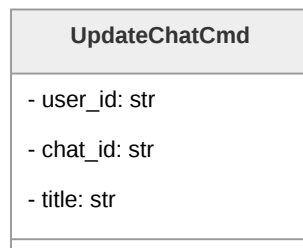


Figura 140: Diagramma della classe **UpdateChatCmd**

DeleteChatCmd

DeleteChatCmd trasporta gli identificativi necessari per l'eliminazione di una conversazione e di tutti i messaggi associati.

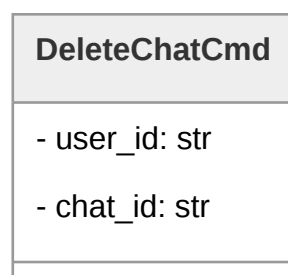


Figura 141: Diagramma della classe **DeleteChatCmd**



AddChatMessageCmd

AddChatMessageCmd contiene tutti i parametri necessari per aggiungere un messaggio a una conversazione: identificativi di **Utente^G** e chat, contenuto testuale (**text**) e tipologia del mittente (**sender**). Questo comando viene utilizzato sia per memorizzare i messaggi inviati dall'**Utente^G** che le risposte generate dal chatbot, mantenendo la coerenza dello storico conversazionale.

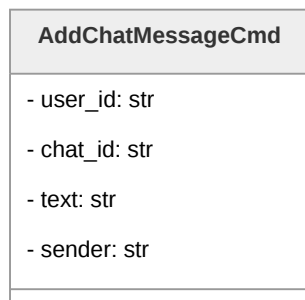


Figura 142: Diagramma della classe **AddChatMessageCmd**

ChatDTO

ChatDTO è il Data Transfer Object utilizzato per la serializzazione e deserializzazione delle conversazioni nelle comunicazioni tra frontend e **Backend^G**. Contiene tutti i campi dell'entità **Chat** di dominio e fornisce i metodi **to_dict** per la conversione in dizionario Python (successivamente serializzato in JSON) e **from_domain** per la creazione a partire da un oggetto di dominio.

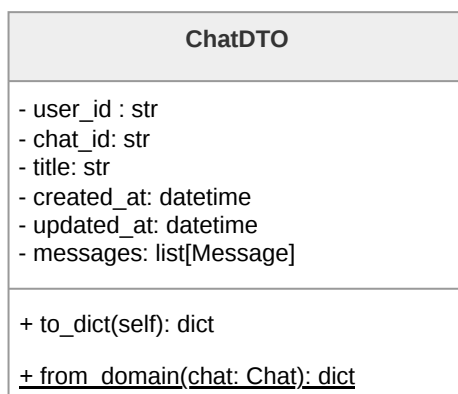


Figura 143: Diagramma della classe **ChatDTO**



ChatMessageDTO

ChatMessageDTO rappresenta il Data Transfer Object per i singoli messaggi. Analogamente a **ChatDTO**, fornisce metodi per la conversione bidirezionale tra oggetti di dominio e strutture dati serializzabili.

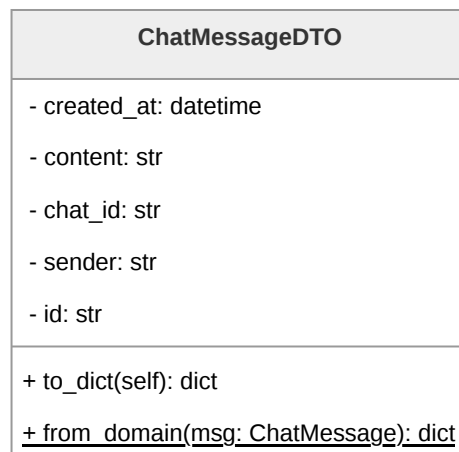


Figura 144: Diagramma della classe **ChatMessageDTO**

LambdaHandler

LambdaHandler costituisce il punto di ingresso della funzione Lambda. La funzione **lambda_handler** riceve gli eventi HTTP, li instrada ai gestori appropriati, coordina l'interazione tra **ChatbotCRUDService** e **ChatbotLLMService**, e costruisce le risposte HTTP conformi alle specifiche dell'API.

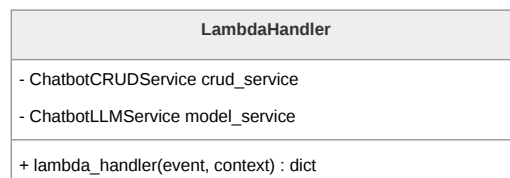


Figura 145: Diagramma della classe **LambdaHandler**



3.5.6 Contatti fidati, invio SOS e Dead man's switch

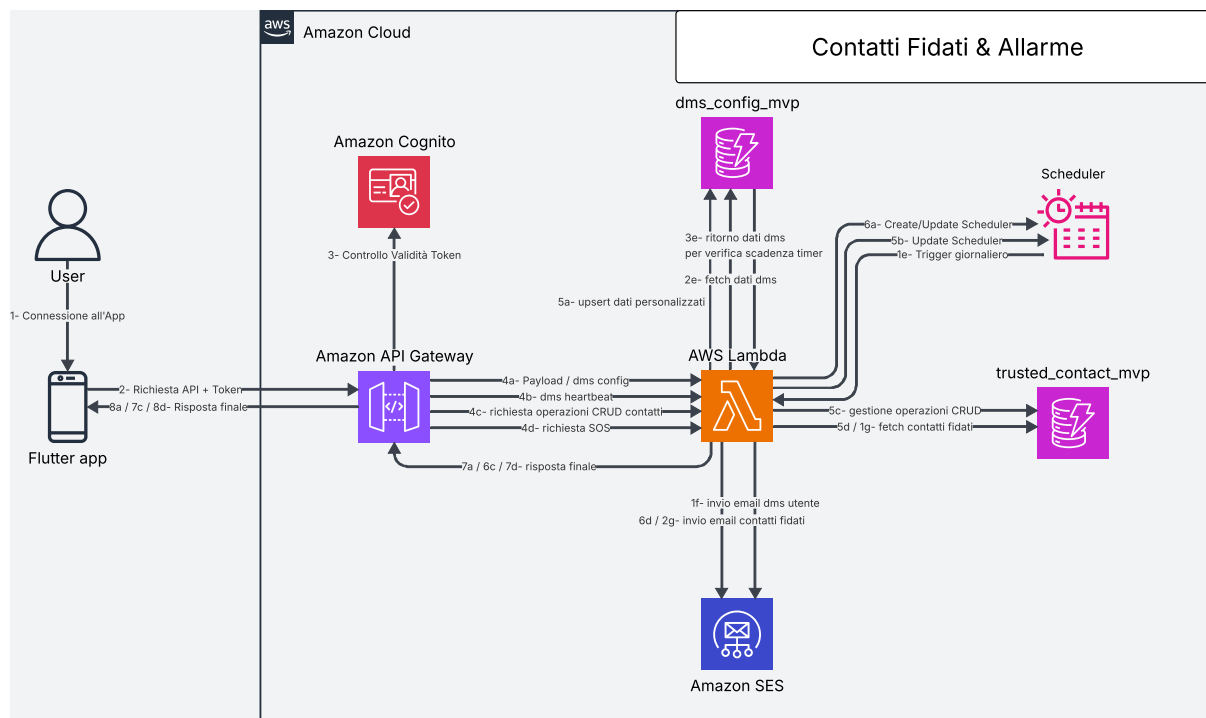


Figura 146: **Architettura^G** del sistema di contatti fidati, SOS e Dead man's switch

Architettura^G e Flusso di Elaborazione: contatti fidati, SOS e Dead man's switch

La sezione relativa ai contatti fidati, all'invio di messaggi di emergenza e al meccanismo di Dead man's switch è progettata per garantire la gestione automatizzata dello stato di attività dell'**Utente^G** e la possibilità di attivare procedure di emergenza sia manualmente che in modo automatico.

API Gateway gestisce l'esposizione delle API e verifica, per ogni richiesta, che l'**Utente^G** sia correttamente autenticato all'applicazione. Le richieste valide vengono inoltrate a una funzione Lambda centrale, responsabile della gestione della logica applicativa relativa ai contatti fidati, alla configurazione del Dead man's switch e all'orchestrazione dell'invio delle email tramite Amazon SES.

I dati sono memorizzati su due database DynamoDB distinti:

- **dms_config_mvp**: contiene la configurazione del Dead man's switch, includendo il titolo e il corpo dell'email di inattività, il primo e secondo timer di inattività, l'ultimo accesso dell'**Utente^G** e lo stato corrente del sistema (attivo, prima inattività, seconda inattività).
- **trusted_contact_mvp**: memorizza le informazioni relative ai contatti fidati associati a ciascun **Utente^G**, inclusi nome, email e numero di telefono.



Il sistema implementa inoltre un meccanismo di monitoraggio automatico basato su AWS EventBridge. Uno scheduler giornaliero attiva la Lambda centrale che verifica lo stato di attività degli utenti e determina eventuali condizioni di inattività prolungata.

Il flusso di elaborazione del Dead man's switch è il seguente:

- **Trigger programmato:** EventBridge attiva quotidianamente la Lambda, che avvia il controllo dello stato degli utenti.
- **Recupero configurazioni:** la Lambda effettua il fetch dei dati presenti in `dms_config_mvp`, ottenendo i timer di inattività, l'ultimo accesso registrato e lo stato corrente del Dead man's switch.
- **Valutazione dello stato di inattività:** la funzione confronta il tempo corrente con l'ultimo accesso dell'**Utente^G**. Se viene superata la prima soglia di inattività, il sistema entra nello stato di "prima inattività". Al raggiungimento della seconda soglia di inattività, viene attivato lo stato di "seconda inattività".
- **Invio email di prima inattività:** al superamento della prima soglia, la Lambda invia tramite Amazon SES un'email all'**Utente^G** utilizzando il titolo e il corpo configurati, notificando lo stato di inattività.
- **Attivazione SOS (seconda inattività):** al superamento della seconda soglia, la Lambda recupera l'elenco dei contatti fidati da `trusted_contact_mvp` e invia, tramite SES, un'email di SOS predefinita a ciascun contatto.
- **Aggiornamento stato:** lo stato del sistema viene aggiornato in DynamoDB per evitare invii ripetuti non necessari.

Oltre al flusso automatico, il sistema supporta l'invio manuale di SOS e la gestione in tempo reale dello stato di attività dell'**Utente^G** tramite heartbeat.

Il flusso di heartbeat è il seguente:

- **Invio heartbeat:** All'avvio dell'app, il client invia una richiesta all'API POST `/dms/heartbeat/` per notificare l'utilizzo attivo dell'applicazione.
- **Aggiornamento stato Backend^G:** la Lambda aggiorna il campo "ultimo accesso" nella tabella `dms_config_mvp` e, se necessario, resetta lo stato di inattività.

L'invio SOS **manuale** segue invece il seguente flusso:

- **Richiesta di emergenza:** il client invia una richiesta HTTPS all'API POST `/trusted_contacts/`
- **Recupero contatti:** la Lambda recupera tutti i contatti fidati associati all'**Utente^G** da `trusted_contact_mvp`.
- **Invio email SES:** la funzione Lambda invia immediatamente un'email di emergenza a tutti i contatti fidati tramite Amazon SES.



La gestione dei contatti fidati è completamente CRUD e permette all'**Utente^G** di mantenere aggiornato il proprio elenco di destinatari di emergenza.

API associate

- GET /dms/: restituisce tutte le opzioni configurabili del Dead man's switch, incluse soglie di inattività e template email.
- POST /dms/: salva o aggiorna la configurazione del Dead man's switch dell'**Utente^G**.
- POST /dms/heartbeat/: registra il segnale di attività dell'**Utente^G** aggiornando l'ultimo accesso.
- GET /trusted_contacts/: restituisce la lista dei contatti fidati associati all'**Utente^G**.
- POST /trusted_contacts/: crea un nuovo contatto fidato.
- PUT /trusted_contacts/{trusted_contact_id}/: aggiorna un contatto fidato esistente.
- DELETE /trusted_contacts/{trusted_contact_id}/: elimina un contatto fidato.
- POST /trusted_contacts/sos/: attiva manualmente l'invio di una email di emergenza a tutti i contatti fidati.

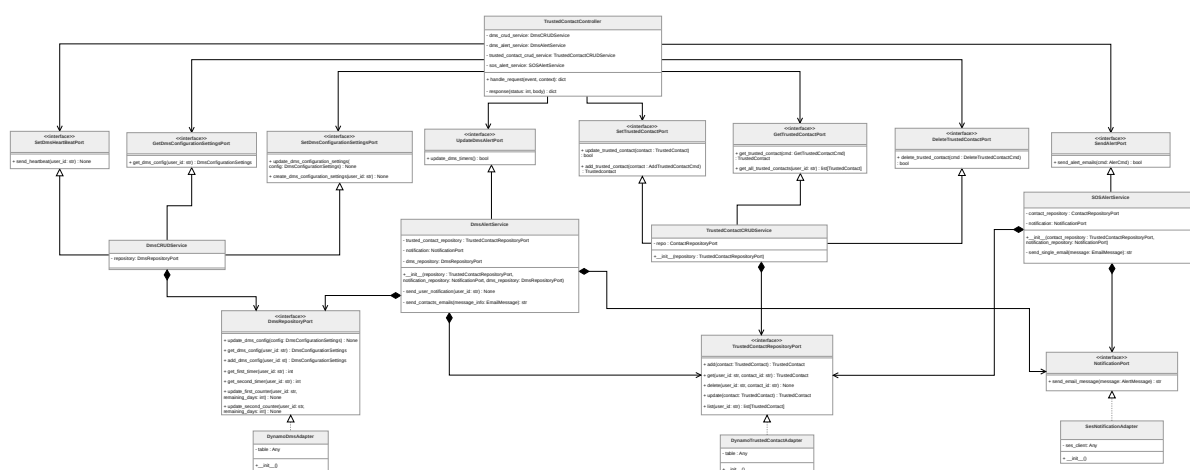


Figura 147: Componenti del microservizio contatti fidati, SOS e Dead man's switch

Architettura^G logica: Modulo contatti fidati, SOS e Dead man's switch



TrustedContact

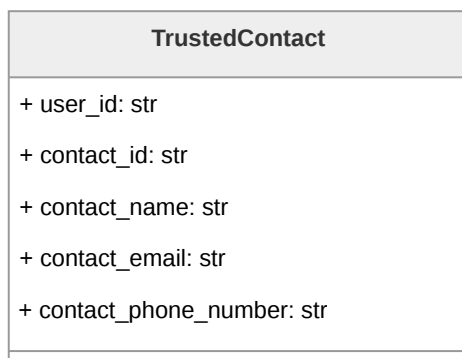


Figura 148: Diagramma della classe **TrustedContact**

Rappresenta un contatto fidato associato a un **Utente^G**.

Descrizione attributi:

- **user_id** identifica l'**Utente^G** proprietario del contatto;
- **contact_id** identifica univocamente il contatto;
- **contact_name**, **contact_email** e **contact_phone_number** ne rappresentano i dati anagrafici.

TrustedContactDTO

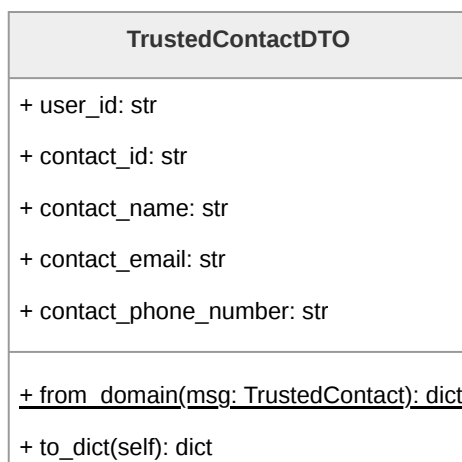


Figura 149: Diagramma della classe **TrustedContactDTO**



Rappresenta il Data Transfer Object utilizzato per la serializzazione e deserializzazione dei contatti fidati nelle comunicazioni tra frontend e **Backend^G**. Contiene tutti i campi dell'entità di dominio **TrustedContact** e fornisce i metodi **to_dict** per la conversione in dizionario Python (successivamente serializzato in JSON) e **from_domain** per la creazione a partire da un oggetto di dominio.

AddTrustedContactCmd

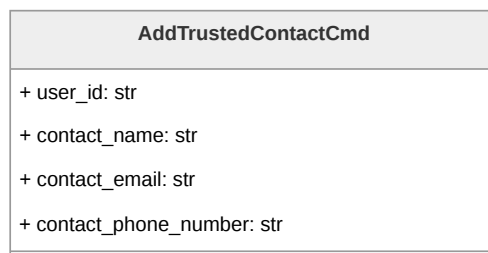


Figura 150: Diagramma della classe **AddTrustedContactCmd**

Command Object utilizzato dal controller per trasportare i dati necessari all'aggiunta di un contatto fidato.

GetTrustedContactCmd

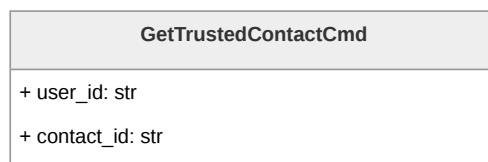


Figura 151: Diagramma della classe **GetTrustedContactCmd**

Command Object utilizzato dal controller per trasportare i dati necessari per il fetch di un contatto fidato.

DeleteTrustedContactCmd

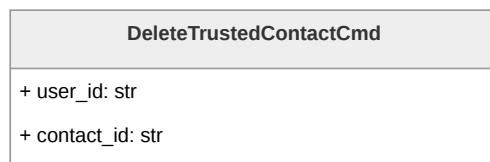


Figura 152: Diagramma della classe `DeleteTrustedContactCmd`

Command Object utilizzato dal controller per trasportare i dati necessari per la cancellazione di un contatto fidato.

DmsConfigurationSettings

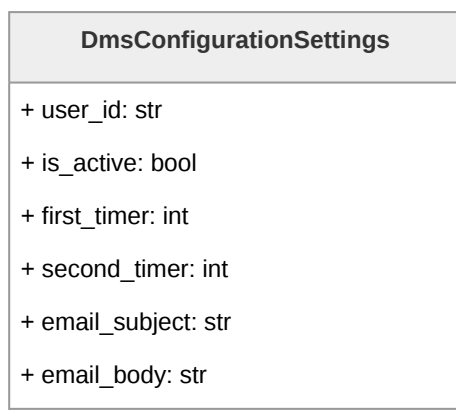


Figura 153: Diagramma della classe `DmsConfigurationSettings`

Rappresenta la configurazione del Dead Man's Switch associata ad un **Utente^G**.

Descrizione attributi:

- `user_id` identifica l'**Utente^G** a cui è associata la configurazione del dead man's switch;
- `is_active` indica se il dead man's switch è attivo;
- `first_timer` e `second_timer` definiscono la durata in giorni dei due timer;
- `email_body` contiene il corpo della mail riferita al dead man's switch;
- `email_subject` contiene l'oggetto della mail riferita al dead man's switch;



DmsConfigurationSettingsDTO

DmsConfigurationSettingsDTO
+ user_id: str + is_active: bool + first_timer: int + second_timer: int + email_subject: str + email_body: str
+ from_domain(msg: DmsConfigurationSettings): dict + to_dict(self): dict

Figura 154: Diagramma della classe **DmsConfigurationSettingsDTO**

Data Transfer Object utilizzato per la serializzazione e deserializzazione delle conversazioni nelle comunicazioni tra frontend e **Backend^G**. Contiene tutti i campi dell'entità di dominio **DmsConfigurationSettings** e fornisce i metodi **to_dict** per la conversione in dizionario Python (successivamente serializzato in JSON) e **from_domain** per la creazione a partire da un oggetto di dominio.

AlertCmd

AlertCmd
+ user_id: str + user_name: str + latitude: float None + longitude: float None

Figura 155: Diagramma della classe **AlertCmd**

Command Object utilizzato dal controller per trasportare i dati necessari all'invio dell'alert SOS.

Descrizione attributi:

- **user_id**, identificativo dell'**Utente^G**;
- **user_name**, nome dell'**Utente^G** che ha azionato l'alert;



- **latitude** e **longitude** indicano le coordinate geografiche in cui si trova l'Utente^G al momento dell'invio dell>alert.

EmailMessage

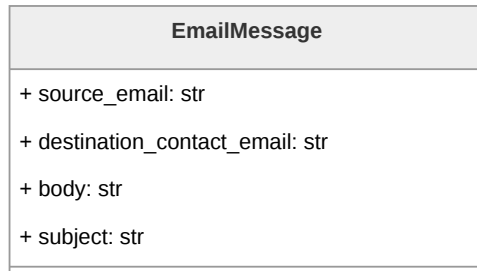


Figura 156: Diagramma della classe **EmailMessage**

Rappresenta la struttura delle email inviabili dal sistema.

Descrizione attributi:

- **source_email**, indirizzo con cui viene mandata l'email di alert;
- **destination_contact_email**, indirizzo email a cui viene inviato l>alert;
- **body**, corpo della mail di alert,
- **subject**, oggetto della mail di alert.

TrustedContactController

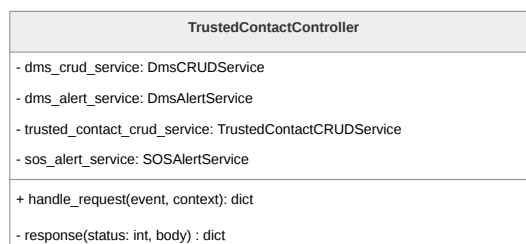


Figura 157: Diagramma della classe **TrustedContactController**

Riceve la richiesta grezza di API Gateway e la instrada verso la port primaria appropriata in base alla route.

Descrizione metodi:



- `handle_request(event, context)`, punto di ingresso del controller, legge la `routeKey` dall'evento e chiama la port primaria corrispondente;
- `response(status, body)`, metodo privato di utilità che costruisce la risposta nel formato atteso da API Gateway.

SetDmsHeartBeatPort

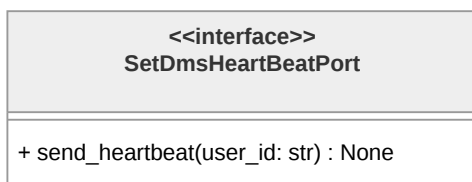


Figura 158: Diagramma della classe `SetDmsHeartBeatPort`

Espone `send_heartbeat(user_id: str)` per il reset dei timer del Dead Man's Switch al rientro dell'`UtenteG` nell'applicazione.

GetDmsConfigurationSettingsPort

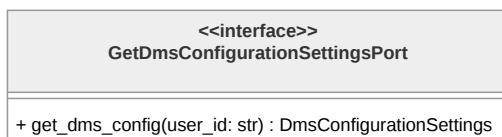


Figura 159: Diagramma della classe `GetDmsConfigurationSettingsPort`

Espone `get_dms_config(user_id: str)` per il recupero della configurazione del Dead Man's Switch.

SetDmsConfigurationSettingsPort

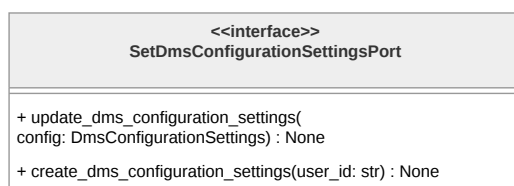


Figura 160: Diagramma della classe `SetDmsConfigurationSettingsPort`



Descrizione metodi:

- `update_dms_configuration_settings(config: DmsConfigurationSettings)`, per l'aggiornamento delle impostazione del dead man's switch;
- `create_dms_configuration_settings(user-id: str)`, per la creazione dei valori di default per le impostazione del dead man's switch.

UpdateDmsAlertPort

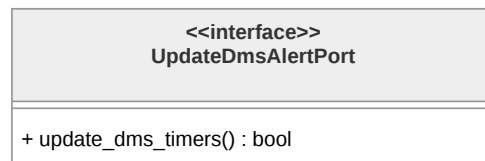


Figura 161: Diagramma della classe `UpdateDmsAlertPort`

Espone `update_dms_timers()` per l'aggiornamento dei timer lato **Backend**^G.

SetTrustedContactPort

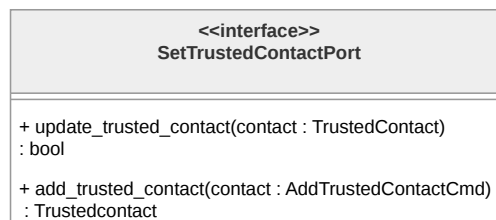


Figura 162: Diagramma della classe `SetTrustedContactPort`

Descrizione dei metodi:

- `add_trusted_contact(contact: AddTrustedContactCmd)` per la creazione di un nuovo contatto;
- `update_trusted_contact(request: TrustedContact)` per la modifica di uno esistente.

GetTrustedContactPort



Figura 163: Diagramma della classe **GetTrustedContactPort**

Descrizione dei metodi:

- **get_trusted_contact(cmd: GetTrustedContactCmd)** per il recupero di un singolo contatto;
- **get_all_trusted_contacts(user_id: str)** per il recupero della lista completa.

DeleteTrustedContactPort

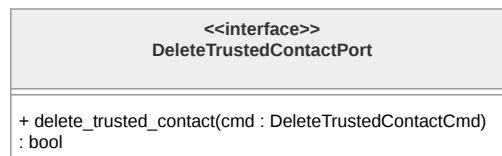


Figura 164: Diagramma della classe **DeleteTrustedContactPort**

Espone **delete_trusted_contact(cmd: DeleteTrustedContactCmd)** per l'eliminazione di un contatto.

SendAlertPort

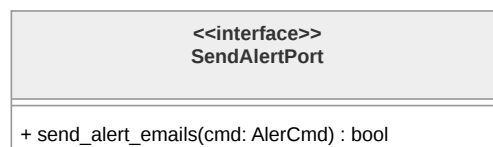


Figura 165: Diagramma della classe **SendAlertPort**

Espone **send_alert_emails(alert_info: AlertCmd)** per l'invio delle email di soccorso ai contatti fidati.



DmsCRUDService

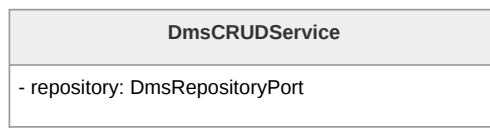


Figura 166: Diagramma della classe **DmsCRUDService**

È il service della **Business Logic^G** responsabile delle operazioni di lettura e scrittura sulla configurazione dei valori delle impostazioni del Dead Man's Switch.

DmsAlertService

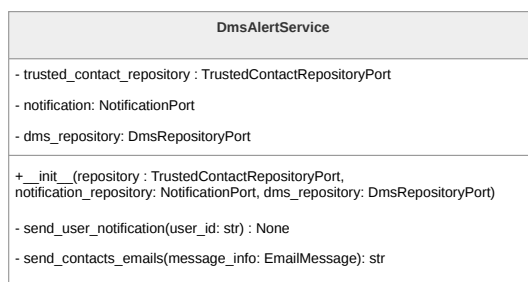


Figura 167: Diagramma della classe **DmsAlertService**

È responsabile dell'invio delle notifiche del Dead Man's Switch. Si occupa del recupero dei contatti fidati, la configurazione del DMS e la costruzione dei messaggi email.

Descrizione metodi:

- **__init__(trusted_contact_repository, notification_repository, dms_repository)**, costruttore che riceve le tre port secondarie tramite dependency injection;
- **send_user_notification(user_id: str)**, metodo privato che recupera la configurazione DMS dell'**Utente^G** e costruisce e invia la notifica all'**Utente^G** alla scadenza del primo timer;
- **send_contacts_emails(message_info: EmailMessage)**, metodo privato che costruisce e invia le email ai contatti fidati dell'**Utente^G** alla scadenza del secondo timer.

TrustedContactCRUDService

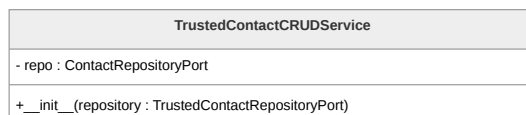


Figura 168: Diagramma della classe **TrustedContactCRUDService**

È il service della **Business Logic^G** responsabile delle operazioni CRUD sui contatti fidati. Descrizione metodi:

- **__init__(Repository^G: TrustedContactRepositoryPort)**, costruttore che riceve la port secondaria tramite dependency injection.

SOSAlertService

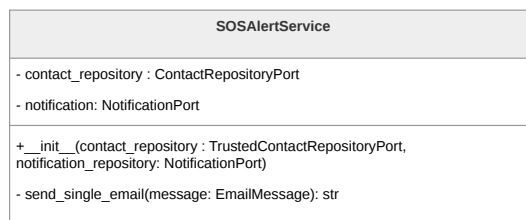


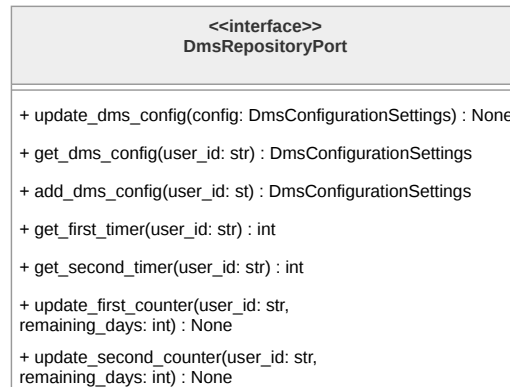
Figura 169: Diagramma della classe **SOSAlertService**

È il service della **Business Logic^G** responsabile dell'invio immediato delle email di soccorso al momento della pressione del pulsante SOS da parte dell'**Utente^G**. Implementa la port primaria **SendAlertPort**.

Descrizione metodi:

- **__init__(contact_repository, notification_repository)**, costruttore che riceve le due port secondarie tramite dependency injection;
- **send_single_email(message: EmailMessage)**, metodo privato che costruisce e invia una singola email a partire da un oggetto **EmailMessage**, delegando l'invio alla **NotificationPort**.

DmsRepositoryPort

Figura 170: Diagramma della classe **DmsRepositoryPort**

Descrizione metodi:

- **update_dms_config(config: DmsConfigurationSettings)**, aggiorna la configurazione del dead man's switch dell'**Utente^G**;
- **get_dms_config(user_id: str)**, recupera la configurazione del dead man's switch associata all'**Utente^G**;
- **add_dms_config(user_id: str)**, crea una nuova configurazione del dead man's switch per l'**Utente^G**;
- **get_first_timer(user_id: str)**, recupera il valore corrente del primo timer;
- **get_second_timer(user_id: str)**, recupera il valore corrente del secondo timer;
- **update_first_counter(user_id: str, remaining_days: int)**, aggiorna il contatore residuo del primo timer;
- **update_second_counter(user_id: str, remaining_days: int)**, aggiorna il contatore residuo del secondo timer.

TrustedContactRepositoryPort

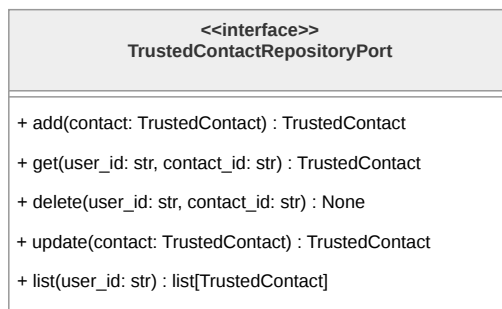


Figura 171: Diagramma della classe **TrustedContactRepositoryPort**

Descrizione metodi:

- **add(contact: TrustedContact)**, aggiunge un nuovo contatto fidato e restituisce l'entità creata;
- **get(user_id: str, contact_id: str)**, recupera un singolo contatto fidato tramite le chiavi identificative, restituendo **None** se non trovato;
- **delete(user_id: str, contact_id: str)**, elimina il contatto fidato corrispondente alle chiavi fornite;
- **update_trusted_contact(contact: TrustedContact)**, aggiorna i dati anagrafici di un contatto fidato esistente;
- **list(user_id: str)**, restituisce la lista completa dei contatti fidati associati all'**Utente^G**.

NotificationPort

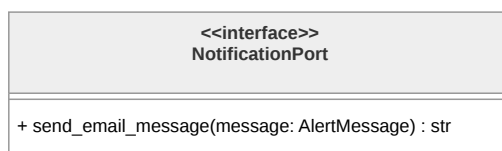


Figura 172: Diagramma della classe **NotificationPort**

È la port secondaria del pattern esagonale per l'invio delle notifiche email. Descrizione metodi:

- **send_email_message(message: EmailMessage)**, invia un messaggio email a partire da un oggetto **AlertMessage**.



DynamoDmsAdapter

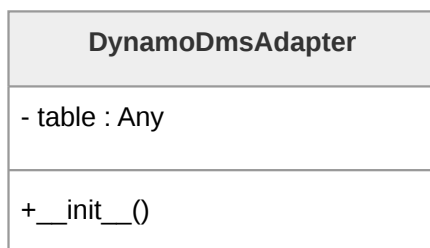


Figura 173: Diagramma della classe **DynamoDmsAdapter**

Contiene tutta la logica specifica per la persistenza della configurazione e dei contatori del Dead Man's Switch su DynamoDB.

DynamoTrustedContactAdapter

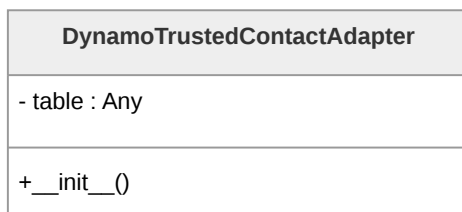


Figura 174: Diagramma della classe **DynamoTrustedContactAdapter**

Contiene tutta la logica specifica per la persistenza dei contatti fidati su DynamoDB.

SesNotificationAdapter

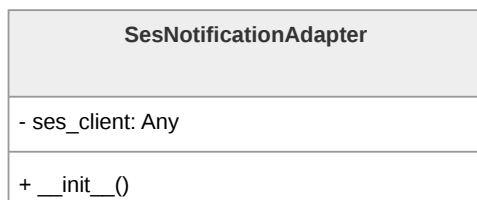


Figura 175: Diagramma della classe **SesNotificationAdapter**

Contiene tutta la logica specifica per l'invio delle email tramite il servizio Amazon SES.



4. Stato dei requisiti funzionali

4.1. Requisiti Funzionali obbligatori

Codice	Descrizione	Stato
RF-OB-1	L'Utente ^G deve poter autenticarsi all'applicazione tramite l'utilizzo di Google Auth	Non soddisfatto
RF-OB-2	Il sistema deve attivare automaticamente la funzionalità di Dead Man's Switch a seguito del login dell'Utente ^G	Non soddisfatto
RF-OB-3	L'Utente ^G deve poter visualizzare un messaggio di errore esplicativo nel caso in cui l'autenticazione tramite Google Auth fallisca, venga annullata o restituisca parametri non validi	Non soddisfatto
RF-OB-5	L'Utente ^G deve poter visualizzare la lista dei contatti fidati	Non soddisfatto
RF-OB-6	L'Utente ^G deve poter visualizzare un messaggio di errore esplicativo nel caso in cui il recupero dei dati dal database fallisca	Non soddisfatto
RF-OB-7	L'Utente ^G deve poter ricaricare la sezione dei contatti fidati dopo la visualizzazione dell'errore di caricamento dei dati dal database	Non soddisfatto
RF-OB-8	Il sistema deve permettere la visualizzazione di un singolo contatto fidato dalla lista dei contatti fidati	Non soddisfatto
RF-OB-9	L'Utente ^G deve poter visualizzare la preview delle informazioni (il nome) del contatto fidato che sta visualizzando dalla lista dei contatti fidati	Non soddisfatto
RF-OB-10	L'Utente ^G deve poter visualizzare i dettagli di un contatto fidato selezionato dalla lista dei contatti fidati	Non soddisfatto



Tabella 2: Tabella dello stato dei requisiti funzionali obbligatori (continua)

Codice	Descrizione	Stato
RF-OB-11	L'Utente ^G deve poter visualizzare il nome del contatto fidato selezionato dalla lista dei contatti fidati	Non soddisfatto
RF-OB-12	L'Utente ^G deve poter visualizzare il numero di telefono del contatto fidato selezionato dalla lista dei contatti fidati	Non soddisfatto
RF-OB-13	L'Utente ^G deve poter visualizzare l'indirizzo email del contatto fidato selezionato dalla lista dei contatti fidati	Non soddisfatto
RF-OB-14	L'Utente ^G deve poter aggiungere un nuovo contatto fidato alla lista dei contatti fidati	Non soddisfatto
RF-OB-15	L'Utente ^G deve poter inserire il nome del nuovo contatto fidato durante l'aggiunta di esso alla lista dei contatti fidati	Non soddisfatto
RF-OB-16	L'Utente ^G deve poter inserire il numero di telefono del nuovo contatto fidato durante l'aggiunta di esso alla lista dei contatti fidati	Non soddisfatto
RF-OB-17	L'Utente ^G deve poter inserire l'indirizzo email del nuovo contatto fidato durante l'aggiunta di esso alla lista dei contatti fidati	Non soddisfatto
RF-OB-18	Il sistema deve riconoscere e gestire le informazioni di numero di telefono e email già presenti nel sistema	Non soddisfatto
RF-OB-19	L'Utente ^G deve poter visualizzare un messaggio di errore esplicativo nel caso in cui le informazioni di numero di telefono e email siano già presenti nel sistema	Non soddisfatto
RF-OB-20	L'Utente ^G deve poter reinserire le informazioni di numero di telefono e email dopo la visualizzazione del messaggio di errore esplicativo	Non soddisfatto



Tabella 2: Tabella dello stato dei requisiti funzionali obbligatori (continua)

Codice	Descrizione	Stato
RF-OB-21	Il sistema deve validare la conformità del numero di telefono inserito o modificato	Non soddisfatto
RF-OB-22	Il sistema deve visualizzare un messaggio di errore esplicativo nel caso in cui il numero di telefono inserito o modificato non sia conforme ai criteri prestabiliti	Non soddisfatto
RF-OB-23	Il sistema deve validare la conformità dell'indirizzo email inserito o modificato	Non soddisfatto
RF-OB-24	Il sistema deve visualizzare un messaggio di errore esplicativo nel caso in cui l'indirizzo email inserito o modificato non sia conforme ai criteri prestabiliti	Non soddisfatto
RF-OB-25	L' Utente^G deve poter modificare i dati di un contatto fidato esistente	Non soddisfatto
RF-OB-26	L' Utente^G deve poter modificare il nome di un contatto fidato esistente	Non soddisfatto
RF-OB-27	L' Utente^G deve poter modificare il numero di telefono di un contatto fidato esistente	Non soddisfatto
RF-OB-28	L' Utente^G deve poter modificare l'indirizzo email di un contatto fidato esistente	Non soddisfatto
RF-OB-29	L' Utente^G deve poter eliminare un contatto fidato	Non soddisfatto
RF-OB-30	Il sistema deve richiedere conferma prima dell'eliminazione di un contatto fidato	Non soddisfatto
RF-OB-31	L' Utente^G deve poter confermare, con un'apposita voce, l'eliminazione di un contatto fidato quando il sistema richiede la conferma	Non soddisfatto
RF-OB-32	L' Utente^G deve poter effettuare il logout dopo essersi autenticato all'applicazione	Non soddisfatto



Tabella 2: Tabella dello stato dei requisiti funzionali obbligatori (continua)

Codice	Descrizione	Stato
RF-OB-34	L'Utente ^G deve poter modificare le informazioni della mail di avviso	Non soddisfatto
RF-OB-35	L'Utente ^G deve poter modificare il titolo della mail di avviso	Non soddisfatto
RF-OB-36	L'Utente ^G deve poter modificare il corpo della mail di avviso	Non soddisfatto
RF-OB-37	L'Utente ^G deve poter visualizzare un'anteprima della mail di avviso	Non soddisfatto
RF-OB-38	L'Utente ^G deve poter visualizzare l'anteprima del corpo della mail di avviso	Non soddisfatto
RF-OB-39	L'Utente ^G deve poter visualizzare un'anteprima del titolo della mail di avviso	Non soddisfatto
RF-OB-40	L'Utente ^G deve poter attivare il Dead Man's Switch	Non soddisfatto
RF-OB-41	L'Utente ^G deve poter disattivare la funzionalità di Dead Man's Switch	Non soddisfatto
RF-OB-42	L'Utente ^G deve poter impostare una password per il diario criptato	Non soddisfatto
RF-OB-43	Il sistema deve validare la conformità della password del diario criptato secondo i criteri pre-stabiliti	Non soddisfatto
RF-OB-44	Il sistema deve visualizzare un messaggio di errore esplicativo nel caso in cui la password del diario criptato non sia conforme ai criteri pre-stabiliti	Non soddisfatto
RF-OB-45	L'Utente ^G deve poter reinserire la password del diario criptato dopo la visualizzazione del messaggio di errore esplicativo	Non soddisfatto



Tabella 2: Tabella dello stato dei requisiti funzionali obbligatori (continua)

Codice	Descrizione	Stato
RF-OB-46	L'Utente ^G deve poter impostare una password per il diario fittizio	Non soddisfatto
RF-OB-47	Il sistema deve validare la conformità e l'unicità della password del diario fittizio rispetto a quella del diario criptato	Non soddisfatto
RF-OB-48	Il sistema deve visualizzare un messaggio di errore esplicativo nel caso in cui la password del diario fittizio non sia conforme ai criteri prestabiliti	Non soddisfatto
RF-OB-49	Il sistema deve visualizzare un messaggio di errore esplicativo nel caso in cui la password del diario fittizio sia uguale a quella del diario criptato	Non soddisfatto
RF-OB-50	L'Utente ^G deve poter reinserire la password del diario fittizio dopo la visualizzazione del messaggio di errore esplicativo	Non soddisfatto
RF-OB-51	L'Utente ^G deve poter inserire la password per accedere al diario desiderato	Non soddisfatto
RF-OB-52	L'Utente ^G deve poter inserire la password per accedere al diario criptato	Non soddisfatto
RF-OB-53	L'Utente ^G deve poter inserire la password per accedere al diario fittizio	Non soddisfatto
RF-OB-54	Il sistema deve visualizzare un messaggio di errore esplicativo nel caso in cui la password inserita per l'accesso al diario risulti errata	Non soddisfatto
RF-OB-55	L'Utente ^G deve poter reinserire la password di accesso al diario dopo la visualizzazione del messaggio di errore esplicativo	Non soddisfatto



Tabella 2: Tabella dello stato dei requisiti funzionali obbligatori (continua)

Codice	Descrizione	Stato
RF-OB-57	L'Utente ^G deve poter visualizzare il contenuto del diario criptato	Non soddisfatto
RF-OB-58	L'Utente ^G deve poter visualizzare la lista delle note presenti nel diario criptato	Non soddisfatto
RF-OB-59	L'Utente ^G deve poter visualizzare l'anteprima della singola nota all'interno della lista delle note del diario criptato	Non soddisfatto
RF-OB-60	L'Utente ^G deve poter ricaricare la sezione delle note del diario criptato dopo la visualizzazione dell'errore di caricamento dei dati dal database	Non soddisfatto
RF-OB-61	L'Utente ^G deve poter creare una nuova nota vuota all'interno del diario criptato	Non soddisfatto
RF-OB-62	L'Utente ^G deve poter eliminare una nota esistente dal diario criptato	Non soddisfatto
RF-OB-63	Il sistema deve richiedere conferma prima dell'eliminazione definitiva di una nota del diario criptato	Non soddisfatto
RF-OB-64	L'Utente ^G deve poter confermare, con un'apposita voce, l'eliminazione della nota dal diario criptato quando il sistema ne richiede la conferma	Non soddisfatto
RF-OB-65	L'Utente ^G deve poter visualizzare il contenuto completo di una nota selezionata dalla lista delle note del diario criptato	Non soddisfatto
RF-OB-66	L'Utente ^G deve poter visualizzare il contenuto testuale della nota del diario criptato	Non soddisfatto
RF-OB-67	L'Utente ^G deve poter visualizzare il contenuto multimediale della nota del diario criptato	Non soddisfatto



Tabella 2: Tabella dello stato dei requisiti funzionali obbligatori (continua)

Codice	Descrizione	Stato
RF-OB-68	L'Utente ^G deve poter visualizzare il contenuto audio presente all'interno della nota del diario criptato	Non soddisfatto
RF-OB-69	L'Utente ^G deve poter visualizzare le immagini presenti all'interno della nota del diario criptato	Non soddisfatto
RF-OB-70	L'Utente ^G deve poter inserire del contenuto testuale all'interno di una nota del diario criptato	Non soddisfatto
RF-OB-71	L'Utente ^G deve poter inserire del contenuto multimediale all'interno di una nota del diario criptato	Non soddisfatto
RF-OB-72	L'Utente ^G deve poter inserire un'immagine all'interno di una nota del diario criptato	Non soddisfatto
RF-OB-73	L'Utente ^G deve poter inserire una traccia audio all'interno di una nota del diario criptato	Non soddisfatto
RF-OB-74	L'Utente ^G deve poter modificare il contenuto testuale presente all'interno di una nota del diario criptato	Non soddisfatto
RF-OB-75	L'Utente ^G deve poter rimuovere del contenuto multimediale presente all'interno di una nota del diario criptato	Non soddisfatto
RF-OB-76	L'Utente ^G deve poter rimuovere un'immagine presente all'interno di una nota del diario criptato	Non soddisfatto
RF-OB-77	L'Utente ^G deve poter rimuovere una traccia audio presente all'interno di una nota del diario criptato	Non soddisfatto
RF-OB-78	L'Utente ^G deve poter visualizzare la schermata del diario fittizio	Non soddisfatto



Tabella 2: Tabella dello stato dei requisiti funzionali obbligatori (continua)

Codice	Descrizione	Stato
RF-OB-79	L'Utente ^G deve poter visualizzare la lista delle note presenti nel diario fittizio	Non soddisfatto
RF-OB-80	L'Utente ^G deve poter visualizzare l'anteprima della singola nota all'interno della lista delle note del diario fittizio	Non soddisfatto
RF-OB-81	L'Utente ^G deve poter ricaricare la sezione delle note del diario fittizio dopo la visualizzazione dell'errore di caricamento dei dati dal database	Non soddisfatto
RF-OB-82	L'Utente ^G deve poter eliminare una nota esistente dal diario fittizio	Non soddisfatto
RF-OB-83	Il sistema deve richiedere conferma prima dell'eliminazione definitiva di una nota del diario fittizio	Non soddisfatto
RF-OB-84	L'Utente ^G deve poter confermare, con un'apposita voce, l'eliminazione della nota dal diario fittizio quando il sistema ne richiede la conferma	Non soddisfatto
RF-OB-85	L'Utente ^G deve poter creare una nuova nota vuota all'interno del diario fittizio	Non soddisfatto
RF-OB-86	L'Utente ^G deve poter visualizzare il contenuto completo di una nota selezionata dalla lista delle note del diario fittizio	Non soddisfatto
RF-OB-87	L'Utente ^G deve poter visualizzare il contenuto testuale della nota del diario fittizio	Non soddisfatto
RF-OB-88	L'Utente ^G deve poter visualizzare il contenuto multimediale della nota del diario fittizio	Non soddisfatto
RF-OB-89	L'Utente ^G deve poter visualizzare il contenuto audio presente all'interno della nota del diario fittizio	Non soddisfatto



Tabella 2: Tabella dello stato dei requisiti funzionali obbligatori (continua)

Codice	Descrizione	Stato
RF-OB-90	L' Utente^G deve poter visualizzare le immagini presenti all'interno della nota del diario fittizio	Non soddisfatto
RF-OB-91	L' Utente^G deve poter inserire del contenuto testuale all'interno di una nota del diario fittizio	Non soddisfatto
RF-OB-92	L' Utente^G deve poter inserire del contenuto multimediale all'interno di una nota del diario fittizio	Non soddisfatto
RF-OB-93	L' Utente^G deve poter inserire un'immagine all'interno di una nota del diario fittizio	Non soddisfatto
RF-OB-94	L' Utente^G deve poter inserire una traccia audio all'interno di una nota del diario fittizio	Non soddisfatto
RF-OB-95	L' Utente^G deve poter rimuovere del contenuto multimediale presente all'interno di una nota del diario fittizio	Non soddisfatto
RF-OB-96	L' Utente^G deve poter rimuovere un'immagine presente all'interno di una nota del diario fittizio	Non soddisfatto
RF-OB-97	L' Utente^G deve poter rimuovere una traccia audio presente all'interno di una nota del diario fittizio	Non soddisfatto
RF-OB-98	L' Utente^G deve poter modificare il contenuto testuale presente all'interno di una nota del diario fittizio	Non soddisfatto
RF-OB-99	L' Utente^G deve poter inviare un messaggio di SOS ai contatti fidati tramite un'apposita voce	Non soddisfatto
RF-OB-100	Il sistema deve notificare l' Utente^G dell'assenza di connessione internet durante il tentativo di invio del messaggio SOS e proporre l'invio di un segnale acustico	Non soddisfatto



Tabella 2: Tabella dello stato dei requisiti funzionali obbligatori (continua)

Codice	Descrizione	Stato
RF-OB-101	Il sistema deve notificare l' Utente^G della mancanza di contatti fidati configurati durante il tentativo di invio del messaggio SOS e proporre l'invio di un segnale acustico	Non soddisfatto
RF-OB-102	L' Utente^G deve poter avviare un segnale acustico di emergenza indipendente dalle impostazioni di volume del dispositivo	Non soddisfatto
RF-OB-103	L' Utente^G deve poter inviare un messaggio al chatbot e ricevere una risposta generata dall'LLM in base al contesto scelto	Non soddisfatto
RF-OB-105	Il sistema deve notificare l' Utente^G della mancanza di connessione internet durante la consultazione del chatbot	Non soddisfatto
RF-OB-106	L' Utente^G deve poter ripetere l'invio del messaggio o scriverne uno nuovo dopo la notifica di assenza di connessione internet	Non soddisfatto
RF-OB-107	L' Utente^G deve poter cambiare il contesto della chat a "Detective delle Relazioni"	Non soddisfatto
RF-OB-108	L' Utente^G deve poter cambiare il contesto della chat a "Specchio Intelligente"	Non soddisfatto
RF-OB-111	L' Utente^G deve poter creare una nuova sessione di chat con il chatbot	Non soddisfatto
RF-OB-112	L' Utente^G deve poter eliminare una chat dalla cronologia delle chat	Non soddisfatto
RF-OB-113	Il sistema deve richiedere conferma esplicita prima dell'eliminazione definitiva di una chat	Non soddisfatto
RF-OB-114	L' Utente^G deve poter confermare, con un'apposita voce, l'eliminazione di una chat quando il sistema ne richiede la conferma	Non soddisfatto



Tabella 2: Tabella dello stato dei requisiti funzionali obbligatori (continua)

Codice	Descrizione	Stato
RF-OB-115	L' Utente^G deve poter visualizzare il contenuto di una chat selezionata, inclusa la lista dei messaggi scambiati	Non soddisfatto
RF-OB-116	Il sistema deve notificare l' Utente^G di un errore nel caricamento del contenuto della chat	Non soddisfatto
RF-OB-117	Il sistema deve visualizzare la lista dei messaggi scambiati tra l' Utente^G e il chatbot all'interno di una chat	Non soddisfatto
RF-OB-118	Il sistema deve distinguere visivamente i messaggi inviati dall' Utente^G da quelli generati dal chatbot	Non soddisfatto
RF-OB-119	Il sistema deve visualizzare i messaggi inviati dall' Utente^G con la formattazione grafica allineata a destra	Non soddisfatto
RF-OB-120	Il sistema deve visualizzare i messaggi generati dal chatbot con la formattazione grafica allineata a sinistra	Non soddisfatto
RF-OB-121	L' Utente^G deve poter visualizzare la cronologia delle chat con il chatbot	Non soddisfatto
RF-OB-122	L' Utente^G deve poter selezionare e visualizzare una singola chat dalla cronologia	Non soddisfatto
RF-OB-123	Il sistema deve mostrare il titolo identificativo di ogni chat presente nella cronologia	Non soddisfatto
RF-OB-124	L' Utente^G deve poter visualizzare la lista delle community disponibili	Non soddisfatto
RF-OB-125	Il sistema deve notificare l' Utente^G di un errore nel caricamento della lista delle community	Non soddisfatto
RF-OB-126	Il sistema deve mostrare i dati riassuntivi di ogni singola community nella lista	Non soddisfatto



Tabella 2: Tabella dello stato dei requisiti funzionali obbligatori (continua)

Codice	Descrizione	Stato
RF-OB-127	Il sistema deve mostrare il nome di ogni singola community nella lista	Non soddisfatto
RF-OB-128	Il sistema deve mostrare il link di ogni singola community nella lista	Non soddisfatto
RF-OB-129	L'Utente ^G deve poter aprire il link di una community per visitarne il sito web esterno	Non soddisfatto
RF-OB-130	L'Utente ^G deve poter visualizzare la lista delle leggi e normative disponibili	Non soddisfatto
RF-OB-131	Il sistema deve mostrare il titolo di ogni singola legge nella lista	Non soddisfatto
RF-OB-132	L'Utente ^G deve poter visualizzare i dettagli completi di una legge selezionata dalla lista	Non soddisfatto
RF-OB-133	Il sistema deve notificare l'Utente ^G di un errore nel caricamento dei dati di una legge	Non soddisfatto
RF-OB-134	L'Utente ^G deve poter visualizzare la lista dei contatti e degli enti d'emergenza	Non soddisfatto
RF-OB-135	Il sistema deve mostrare i dati riassuntivi di ogni singolo contatto d'emergenza nella lista	Non soddisfatto
RF-OB-136	Il sistema deve mostrare il nome identificativo di ogni contatto d'emergenza	Non soddisfatto
RF-OB-137	Il sistema deve mostrare il numero di telefono associato ad ogni contatto d'emergenza	Non soddisfatto
RF-OB-139	L'Utente ^G deve poter visualizzare la lista del materiale informativo disponibile	Non soddisfatto
RF-OB-140	Il sistema deve notificare l'Utente ^G di un errore nel caricamento della lista del materiale informativo	Non soddisfatto



Tabella 2: Tabella dello stato dei requisiti funzionali obbligatori (continua)

Codice	Descrizione	Stato
RF-OB-141	Il sistema deve mostrare il titolo e il formato di ogni singolo materiale informativo nella lista	Non soddisfatto
RF-OB-142	L'Utente ^G deve poter visualizzare il contenuto di un materiale informativo selezionato dalla lista	Non soddisfatto
RF-OB-143	Il sistema deve notificare l'Utente ^G di un errore nel recupero dei dati del materiale informativo	Non soddisfatto
RF-OB-144	L'Utente ^G deve poter visualizzare la mappa interattiva con i marker dei luoghi sicuri nelle vicinanze	Non soddisfatto
RF-OB-145	L'Utente ^G deve poter aprire un luogo sicuro nella mappa esterna Google Maps	Non soddisfatto
RF-OB-146	Il sistema deve notificare l'Utente ^G di un errore nell'apertura dell'applicazione Google Maps e mantenere la visualizzazione della mappa interna	Non soddisfatto
Tabella 2: Tabella dello stato dei requisiti funzionali obbligatori		

4.2. Requisiti Funzionali desiderabili

Codice	Descrizione	Stato
RF-DE-1	Il sistema deve mostrare all'Utente ^G una spiegazione della differenza tra "per me" e "per altri"	Non Soddisfatto



Tabella 3: Tabella Requisiti Funzionali desiderabili (continua)

Codice	Descrizione	Stato
RF-DE-2	Il sistema mostra all'Utente ^G una breve descrizione del funzionamento del Dead Man's Switch	Non Soddisfatto
RF-DE-3	Il sistema deve permettere all'Utente ^G di inserire più contenuti multimediali con una singola azione nel diario criptato	Non Soddisfatto
RF-DE-4	Il sistema deve permettere all'Utente ^G di inserire più immagini con una singola azione nel diario criptato	Non Soddisfatto
RF-DE-5	Il sistema deve permettere all'Utente ^G di inserire più tracce audio con una singola azione nel diario criptato	Non Soddisfatto
RF-DE-6	Il sistema deve permettere all'Utente ^G di inserire più contenuti multimediali con una singola azione nel diario fittizio	Non Soddisfatto
RF-DE-7	Il sistema deve permettere all'Utente ^G di inserire più immagini con una singola azione nel diario fittizio	Non Soddisfatto
RF-DE-8	Il sistema deve permettere all'Utente ^G di inserire più tracce audio con una singola azione nel diario fittizio	Non Soddisfatto
RF-DE-9	Il sistema deve permettere all'Utente ^G di riprodurre le tracce audio inserite nel diario criptato	Non Soddisfatto
RF-DE-10	Il sistema deve permettere all'Utente ^G di riprodurre le tracce audio inserite nel diario fittizio	Non Soddisfatto
RF-DE-11	Il sistema deve indicare all'Utente ^G che il messaggio del chatbot è in fase di caricamento	Non Soddisfatto



Tabella 3: Tabella Requisiti Funzionali desiderabili (continua)

Codice	Descrizione	Stato
RF-DE-12	Il sistema deve utilizzare i contenuti del diario criptato per personalizzare le risposte del chat-bot	Non Soddisfatto
RF-DE-13	Il sistema deve mostrare all'Utente ^G una spiegazione del tipo di contesto a cui può passare (Detective delle relazioni)	Non Soddisfatto
RF-DE-14	Il sistema deve mostrare all'Utente ^G una spiegazione del tipo di contesto a cui può passare (Specchio intelligente)	Non Soddisfatto
RF-DE-15	L'Utente ^G deve poter comporre automaticamente il numero di un contatto d'emergenza tramite l'apposita voce	Non Soddisfatto

Tabella 3: Tabella Requisiti Funzionali Desiderabili

4.3. Requisiti Funzionali opzionali

Codice	Descrizione	Riferimento
RF-OP-1	L'Utente ^G deve poter impostare il timer di prima inattività (per la funzionalità di Dead Man's Switch)	Non Soddisfatto
RF-OP-2	L'Utente ^G deve poter impostare il timer di seconda inattività (per la funzionalità di Dead Man's Switch)	Non Soddisfatto
RF-OP-3	Il sistema deve inviare un SMS ai numeri di telefono dei contatti fidati all'invio del segnale SOS	Non Soddisfatto
RF-OP-4	Il sistema deve indicare all'Utente ^G se le risorse sono autorevoli e verificate	Non Soddisfatto



Tabella 4: Tabella dello stato dei requisiti funzionali opzionali (continua)

Codice	Descrizione	Riferimento
RF-OP-5	Il sistema deve mettere a disposizione leggi estere	Non Soddisfatto
RF-OP-6	Il sistema deve mostrare leggi aggiornate, giuridicamente valide e complete	Non Soddisfatto
RF-OP-7	Il sistema deve indicare all' Utente^G se i luoghi sicuri visualizzati nella mappa sono autorevoli e verificati	Non Soddisfatto
RF-OP-8	L' Utente^G deve poter selezionare il profilo di utilizzo ("Per me" o "Per altri")	Non soddisfatto
RF-OP-9	Il sistema deve bloccare temporaneamente l'accesso al diario in seguito al superamento del limite massimo di tentativi errati di inserimento password	Non soddisfatto
RF-OP-10	L' Utente^G deve poter interrompere la generazione della risposta dell'LLM durante la consultazione del chatbot	Non soddisfatto
RF-OP-11	Il sistema deve notificare l' Utente^G di un errore durante il cambio di contesto del chatbot e mantenere il contesto precedente	Non soddisfatto
RF-OP-12	L' Utente^G deve poter ritentare il cambio di contesto del chatbot dopo la notifica di errore	Non soddisfatto

Tabella 4: Tabella dello stato dei requisiti funzionali opzionali